Testing Permission-Aware Retrieval for RAG across Vector Stores and Authorization Engines

Syed Muzakkir Hussain
muzakkirhussain011@gmail.com

Abstract

Retrieval-augmented generation (RAG) puts the chunks most similar to a question into a model's prompt. Similarity ignores authorization, so a shared index can hand a subject a chunk it may not read. We measure one layer, the author's own, created in October 2026 and run at several versions: it compiles authorization policies into filters for vector stores, re-checks every hit and fails closed. Permissions are synthetic, no adaptive adversary ran and no other layer was compared. Differential testing of 13 stores found no hit outside the reference on its generated data; separate reproducers show two undocumented truncations (4,096 bytes on Redis, 8,191 characters on MongoDB) that also let a value match by its prefix. On one generated model per engine profile, filters and checks disagreed at one revision only on one translator's type gap. On one synthetic corpus with tenant copies, mean recall@10 at mixed settings ranged from 0.54 to 0.98 over eight approximate stores, and on pgvector people with readable chunks got empty answers in four of eight runs, which the layer reported as complete. A side run on another corpus gave row-level security recall@10 of at least 0.998, at median latencies up to 608 ms. In eleven end-to-end configurations, four runs each with static permissions, no returned chunk lay outside the readable set: with the 3,746 distinct (person, question) pairs taken as independent, a one-sided 95% bound of 8.0×10^{-4} per question, and 1.6×10^{-2} per person, though the 189 people are 13 role types. The re-check dropped nothing in those runs; end to end, only a one-seed mutation control exercised it. In forced-choice tests latency, counts and recall revealed unreadable chunks near a query, and scores a group of them in the collection; the evaluated code served revoked chunks up to 38.4 s after invalidation.

1 Introduction

A retrieval-augmented generation (RAG) assistant answers from the chunks of an index that are most similar to the question [77]. In an organization those chunks come from documents with different readers: a payroll file, a board deck, one customer's contract and a public handbook can share one index. Similarity knows nothing of who may read what, so without enforcement the chunk most similar to an employee's question may be one that employee may not open, and the model will repeat it. OWASP lists this as LLM08, vector and embedding weaknesses [116], and revoked documents have stayed retrievable through a cached index [139].

The remedy is old. Query modification adds the access predicate to the query before it runs [150]; multi-user full-text search showed that statistics computed over unreadable documents reveal them [23], and later attacks turned that observation, and timing, into term and plaintext recovery against shared search indexes and row-level security [49, 165]; enterprise search checked results late, in growing blocks [18]; a search over content that Zanzibar protects checks its candidate results against a central authorization service [118]. Managed RAG services combine these steps [3], and recent work builds permissions into the vector index itself [63, 189].

Three things stand between an organization's policy and the chunks a model sees. *First, the policy is compiled by translators.* Filter APIs are code paths separate from check APIs, every store has its own filter language, and we found no published guarantee that the paths agree; they have diverged, as when SpiceDB's LookupResources missed results its other APIs computed correctly [102]. *Second, the filter runs inside an approximate index.* Recall in filtered nearest-neighbor search depends on a filter's selectivity and its correlation with the question, though neither alone predicts it [81, 120]; a permission filter can be selective, takes a disjunction over many groups, and admits chunks by organizational structure, not meaning. *Third, failures are silent.* Under Truman semantics [137] a subject who receives three chunks, or none, cannot tell whether that is all it may read or all that the index found.

This paper is a measurement study of one enforcement layer, WARDOWL, an open-source library that the author develops and whose public repository was created in October 2026. It compiles an organization's policy into filters for 14 vector stores, from ACL metadata or from four authorization engines, checks hits through those and three more engines, re-checks every hit and fails closed (Section 3). The results describe this layer at the versions we state, on synthetic permission layouts. We ran no adaptive adversary and compare no other layer; our one native baseline is a side run on another corpus (Section 6.4). Under stated assumptions, the re-check rests membership in the readable set on a trusted computing base around the check API (Observation 1), whose size we did not measure: a faulty filter or a lossy index can then cost recall but cannot put a chunk outside the readable set into an answer. We ask how much is lost, where, how fresh the check is, what the answers still reveal, and whether anyone can tell.

We found the following. Outside positive controls built to leak, in what we tested no returned chunk lay outside the subject's readable set at the revision the re-check read; we call such a chunk a *leak*. In the end-to-end runs the re-check dropped nothing, so those runs would look the same with it off; only a one-seed mutation control, and a data-shape profile of one translator, exercised it. The revision can be stale: revoked chunks were served for up to 38.4 s after an invalidation, past the stated bound. Other observables of an answer revealed information about unreadable chunks. And on our layouts retrieval quality ranged from near perfect to missing most of the exact top ten for some people, unseen by the application.

Contributions.

- *A formulation* (Section 2). We adapt the sound and maximal criteria of Wang et al. [167] to top-k retrieval under an external authorizer, replace their indistinguishability notion of security with a weaker property of the returned set, add freshness, with a measured staleness δ per path, and count-honesty for the claim that an answer is complete, and state the trusted computing base.

- *Testing of the enforcement path* (Sections 4, 5 and 9). Differential testing across 13 of the 14 stores found no hit outside the reference on its generated data and no altered metadata; two truncations, which at long values lose matching chunks and, as separate reproducers show, also admit a longer value by its prefix; two availability faults in the layer's adapters; and limits that fail closed, not all of them documented. Only four of the seven engines have filters to compare. At one revision, over 6.3 million user-document pairs from one generated model per profile, filters and checks never disagreed in the profiles that hold the model as intended. No chunk outside the readable set was returned in 1,050 fault episodes (a further 40 control episodes turned denials into allows and leaked, as they must) or in four runs of each of eleven end-to-end configurations (8.0×10^{-4} per question, 1.6×10^{-2} per person and 0.21 per role type, one-sided 95%, under independence assumptions Section 9 qualifies). A mutation control

on one seed and an exact store shows that the leak checks fail when the filter is broken on purpose and that the re-check contains that fault.

- *An audit, diagnosis and repair of retrieval quality* (Sections 6 and 7). Mean recall@10 ranged from 0.55 to 0.98 over eight stores searched approximately, on one synthetic corpus with 19 near-identical copies of each company document, under this layer's schema and at settings some of them tuned. A diagnosis grid with another question mix gave Qdrant 0.656 in the shared layout and 0.952 with per-tenant graphs, consistent with an islanded permitted subgraph that we did not measure directly; plans recorded under an exploratory hypothesis tie pgvector's two modes to PostgreSQL's plan cache. A count flagged all 2,657 truly short answers without a false alarm, which on unchanging data follows from what a count is, but no count sees a full answer of the wrong chunks.

- *Side channels of filtered retrieval* (Section 8). In a pre-registered forced-choice game, shown one query's observation in two worlds and told that in one the query's m nearest non-hub passages are unreadable chunks (the worlds differ in 1,233 such chunks, the others beyond the post-filter's cap of 1,000 candidates for that query), a subject picked that world from the widening post-filter's latency with paired AUC 0.66 to 0.98 at $m = 100$ and 1.000 at $m = 1{,}000$ when it could read 10% or 50% of the background; reading 1%, it had no forward latency channel (0.03 to 0.59) but a count channel at $m = 1{,}000$ (1.000). BM25 and IDF scores gave 0.98 to 1.00 at those m, in worlds that differ by 1,233 chunks whatever m, a known channel that prior work takes to term and text recovery [49, 165]. Of the game's hypotheses H1 and H2 were falsified, and H3b, H4, H5a and H6 are inconclusive as printed.

We do not propose an index, and we do not defend against prompt injection in readable content. The library, the conformance kit, the pre-registrations and raw outputs, with the gaps that Section 11 lists, are released at `https://github.com/muzakkirhussain011/wardowl-paper-artifact`. Appendix A maps every experiment to its versions and runs, and Appendix B lists every hypothesis with its registration and verdict; labels such as P5 below refer to that ledger.

2 Problem Statement and Threat Model

2.1 Permission-aware top-k retrieval

Let D be a corpus of chunks, each with an embedding $\mathbf{x}_d$ and metadata m_d, and let $\sigma(q, d)$ be the similarity between a query embedding q and $\mathbf{x}_d$. For $X \subseteq D$, $\mathrm{top}_k(q, X)$ denotes the $\min(k, |X|)$ chunks of X most similar to q, ties broken by a fixed order on chunk identifiers. Vector stores answer this query approximately, with indexes such as HNSW [86], DiskANN [151] or IVF [43]; a filtered query $\mathrm{search}(q, \varphi, k)$ restricts the answer to $E(\varphi) = \{d \in D : m_d \models \varphi\}$, filtering before, after or during index traversal [31, 120].

A *subject* s is the principal on whose behalf a query runs; an agent acting for a user is authorized as that user [70]. Given the authorization data at revision τ, committed at time $t(\tau)$, the *policy* decides $P_\tau(s, d) \in \{0, 1\}$, whether s may read chunk d. The *readable set* is $R_\tau(s) = \{d \in D : P_\tau(s, d) = 1\}$, written $R(s)$ when the revision is clear, and the *exact answer* is $T^\star = \mathrm{top}_k(q, R_\tau(s))$. Our experiments compute P with an oracle, a separate plain implementation of the same predicate that does not call the layer. An external *authorizer* enforces the policy: a relationship-based engine in the style of Zanzibar [118], a policy engine, or ACL metadata copied from a source system. Its *check API* answers $\mathrm{check}_\tau(s, d) \in \{0, 1\}$, reading revision τ. Many authorizers also expose a *filter API*, such as

OpenFGA's ListObjects, SpiceDB's LookupResources, OPA's partial evaluation or Cerbos's query plans [13, 28, 104, 108], whose output can be translated into a store predicate φ_s.

An *enforcement layer* sits between the RAG application and the store. For each request it (i) takes s from a verified token; (ii) obtains φ_s when a filter can be compiled; (iii) searches the store with φ_s for at least k candidates, or without a filter when none can be compiled; (iv) re-checks every candidate with check_τ, dropping those that fail and counting them as *violations*; (v) widens the search until k candidates survive or a cap is reached; and (vi) fails closed, returning nothing, when the authorizer cannot answer. It returns a ranked list H of at most k chunks and a flag $c \in \{0, 1\}$ claiming that H is complete.

2.2 Correctness criteria

Wang et al. judge fine-grained access control in relational databases by three criteria: an answer is *sound* if it is consistent with the answer computed without access control, *secure* if it does not depend on information the policy withholds, so that database states the policy cannot tell apart give the same answer, and *maximum* (our *maximal*) if it returns as much as the first two allow [167]. We adapt soundness and maximality to top-k retrieval under Truman semantics, in which a query is answered over the subject's restricted view rather than rejected [137], so the reference answer is the exact answer over the readable set. We do not adapt their security. What we call *secure* below is much weaker: it asks only that each returned chunk be readable, and an empty answer satisfies it. Their notion fails wherever an answer changes with unreadable content, as Section 8 shows it does.

Definition 1 (Criteria for permission-aware top-k retrieval)**.** *Fix a subject s, a query q and a depth k, for a request issued at time t. Each hit d is checked at some revision τ_d (different hits may be checked at different revisions, for example when some decisions come from a cache), and τ is the latest of these revisions, or, when no hit was checked, the latest revision committed before t. φ_s is the predicate the store searched under (the constant true for an unfiltered search), $\sigma_{(k)}$ the k-th highest similarity in $R_\tau(s)$ (the lowest if $|R_\tau(s)| < k$), and $T^\star_\epsilon = \{d \in R_\tau(s) : \sigma(q, d) \geq \sigma_{(k)} - \epsilon\}$ the chunks that belong to the exact answer up to ties within ϵ. A result (H, c) is*

- secure *if $d \in R_{\tau_d}(s)$ for every $d \in H$: no returned chunk lies outside the readable set at the revision its check read;*
- δ-fresh *if it is secure and every τ_d includes every write committed before $t - \delta$, so that a permission revoked more than δ before the request, and not granted again, admits nothing in it;*
- sound *if $H \subseteq T^\star_\epsilon$ (mis-retrieval, returning chunks that are not among the exact answers, violates it);*
- complete *if $|H| = \min(k, |R_\tau(s)|)$ (under-retrieval violates it);*
- maximal *if it is sound and complete, that is, H is $\text{top}_k(q, R_\tau(s))$ up to ties within ϵ; and*
- count-honest *if $c = 1$ implies $|H| = \min\big(k, |R_\tau(s) \cap E(\varphi_s)|\big)$.*

Security, and freshness for the δ that the serving path states, are required of every result, and count-honesty is required of the flag; soundness and completeness are what approximate search trades away. By these requirements the code we ran did not qualify on every path: it exceeded its stated δ after an invalidation (Section 5.1), and the release we ran end to end was not count-honest (Section 3). Security concerns the returned set only; everything else a caller observes we treat as a side channel (Section 8). Appendix C.1 discusses the criteria against their sources.

We make three assumptions. (A1) *Correct checks*: $\text{check}_\tau(s, d) = P_\tau(s, d)$ for every subject, chunk and revision τ the check reads. (A2) *Faithful metadata*: for every candidate the store returns, the identifier and attributes that the check evaluates belong to the chunk whose content is returned. (A3) *Fail closed*: an error, timeout or malformed answer from the authorizer ends the request with no hits.

Observation 1 (Security does not depend on the filter or the index)**.** *Under* (A1) *to* (A3)*, if the layer returns only candidates that passed* check*, every result it returns is secure, whatever predicate φ_s was compiled and however the store evaluated or approximated it. If, moreover, every check reads a revision that includes every write committed before $t - \delta$, every result is δ-fresh.*

Argument. Each $d \in H$ passed $\text{check}_{\tau_d}(s, d) = 1$ on the identifier and attributes the store returned with it. By (A2) these describe the returned content, and by (A1) $P_{\tau_d}(s, d) = 1$, so $d \in R_{\tau_d}(s)$. If a check cannot be completed, (A3) gives $H = \emptyset$. The freshness part restates the condition on τ_d. No step refers to φ_s or to the store's search procedure. □

The statement is close to definitional, hence an observation: it is an argument about a code path, and no rate we report estimates it. Its content is what it leaves out: no assumption about φ_s, the filter APIs, the translators, the store's filter semantics or its index, and it holds for every query, chosen adaptively or not. The re-check sees filter faults in one direction only: a *false admit*, where φ_s accepts a chunk outside $R_\tau(s)$, becomes a violation; a *false denial*, where φ_s rejects a readable chunk, silently costs completeness, so only an offline oracle finds it.

Trusted computing base. The observation trusts (i) the check API and the authorization data it reads (A1), including the freshness of ACLs copied into chunk metadata; (ii) the layer's mapping of a request to checks, from the verified token to the adapter code; (iii) the binding of content to identifier and metadata in the store and in ingestion (A2); (iv) the decision cache and the engine's own staleness, which together set δ; and (v) the fail-closed path (A3). It does not trust the filter APIs, the translators, the store's filter semantics or index, or the widening loop. We give no measure of its size. The trusted part has failed: two faults in the layer's own adapters let the check itself admit (Table 7 in Appendix F), both fixed before the version we evaluate, and in the code we ran the cache held revoked grants past its bound (Section 5.1). With the re-check turned off, the filter and its translator join the trusted computing base.

Staleness and count certificates. If the engine may lack the writes of the last δ_e seconds and the layer caches each decision for at most a time to live ℓ, the layer is $(\ell + \delta_e)$-fresh, or δ_e-fresh without a cache. On a short answer, a layer can set $c = 1$ count-honestly only if the store searched exactly under φ_s, or if a filter-only count shows that every chunk passing φ_s was among the candidates (Appendix C.2).

2.3 Threat model

The adversary is an authenticated subject, or an agent or application acting for one, who may issue any number of arbitrary queries through the layer, chosen adaptively and including red-team prompts, to obtain the content of any chunk outside its readable set or to learn whether such chunks exist near a query. It does not control the authorizer, the store or the layer's host. Only Observation 1 speaks to this adversary: we ran no adaptive attack, and our red-team prompts are fixed templates that change only the query text. *In scope* are disclosure across users and tenants through retrieval results; faults in filter compilation, store filter semantics and approximate indexes; authorizer outages and malformed answers; identity confusion [65]; and revocation within the δ

the path states. *Out of scope* are prompt injection and poisoning in readable content [191]; wrong upstream permissions; extraction within the readable set [182]; and compromise of the store or its hosts. We say that a side channel *reveals* information about unreadable chunks, reserving *leak* for returned content, a narrower use than in much of the security literature, where such a disclosure is also a leak. Appendix C.3 gives the full model.

2.4 Metrics

A *leak* is a returned chunk outside $R_\tau(s)$ as the oracle decides it, or a canary string from an unreadable document found in a hit, a prompt or an answer [25]. Hits cluster by question and by subject, so when no leak occurs we state the rate per question and per subject, with the one-sided 95% bound $1 - 0.05^{1/n} \approx 3/n$ for zero events in n trials [64]. That formula assumes independent trials. Our questions and people are only partly so, and a deterministic predicate does not generate Bernoulli trials, so read these bounds as a summary of how many distinct cases ran clean (Section 9). They hold over the question generator and the injected faults; for an adaptive adversary only Observation 1 and its trusted computing base speak. *Permission-recall@k* (recall, for short) is $|H \cap T^\star_\epsilon| / \min(k, |R_\tau(s)|)$ with $\epsilon = 10^{-4}$, against exact search over the readable set, so it charges filter false denials and index misses alike. The *under-retrieval rate* is the share of retrievals that are not complete, the *empty-result rate* the share returning none to subjects who can read something, and the *false-completeness rate* the share of results with $c = 1$ that are not maximal. *Selectivity* is the fraction of the corpus that passes a filter. Rates carry 95% intervals from a cluster bootstrap over subjects, conditional on the index builds we made, and comparisons are paired on the same questions; the measured correlation axes are in Appendix C.4.

3 The Enforcement Layer Under Test

WARDOWL, the layer we test, is a Python library that implements steps (i) to (vi) of Section 2.1 between a RAG application, an authorizer and a vector store. The enforcement mechanism is not ours: Table 6 in Appendix D gives the precedent of each step. We add a single reference semantics for permission filters, which makes the mechanism testable across stores, and a count-honest completeness flag, which does not overstate how many chunks an approximate index returned.

An authorizer offers a batch check and either a filter that selects *exactly* what s may read or nothing. Besides ACLs copied into chunk metadata, WARDOWL supports seven engines: its adapters for OPA, OpenFGA, SpiceDB and Cerbos offer filters, and those for Ory Keto, Cedar and Casbin offer checks only. That is a limit of the adapters: Cedar, for one, exposes partial evaluation [26], which the adapter does not use. Adapters cover 14 stores, and an in-memory backend that evaluates the reference semantics is their oracle.

Filter semantics. Filters are built from $\mathrm{In}(f, V)$, And and Or. Let $\mathcal{V}$ be the strings and integers, Booleans excluded, with strings compared by code point, without case folding or Unicode normalization, and never equal to integers. One function defines, for metadata m and $V \subseteq \mathcal{V}$,

$$m \models \mathrm{In}(f, V) \iff \begin{cases} \exists x \in m(f) : \ x \in \mathcal{V} \wedge x \in V & \text{if } m(f) \text{ is a list,} \\ m(f) \in \mathcal{V} \wedge m(f) \in V & \text{otherwise,} \end{cases} \tag{1}$$

with And and Or as conjunction and disjunction. Every store translation must agree with it. Empty And and Or are rejected, a missing field never matches, and a filter that can match nothing returns no hits without a query, because several stores read a missing or empty filter as "match everything".

Retrieval. The layer pre-filters when the backend supports filters and the authorizer returns φ_s: one search with φ_s, every hit checked again, those that fail dropped and counted as violations. Otherwise it post-filters: it fetches $4k$ candidates, checks those not seen before, and multiplies the fetch by four until k hits are permitted, the store returns fewer than requested, or a cap is reached (1,000 by default). Any exception or malformed answer from the authorizer ends the request with no hits. With the re-check off, a filter that over-approximates the policy is returned unchecked.

Completeness. The release we ran end to end (2.1.0) set $c = 1$ for any pre-filtered result without violations, an empty one included, so it was not count-honest. The current code (PR #166, not yet in a release) lets a backend declare exact search; with e the declaration, v the violations and H_0 the store's answer, a pre-filtered result has $c = (|H_0| = k \vee e) \wedge (v = 0)$. The flag is thus count-honest relative to φ_s, and only to it: a false denial shrinks $E(\varphi_s)$ unseen, and a full answer is taken at the store's word, so $c = 1$ does not certify maximality; and an unfiltered post-filter reads a short list as the end of the store, which fails for pgvector with iterative scans off. A caching wrapper stores decisions for a time to live (30 s by default), a stated revocation bound with explicit invalidation that the code we ran exceeded behind stale engines (Section 5.1), and no interface lets a client or a model choose the subject. Appendix D details the flag, composition, tenant scopes, the cache and identity.

4 Differential Conformance Testing of Filter Semantics

Observation 1 shields security from filter faults, but not maximality, and it rests on stores returning the metadata they stored (A2). A filter that admits too much shows up online as violations; one that admits too little, or a store that alters metadata, does not. We therefore test every store translation and every authorizer offline against Equation 1: differential testing in the line of the logic-bug oracles for database engines [136], with property-based generation [84], applied to metadata-filter semantics across vector stores, for which we found no such oracle [168, 178].

The kit. From a seed, the kit draws small corpora with confusable values (`1` and `"1"`; `"a"` and `"A"`; é as one code point or two), missing and dotted fields, nested filters and subjects in 300 extra groups. It checks searches against $E(\varphi)$: no document outside $E(\varphi)$, $\min(k, |E(\varphi)|)$ of them, and text and metadata identical to what was stored. Stores whose index is approximate even on these corpora (Cassandra, and Vespa and Redis with HNSW) run in a sound-only mode. For authorizers it requires check to agree with $m \models \varphi_s$ on every hit, which shows consistency, not a correct policy. Its own suite plants 18 faulty backends and six faulty authorizers that it must reject (Appendix E).

The campaign. We ran the kit with fixed versions, seeds and counts against 13 stores and 13 engine targets (OPA, OpenFGA, SpiceDB, Cerbos, Cedar and Casbin, each with two policies, and Ory Keto), beside one control of each kind. The adapters for Keto, Cedar and Casbin offer no filter, so there the kit exercises the check path only; OPA, OpenFGA, SpiceDB and Cerbos are the engines compared with a reference answer. Pinecone needs an API key we did not have, so its adapter is not covered. A run is one (target, seed, variant), of 20 seeds and nine or seven data variants, and takes the worst outcome of its examples. The first campaign completed 4,315 runs: 3,612 passed, 84 were violations and 619 were errors. The main result is a null: both controls passed every run, no target returned a hit outside the reference on the generated data, and none altered stored metadata. Of the violations, 39 came from the layer's Cerbos translator, whose In also matches a scalar where Cerbos's check denies: a type gap in the adapter, not a fault of Cerbos, which the project's review had found and the campaign reproduced (Appendix E.2). The other 45 were Redis (26) and MongoDB (19), which cut a long term when they index it and look up the

Store or engine	Finding	Found by	Effect	In WARDOWL; upstream
Redis 8.10; MongoDB Atlas 8.0, 8.3	Long values are cut when indexed (at 4,096 bytes; at 8,191 characters) and looked up whole	Campaign	Recall; false admit	Digest of long terms (PR #167); reported
Elasticsearch, OpenSearch, Milvus, Qdrant	Size and syntax limits, some documented and some not, refuse a write or query	Campaign	Availability	None needed: fail closed
Chroma 1.5.9, local mode	A flat `$or` of about 1,000 clauses exceeds SQLite's expression depth; one of about 16,500 or more kills the client process	Campaign; filter size	Availability; crash	Nested clauses, then local filtering (PR #187); reported
OpenFGA, SpiceDB (adapters)	A subject outside the engine's limits makes it refuse the whole request	Campaign	Availability	Refused before sending (PR #167)
Cerbos (adapter)	CEL `in`, `==` and `hasIntersection` translated to In, which also matches a scalar	Review; campaign	False admit	Translator restricted, documented; note filed
pgvector 0.8	Iterative scans stop at either of two limits: few or no rows under selective filters	End-to-end runs	Recall	Honest completeness; limits documented
pgvector via psycopg	The plan cache chooses at run time between an exact generic plan and lossy custom plans	Diagnosis grid	Recall or speed	None yet; rule documented
SpiceDB 1.56	Under `minimizeLatency` a revoked permission admitted for 8.7 s in 1 of 40 trials, unexplained (a rerun's 40 trials: at most 5.2 s), and could reappear after it had gone	Engine study	Stale	Declares 11 s (PR #186)
SpiceDB, Cerbos (cache)	After `invalidate()` the cache refilled from the engine's stale answer and held the revoked grant for its full time to live	Engine study	Stale	Holds back for the declared staleness (PR #186)
SpiceDB (adapter)	An empty LookupResources stream was read as "nothing readable"	Engine study	Recall	No filter: post-filter (PR #186)
OPA (partial evaluation)	The residual keeps `not (x in [])` even for users with no ban, and the layer's filters have no negation	Engine study	Speed	None; such users are post-filtered

Table 1: This paper's findings on stores and engines. No row has been confirmed by the maintainers of the store or engine. Table 7 in Appendix F gives all 31 rows, among them 18 that record the project's development before this paper.

whole term when they search: at 4,096 bytes of tag value, which the layer's hex encoding reaches at 2,048 bytes of field and value, and at 8,191 characters. A value equal to the kept prefix of a longer one also matches it, a false admit that the re-check should catch; the reproducers, which do not use the layer, show the admit, and the campaign's generator holds no such pairs. We did not measure whether real ACL values reach such lengths. Of the errors, 479 were OpenFGA and SpiceDB refusing whole batches with HTTP 400, and 140 were stores' limits, some documented and some not, which fail loudly and return nothing.

We changed the four adapters. The second campaign, of 4,480 runs, had 3,822 passes, 39 violations, 479 refusals and 140 errors, as the diagnosis predicted, and Redis and MongoDB passed every long- and huge-string run. The 479 refusals are 85.5% of OpenFGA's and SpiceDB's 560 runs, and only 81 runs reached them with every subject, so the campaign's evidence on them is thin; Section 5 tests them on data within their limits.

Table 1 collects what this paper's experiments exposed. Counted by kind, the campaign found two store behaviors with a correctness effect, two availability faults in the layer's own adapters and the limits of five stores, some documented and some not, which fail closed, and it reproduced one known gap. Four of the table's eleven rows are faults in WARDOWL's own adapters or cache that a store's or engine's behavior exposed, so part of what the study found is its own instrument; the reproducers of the reported behaviors use only the vendors' clients.

5 Do Authorization Engines' Filters Agree With Their Checks?

Observation 1 lets a pushed-down filter be wrong without a leak, but not without cost. We ask whether the filters WARDOWL compiles from four engines admit exactly what the same engines' checks allow, separating a translation fault, which persists at one revision, from revision skew, a write between the two calls, and from truncated list APIs. We then measure how long a revocation keeps admitting and whether the layer fails closed. We tested OpenFGA 1.22.0, SpiceDB 1.56.2, OPA 1.21.2 and Cerbos 0.56.0 on five seeds of a generated world of 120 users, the subjects, and 1,500 documents, against an oracle that is a separate plain implementation of the same model. *One revision* means SpiceDB's `atExactSnapshot`; OpenFGA has no snapshot token, so its store was read quiescent; OPA and Cerbos are stateless per request (Appendix G).

Agreement at one revision. In the seven profiles that hold the model as intended, 6.3 million (user, document) pairs produced no disagreement between filter and check, and every check agreed with the oracle (one-sided 95% bound on users with any disagreement: 0.5% per profile; X4-H1 confirmed; Table 8 in Appendix G.2). These pairs repeat one generated model per profile over five seeds, and the OPA and Cerbos profiles omit bans (Cerbos's also folders), so this has little power to find a schema-shape fault such as CVE-2025-65111, in which SpiceDB before 1.47.1 missed LookupResources results [102]. The only disagreement at one revision is the Cerbos type gap: when 10% of documents store a one-value ACL list as a plain value, the adapter's filter admits what the check denies in 569 pairs (0.063% of pairs; 52.5% of users, 95% interval 48.3% to 56.3%), and the re-check turns each into a violation (X4-H2 confirmed). Neither translator dropped a negation, but vacuously: with bans no user received a filter (X4-H3 confirmed as registered), so negation in a filter is untested. OPA's partial evaluation keeps `not (x in [])` even for the 73 users with no ban in scope, which falsified our availability prediction: one ban rule makes that filter unavailable to everyone, who are then post-filtered.

Revision skew and truncation. Unpinned under a writer committing 20 toggles a second, filter and check disagreed in every OpenFGA trial, in 81.5% of SpiceDB trials with `fullyConsistent` (95% interval 76.0% to 86.5%) and in 13.0% of OPA trials (8.5% to 18.0%). In those conditions a write between the calls explained every disagreeing pair, an attribution that is generous where windows hold dozens of writes (medians of 59 and 208 for OpenFGA), and none reproduced; under SpiceDB's default `minimizeLatency` it explained 226 of 228. Pinned to one SpiceDB snapshot, 200 trials produced no disagreement (X4-H4 confirmed). An online violation is thus evidence of a translation fault only when filter and check read one revision. ListObjects and LookupResources with the layer's limit of 1,000 truncated without a signal, and the layer gave no filter at the limit in all 540 calls (X4-H5 confirmed), at a cost: with 1,000 or 1,001 readable documents among 100,000, recall@10 fell to 0.855 to 0.891 (X4-H6 confirmed).

5.1 Revocation and faults

Table 2 gives the measured δ of each serving path, and a dagger marks a violation of the δ the path states. The cache's time to live is the realized delay (X7-H2 confirmed), and invalidation ends the delay at once, or within the shared replica's 1 s local copy. Permissions copied into chunk metadata took effect within milliseconds of the write to the store (X7-H4 confirmed); the delay of copying them from a source system was not measured. Two sources of truth were stale themselves. Cerbos reloaded a rewritten policy file in a median 2.5 s. SpiceDB under `minimizeLatency`, the layer's default, took a median 3.3 s; in 28 polls a revoked chunk reappeared after it had disappeared, and one of 40 trials took 8.7 s, beyond the 6.5 s we had registered (X7-H5 falsified), which we have not

Path (revocation)	No cache	ttl 5 s	ttl 30 s	+ invalidate	shared + inv.
Metadata ACL, pgvector or Qdrant	<0.01	<0.01	<0.01	<0.01	<0.01
OpenFGA (tuple delete)	<0.01	5.02	30.02	<0.01	1.01
OpenFGA, check cache, `MINIMIZE_LATENCY`	<0.01	5.02	30.02	<0.01	1.02
SpiceDB, `fullyConsistent`	<0.01	5.02	30.02	<0.01	1.02
SpiceDB, `minimizeLatency`	3.25†	5.02	30.02	30.04†	31.05†
OPA (data update)	<0.01	5.03	30.03	<0.01	1.02
OPA (policy update)	0.01	5.02	30.03	0.01	1.02
Cerbos (policy file reload)	2.55	5.02	30.02	30.03	31.04

Table 2: Median seconds from a revocation to the first of three consecutive retrievals without the revoked chunk; 40 trials per cell, the cache warmed just before. "Shared" polls a second replica sharing the cache through Redis. †: at least one trial exceeded the stated bound, the ttl (none after invalidation, 1 s for the replica) plus the engine's stated staleness, by more than the 1 s we allowed for polling; Cerbos states no staleness.

explained; a rerun's 40 trials took at most 5.2 s. Invalidation did not survive such an engine: the next call refilled the cache from the stale snapshot, so 79 of 80 SpiceDB revocations with invalidation exceeded their bound (medians 30.0 and 31.0 s, maximum 38.4 s; X7-H1 and X7-H3 falsified in these cells), and Cerbos's took as long.

We fixed this and four other defects the run exposed: after an invalidation the cache now caches nothing for the engine's declared staleness (11 s for SpiceDB without full consistency, set from the one 8.7 s trial). A targeted rerun, an implementation check specified in writing after the main results and before running, confirmed every prediction (A2-H1 to A2-H8): all 160 SpiceDB and Cerbos revocations with invalidation took effect within the declared staleness plus 1 s, and 1 s more on the replica, at medians of 4.8 s (at most 5.2 s) and 3.9 s against 30 and 31 s before. With a maximum of 5.2 s the rerun shows that the fix holds, not that 11 s is tight, and these fixes have not been through the end-to-end runs.

We injected 14 faults into each engine, and five or eight into pgvector and Qdrant, in 1,090 episodes. Outside the control below, no retrieval returned a chunk outside the user's readable set, in 623,279 requests over 1,050 episodes. The one-sided 95% formula gives 2.8×10^{-3} per episode (X7-H6 confirmed), but these episodes are not samples from a population of faults: they replay 13 fault kinds, each episode cycling 10 fixed (user, query) pairs, and the same formula over the 13 kinds gives 0.21. The control, 40 episodes that turned the engine's denials into allows, is the fault class that does leak: it leaked in every engine, as a wrong check must, and is excluded from the bound. Every engine-caused failure was the layer's authorization error with a message naming no host. Recovery took a median 0.02 to 0.18 s after a transport fault's removal and 0.08 to 0.47 s after a killed server's restart. Two behaviors needed work: an empty LookupResources stream read as "no readable documents" (4,746 retrievals returned nothing without an error, losing recall only), and store failures surfaced as the clients' own exceptions. The rerun confirmed both fixes: in the 10,303 requests of its 20 SpiceDB episodes no chunk leaked, and its 120 store episodes reported none either (2.1×10^{-2} per episode over all 140). For a connection that is accepted but never answered, only a deadline on the whole retrieval, which the layer does not yet offer, would bound the wait.

6 Retrieval Quality Under Permission-Shaped Filters

No configuration of Section 9 returned a chunk outside the subject's readable set at the revision the re-check read; this section asks what the permission filter cost. Recall is permission-recall@k

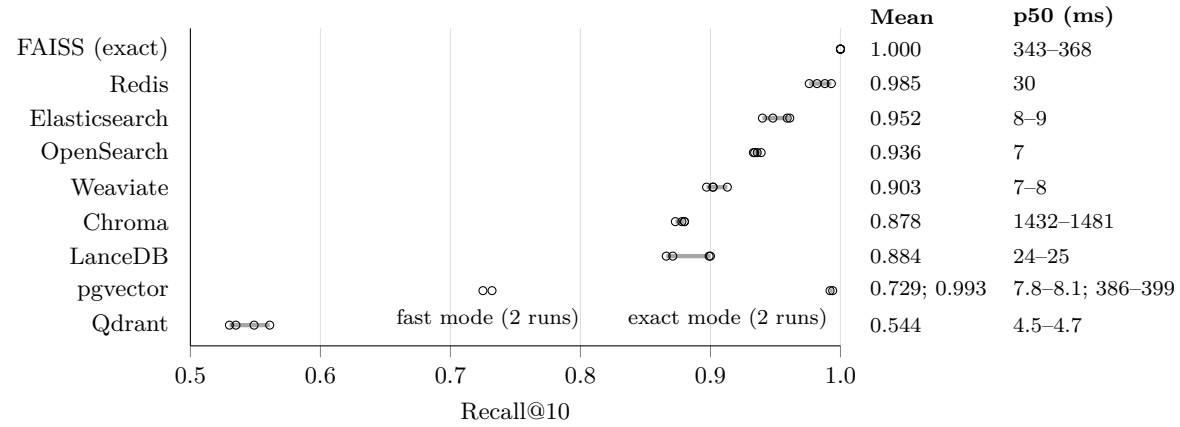


Figure 1: Recall@10 in the enterprise scenario at the settings of Appendix P, some tuned and some default, four runs (seeds 7, 11, 13, 17). Each circle is one run and each bar the range over runs, except pgvector's, which spans only its two exact-mode runs; pgvector ran in one of two modes per run (Section 6.3), and its right-hand values are per mode, fast then exact. p50 is the range of per-run median retrieval latency. Table 9 in Appendix H.1 adds recall@5 and overlap by person type.

(Section 2.4). Every result measures a store under this layer's schema and settings, not the store alone: for example, metadata in one `jsonb` column tested with `@>` on pgvector, and list values as boolean keys on Chroma. Intervals here resample people within fixed index builds; where the range over builds is larger, as for Qdrant, we say so.

6.1 An audit at the harness's settings

The enterprise scenario (Section 9) holds 107,540 chunks in 20 tenants. Figure 1 shows the spread at the settings of Appendix P, some tuned (pgvector's iterative scans, LanceDB's re-ranking) and some defaults. Mean recall@10 over the four runs ranged from 0.54 for Qdrant (0.530 in its worst run) to 0.98 for Redis across the eight approximate stores, with pgvector in its fast mode; the exact control, FAISS, scored 1.000 and pgvector in its exact mode 0.993. At the stores' own defaults the same corpus gave pgvector 0.112 and LanceDB 0.439 (Table 11). The figure compares configurations, not stores, and its numbers hold for this corpus and question mix. In this corpus only one of twenty near-identical tenant copies of a document passes a person's filter, and Section 6.2 shows that Qdrant's loss depends on that layout. The mean also moves with the question mix: on the same builds of the frontier experiment, Qdrant's as-run setting scored 0.576 on these 1,105 questions and 0.663 on all 1,468 scorable pairs (0.656 on the diagnosis grid below), and pgvector's custom plans 0.730 and 0.661. No store other than pgvector moved its recall@10 by more than 0.04 across the runs.

Two failures stand out. pgvector gave empty answers to questions from some of the least-privileged people of the scenario, although each could read 624 chunks: seven empty answers in the seed-17 run, on seven people who each got hits on their other four questions, and one in the seed-11 run. The evaluated release called these empty answers complete. In the seed-11 and seed-13 runs the same configuration searched exactly, at fifty times the latency; in the seed-7 run it searched fast, like seed 17, and left no answer empty. Qdrant never returned a short answer in these runs, yet for ordinary employees it returned only about half of the exact top five (0.52 in the first run), while for the least-privileged people it was perfect. pgvector thus *under-retrieves*, which a count can detect, and also returns wrong chunks; Qdrant only *mis-retrieves*, which only an exact baseline reveals. We pre-registered hypotheses for both before running a diagnosis grid of 1,489 questions

and three index builds per layout (Appendix H.4).

6.2 Qdrant: a loss consistent with an islanded permitted subgraph

Qdrant plans each segment from the filter's estimated size: below a threshold (about 3,333 points at 768 dimensions, by our arithmetic) it scans the payload index exactly, and above it it uses the filterable graph [127], whose traversal, in the source at v1.19.2, expands only neighbors that pass the filter. In the scenario every tenant value and the employee and contractor groups exceed the cap below which extra edges keep a filtered subgraph connected [34]. Our hypothesis: near a question about company documents, only one of the twenty tenant copies passes the filter, so the permitted company region is a sparse island in the graph, while the shared passages, all permitted, form a large connected region that the traversal enters and does not leave; the least-privileged people's filter is small enough for an exact scan. We did not measure connectivity or reachability, and per-tenant graphs remove the copies and the island at once, so the two are not separated.

The registered predictions, which test recall by question category and by intervention, held in both runs of the grid (Q1 confirmed; Table 10 in Appendix H.2). Recall@10 for the least-privileged people was 1.000, against 0.369 (95% interval 0.334 to 0.402) for company documents among everyone else, and 97.5% of the top-five hits that did not count were shared passages. ACORN, which explores neighbors of filtered-out neighbors [120], lifted recall@10 over all questions from 0.656 to 0.808 at a 3% latency cost; per-tenant graphs [126] lifted it to 0.952, faster than the as-run layout, and a separate collection for the shared passages reached 0.949. These means have ranges of 0.086 and 0.127 over three builds of the same data, as run and with ACORN, wider than any interval over people. Raising `hnsw_ef` from 128 to 512 added 0.103 (0.090 to 0.116) at 1.6 times the latency (Q4 inconclusive). We expected ACORN's default gate to engage for some people and not others; it engaged for everyone (Q2 falsified). At `hnsw_ef` 128 ACORN did not reach the layout change; in the frontier experiment, with its gate at 1.0 and `hnsw_ef` 512, it reached 0.953 against 0.941 for per-tenant graphs at `hnsw_ef` 128, with about 60% of their throughput (0.61 to 0.63 on each of the three builds). Q3, on segment layout, stayed inconclusive, since both layouts ended with one segment holding points. For layouts like this one, changing the layout cost less than reaching the same recall through search-time effort: when permission groups or tenants cover large, semantically interleaved parts of a collection, give each tenant its own graph, or keep widely shared content apart.

6.3 pgvector: a plan cache, then a scan budget

pgvector 0.8 scans the HNSW graph in order of distance and applies the filter to each row; with iterative scans it continues until enough rows pass or it reaches a limit [121]: `hnsw.max_scan_tuples` (20,000 by default) or a memory bound. We hypothesized that the limit caused the empty answers. A decision taken before the scan matters more: whether PostgreSQL runs the HNSW scan at all.

The plan cache. The layer binds the filter's values and k as parameters. psycopg prepares a statement on its sixth execution on a connection [159]; PostgreSQL then uses a generic plan when its estimated cost is not much higher than the custom plans' average [156], and that plan reads the rows the GIN index returns and sorts them: an exact search. Under the default configuration, after a vacuum, only 0.9% of the searches on the scenario's questions used the HNSW index, and recall@5 was 0.996 at a median of 362 ms; planned with their own values, 98.1% used it, with recall@5 of 0.680 at 22 ms, against 0.691 in the first execution of the audit's seed-7 run and 0.703 in the execution the artifact holds (P5 confirmed, an exploratory hypothesis specified in writing after the grid's first run and before its second; P4, which had predicted that generic plans make no difference, falsified, with recall@10 lower by 0.343 under custom plans).

One replay pair, on one build and always in the same order, points to the state of the GIN index. Replaying the audit's pass right after loading, when the index's pending list held 388 pages, PostgreSQL never switched: recall@5 was 0.697 at 7.0 ms, the audit reproduced. The same replay after `VACUUM (ANALYZE)`, with the list empty, ran 2,148 generic executions, with recall@5 of 0.998 at 335 ms. That command changes more than the pending list, so the pair does not isolate it. PostgreSQL's cost estimate charges every pending page at the start of the scan [158], which is consistent with a generic plan that looks expensive after a bulk load until a vacuum merges the list. Autovacuum runs on its own schedule, and the variation across the end-to-end runs would fit it, but we did not record its timing there and cannot say what decided their modes: the same configuration ran fast by its median in three of eight runs, one of them a mixture, and exact in five. A measurement of pgvector through prepared statements can therefore report one of two systems, 16 to 64 times apart in latency and about 0.3 apart in recall, with no benchmark setting naming which. Planning every search with its own values removed the choice in the frontier experiment (Appendix I).

The scan's budget. With the HNSW scan forced, recall@10 was 0.656 on the grid's question set and 8.9% of retrievals with $k = 5$ were empty. Both limits bind: raising the tuple limit alone lowered the empty rate only to 7.3%, raising the memory alone changed nothing, and raising both removed every empty answer (P1 inconclusive, against a reference setting changed after the first run). The loss remained: with both limits raised recall@10 was 0.780, and 0.866 on a better graph. The rest is mis-retrieval, since a relaxed-order scan stops at the first k permitted rows it meets, but the loss was not concentrated as predicted (P2 inconclusive, same changed reference). For the least-privileged the planner's row estimate was a median 11.1 times their readable set (P3 inconclusive in the first run, confirmed in the second, the run that judges it).

6.4 A native baseline, in a side run

An earlier run of the library's own benchmarks, on 2026-10-22, compared it with native enforcement (Appendix O.1). On FiQA (57,600 chunks, 200 questions, readable shares of 0.06% to 100%), a PostgreSQL row-level security policy on the same table and HNSW index returned no unreadable chunk and reached recall@10 of 0.998 to 1.000, with median latency growing with the readable share from 2.1 to 608 ms; the layer's pre-filter reached 0.948 to 1.000 at 2.8 to 82.2 ms. The plans were not captured; the latency's shape is what an exact scan of the admitted rows would give. This is a side run on a different corpus, commit and harness, without tenant copies, and without measuring side channels, so it does not say what row-level security would do on the enterprise corpus. Like the exact configurations above, it shows that enforcing permissions need not lose recall, at that latency. Document-level security in Elasticsearch and OpenSearch was not measured.

6.5 Recall against throughput

To separate a filter's selectivity from its correlation with the question, the frontier experiment adds a corpus whose layouts we control: the first million passages of MS MARCO [19], in a hierarchy of teams assigned at random (*uncorrelated*) or by hierarchical k-means over the embeddings (*clustered*), with 6,200 question–subject pairs and selectivities from 0.06% to 100%. *Negatively correlated* questions have nothing readable among their 1,000 nearest passages. Ten approximate stores run against FAISS over a flat index, each at its default, at five levels of search effort and with its own options, in three fresh builds; Chroma was too slow to reach a timed pass. Eight of the audit's nine stores reproduced the recall@10 of its first run within 0.03. Appendix I gives the design.

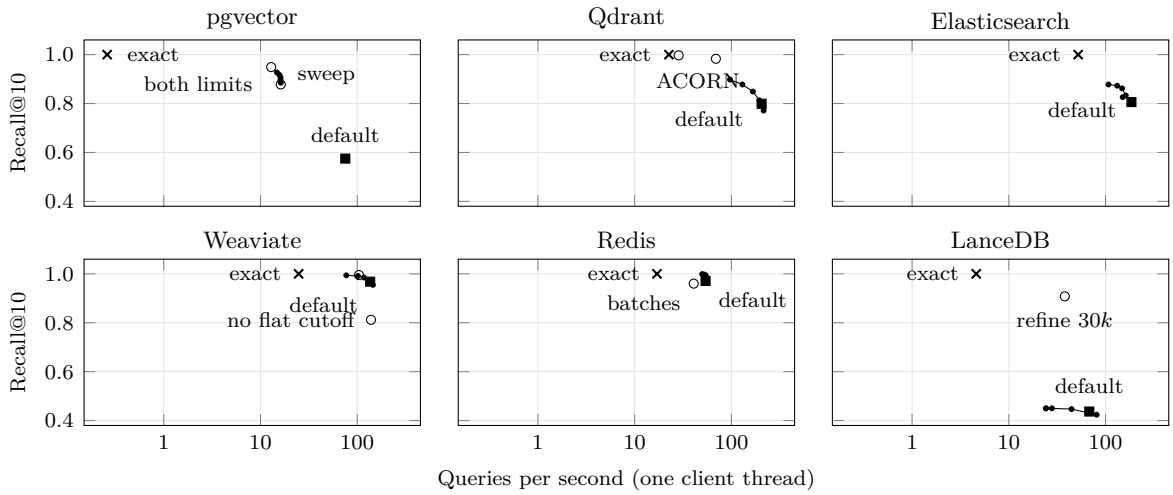


Figure 2: Recall@10 against throughput on MS MARCO, one panel per store: square, the default; line, the five-level effort sweep; circles, store options; cross, the store's exact search. Recall is the mean over three builds and throughput that of the store's first build, in its own job on its own runner, so compare within a panel, not across panels. LanceDB's post-filter (0.175) lies below its panel. Table 11 in Appendix I has these values and the stores not drawn.

Defaults and effort. At the defaults, recall@10 ranged from 0.437 and 0.575 to 0.998. The two lowest are documented defaults: pgvector without iterative scans came back empty on 32.4% of retrievals, and LanceDB's IVF-PQ index ranks by quantized distances, so re-ranking 30 times k candidates reached 0.908. Elsewhere effort bought recall at a price in throughput, and the stores' own options bought more: Qdrant reached 0.898 at the top of its sweep but 0.997 with ACORN. Four stores lost recall against their own 100% selectivity baseline over the whole mix of filters: pgvector (0.575 against 0.916), Qdrant (0.799 against 0.978), Elasticsearch (0.806 against 0.870) and LanceDB (0.437 against 0.517).

Correlation and selectivity. Negatively correlated questions lost at least 0.05 of recall@10 in four stores, not the six we had predicted (H2.1 inconclusive): pgvector 0.424 (95% interval 0.415 to 0.433), LanceDB 0.254, Elasticsearch 0.137 and Weaviate 0.055. Redis paid in latency instead. Local selectivity predicted recall better than global selectivity in three stores and worse in five (H2.2 falsified). At the top of each sweep the gap narrowed and remained in two of the four stores that had one (H2.3 confirmed by its rule, though only pgvector's narrowing is large): pgvector's fell from 0.424 to 0.172, LanceDB's moved by 0.001, Weaviate's closed and Elasticsearch's widened. On clustered layouts with negatively correlated questions, filters admitting 0.5% to 10% of the corpus were harder than the smallest (0.06%) in five of nine stores (H2.4 confirmed; it does not ask whether recall recovers at larger selectivities): Qdrant's recall@10 at those four levels, from the smallest up, was 1.000, 1.000, 0.460 and 0.489. In this regime very selective permission filters were the easy case for most stores.

Filter size. Varying only a disjunction's length from 1 to 40,000 groups, with the same 10% of a 100,000-chunk corpus readable at every level, recall moved by at most 0.012 with length in any of the ten approximate stores that accepted every level (X3-H4 confirmed for eight of the nine stores it names, inconclusive for Chroma; pgvector and Cassandra were exploratory), while median latency grew 272-fold on pgvector and 11 to 99 times on eight other servers. Chroma alone refused, from 1,000 groups, and at about 16,500 groups, with a 16 MiB stack, its local client process crashed

	Recall@10		Short			False complete		Triggered (ms)	
Corpus, store	report	exact	report	B 10,000	exact	report	exact	p50	p95
Enterprise, pgvector	0.667	0.782	0.116	0.000	0.000	0.551	0.480	30.1	40.2
Enterprise, Qdrant	0.654	0.654	0.000	0.000	0.000	0.581	0.573	1.57–1.58	1.69–1.84
MS MARCO, pgvector	0.900	0.915	0.017	0.001	0.000	0.301	0.296	108.6	220.7
MS MARCO, Qdrant	0.807	0.842	0.035	0.035	0.000	0.356	0.343	37.9	50.4

Table 3: Repair: *report* adds no query; *exact* repairs without a bound; B 10,000 is the default bound. Means over retrievals with $k = 10$. Short: fewer than $\min(k, R)$ hits, where R is what the subject may read. False complete: the share of results marked complete that are not maximal. Triggered: time spent on the short result (count and repair) with exact repair without a bound; Qdrant on the enterprise corpus triggered only for the one person who may read nothing, and repaired nothing.

(Appendix J).

7 Detecting and Repairing Under-Retrieval

The flag of Section 3 stops the layer from calling a short approximate answer complete. Here we decide cheaply why an answer is short, and repair it within a stated cost.

Design. When a pre-filtered search on a backend that does not declare exact search returns $h < k$ hits, the layer asks for $n = \min(\#\{d : m_d \models \varphi_s\},\ L)$, a filter-only count that stops at a limit L. If $n \leq h$, the store returned every chunk the filter admits (*exhausted*); if $n > h$, chunks that pass the filter were missed (*missed*). After a proven miss the layer can search again exactly under the same filter, when the count is at most a bound B (10,000 by default); above the bound the result says *over_limit*. The alternative *widen* repeats the search with more effort and certifies nothing. Every hit passes the same re-check, so Observation 1 holds unchanged. A sampled *monitor* also runs the exact search for a share of requests and reports the overlap of the two top-k lists.

Evaluation. We evaluate the policies on pgvector and Qdrant, on the enterprise scenario (1,468 questions with something to read) and the 6,200 pairs of the MS MARCO layouts, three fresh builds per store and corpus. Seven hypotheses were pre-registered. Table 3 gives the main arms and Table 14 in Appendix K.2 all of them.

Results. The count raised no false alarm and flagged every truly short first search as *missed*: 992 of 992 for pgvector on the enterprise corpus, and 574 of 574 for pgvector and 1,091 of 1,091 for Qdrant on MS MARCO, pooling $k = 5$ and 10 over three builds of the same pairs (H5.1 confirmed). On unchanging data this follows from what a count is, since a count above h proves a miss, so it checks the stores' counts and the harness and is no discovery; a write between the search and the count, the case that could make it wrong, was not tested. Repaired exactly without a bound, no answer was empty or short, and the repaired answers reached recall@10 of 1.0000 (H5.2 confirmed for the unbounded arm), as exact search should. The bound decides where repair applies: on MS MARCO every short answer of Qdrant had more than 10,000 readable chunks, so the bounded arms repaired none and reported each as *over_limit* and not complete. The cost on queries that did not trigger is inconclusive (H5.3). No repair touched Qdrant's loss on the enterprise corpus (H5.4 confirmed). The 1% monitor estimated Qdrant's recall within 0.030 and 0.004 (H5.5 confirmed), so on these corpora a 1% sample was enough to see the loss that the flag cannot. Exact repair beat widening by 0.085 of recall@10 (95% interval 0.064 to 0.110) on the 421 questions that triggered in both arms (H5.6 confirmed). On the enterprise corpus Qdrant's per-tenant graphs reached recall@10

of 0.939 while the best library arm on the shared layout reached 0.654 (H5.7 confirmed). Neither count nor repair sees a full answer of the wrong chunks, which was most of the loss on both stores; only a sampled exact search sees it, and only a better index or layout removes it.

8 Side Channels in Filtered Retrieval

Observation 1 bounds what a subject can *read*; it says nothing about what the timing or shape of an answer reveals. Büttcher and Clarke required of secure full-text search that no aspect of the output depend on content the user may not search [23]. Each enforcement path departs from that in a predictable way: a widening post-filter works longer when unreadable chunks crowd the top of the ranking, an approximate filtered search passes unreadable nodes on its way to readable ones, and a keyword score computed over the collection counts unreadable documents. Stronger attacks than ours exist on the last channel and on row-level security timing (below). We ask a narrower question, for the paths of this layer: how much can a subject learn from these departures about whether unreadable chunks exist near a query, and what does removing each one cost?

8.1 A distinguishing game

Two worlds of equal size differ in the 1,233 unreadable chunks of one group of nine probes. In W_1 each probe's m secrets are its m most similar non-hub passages, for m from 1 to 1,000; in W_0 each secret is replaced by a passage of another domain, far from every probe and cut to the same length, at the same position in the index. So a probe labeled m has m secrets near it, and the other $1{,}233 - m$ lie beyond the post-filter's cap of 1,000 candidates for it, though not outside the index; each world is a separate build. The attacker's readable set is identical in both worlds. It issues the probe and sees what a caller of the layer saw in the version we measured: the latency, the number of hits, the completeness flag, the mean score of the hits, and the hits themselves, from which it can compute recall@10 against the exact top ten of its own readable set. The detection fields of Section 7 are also visible to a caller; they postdate this experiment and are an untested observable. The *differencing attacker*, our primary adversary, sees the same probe in both worlds, as someone who repeats a query before and after a document is added (we compared separately built indexes; adding to a live index was not measured), and must say which observation is W_1; its success is the paired AUC, the share of probes whose W_1 observation lies further in the predicted direction than their W_0 observation. An attacker who knows the sign of a difference gains as much below 0.5 as above it, so the size of a channel is $|\mathrm{AUC} - 0.5|$. This is a forced choice: the attacker is told that one of the two worlds holds the secrets. It does not measure detecting a change against no change, which needs a false-positive rate on identical content, measured only for some configurations below. The secrets are the m passages nearest the probe, the harness and the servers share one runner with one client thread, and an observation is the median of five repetitions, so these numbers say nothing about an attacker across a network or under concurrent load.

The background is all 57,638 FiQA passages [85], and three attackers read 1%, 10% and 50% (p) of it. Over five seeds we measure each store's approximate pre-filter at the settings the pre-registration fixed (pgvector with iterative scans in relaxed order at `ef_search` 40, Qdrant's default index, Elasticsearch with `num_candidates` 100), with FAISS as an exact control; the post-filter at the layer's defaults; keyword search; and mitigations: constant-work widening, which fetches the cap in one round; exact pre-filtered search; and latency padding. A channel is *present* when the paired AUC reaches the hypothesis's threshold and its 95% interval's lower end exceeds 0.55, and *absent* when the interval lies within $[0.40, 0.60]$. Appendix L.1 gives the design, and Table 15 the hypotheses.

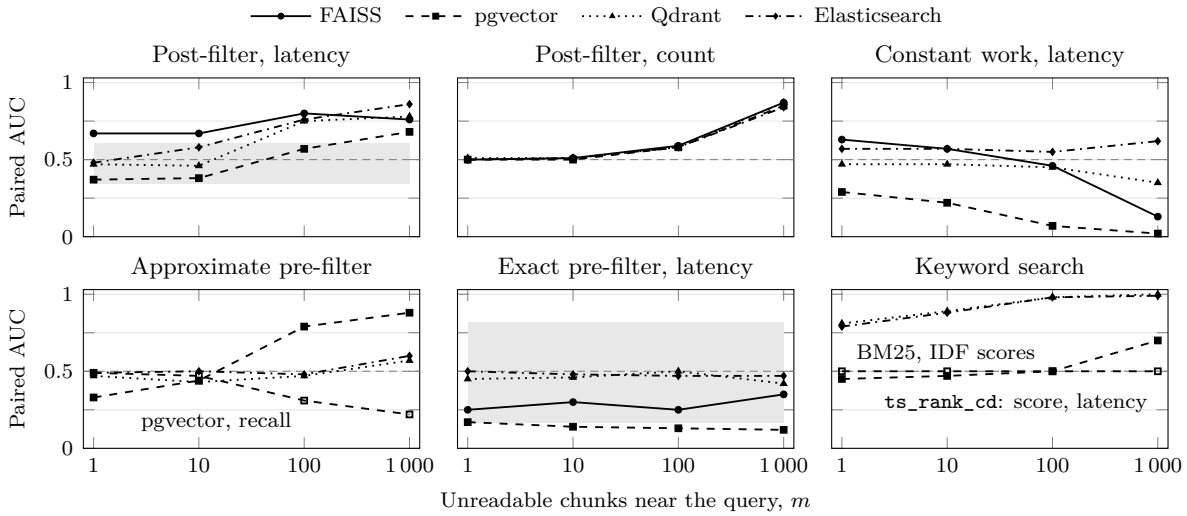


Figure 3: Paired AUC of the differencing attacker against m, pooled over the three attackers (120, 120, 80 and 40 probes at $m = 1$, 10, 100 and 1,000). Above the dashed line the observable moves as predicted for W_1 (latency up; count, recall and score down), below it the other way. Gray bands: the range of paired AUCs between builds of identical content, for FAISS's post-filter (first panel) and pgvector's exact search (fifth). Pooled curves mix attackers, so FAISS's first-panel fall at $m = 1{,}000$ comes from the attacker reading 1%. Keyword panel: the worlds differ in all 1,233 secrets at every m; open squares are `ts_rank_cd`'s score, filled squares its latency. Table 16 in Appendix L.2 has the values.

8.2 Results

Every pre-registered cell ran: 520,320 retrievals without an error, none returning a chunk outside the attacker's readable set. Figure 3 shows the channels by m. Three cautions come first. Two builds of identical content gave paired latency AUCs of 0.342 to 0.608 for FAISS's post-filter, and, per m, $|\text{AUC} - 0.5|$ up to 0.38 for its pre-filter, 0.33 for pgvector's exact search and 0.08 for Qdrant's exact search (0.24, 0.31 and under 0.01 pooled over m); the other configurations were not measured that way, and Elasticsearch's run was lost. A latency AUC anywhere from about 0.12 to 0.82, the range these builds of identical content reached, is therefore not by itself evidence about the secrets. We read growth with m as the sign of a channel and a flat curve as an offset, a criterion we adopted after seeing these results; same-content differences can themselves rise at $m = 1{,}000$, the cell with the fewest pairs, so a rise there alone does not establish a channel. Second, every W_0^g is built later in its seed than W_1, so build age is aligned with world in every latency cell. Third, intervals resample (seed, probe) pairs while the demonstrated noise is per build, so latency intervals are too narrow. We apply no correction for multiple comparisons; the verdicts rest on the cells the hypotheses name.

Widening (H1 falsified). Where the prediction rested on the loop alone, the post-filter's latency separated the worlds as predicted. At $m = 100$ one more widening round made W_1 slower, with paired AUCs of 0.850 (0.762 to 0.925) on FAISS, 0.800 (0.713 to 0.887) on Qdrant and 0.825 (0.738 to 0.900) on Elasticsearch at $p = 10\%$, and 0.925 to 0.975 on all four stores at $p = 50\%$; pgvector's 0.662 (0.550 to 0.762) at $p = 10\%$ was inconclusive. At $m = 1{,}000$ these attackers reach 1.000. The attacker reading 1% has no forward latency channel: 0.113 to 0.588 at $m = 100$ and 0.025 to 0.575 at $m = 1{,}000$, reversed on pgvector and FAISS and, at the edge of its interval, on Qdrant, while its count AUC at $m = 1{,}000$ is 1.000. Two parts failed. At $m = 1{,}000$ and $p = 50\%$ the count AUC was 0.613 to 0.688, below the falsification bound of 0.80, because readable hub passages among the candidates filled the answer. And FAISS's latency at $m = 1$ gave 0.667 (0.617 to 0.717) on

identical answers: two builds of the same content differed by as much, though the reading we fixed in advance for that follow-up is formally inconclusive.

Constant work (H2 falsified). Fetching the cap in one round removed the widening channel from latency pooled over m and p on FAISS (0.517) and Qdrant (0.456) and left the count channel, but pooling hides a reverse channel: at $m = 1{,}000$ FAISS read 0.133, Qdrant 0.350 and pgvector 0.017 (0.000 to 0.050).

Pre-filtering. pgvector's iterative scan separated the worlds (H3a confirmed): at $m = 1{,}000$ every probe was slower on W_1 at $p = 10\%$ and 50%, and it returned better answers there, with recall@10 of 0.968 and 0.978 against 0.707 and 0.665. Elasticsearch was inconclusive (H3b). Qdrant showed no channel in any observable at $p = 1\%$ and 10% (H3c confirmed). Exact search returned the same answer in both worlds on all four stores, yet every H4 verdict is inconclusive through latency: on pgvector and FAISS because of build offsets, and on Qdrant and Elasticsearch because their intervals at $m = 1{,}000$ were too wide at 40 clusters (the in-memory control failed a guard, with 15 clusters).

Collection statistics (H5). BM25 on Elasticsearch and IDF-weighted sparse search on Qdrant separated the worlds through their scores: paired AUC 0.983 and 0.992 on Elasticsearch and 0.979 and 1.000 on Qdrant at $m = 100$ and 1,000. At $m = 1$ and 10 the AUCs were 0.794 to 0.886, below the predicted 0.90 (H5a inconclusive). No cell isolates m documents, at any m, because the two worlds differ in all 1,233 secrets of the probe's group. `ts_rank_cd`, which uses no collection statistics, showed no score channel (H5b confirmed), and reciprocal-rank fusion changed the hit list for 56.7% of probes at $m = 1{,}000$ (H5c confirmed). This channel is old and has been taken much further: Wang et al. exploited it on shared indexes in Elasticsearch and on services such as GitHub to discover and count terms in other tenants' documents [165], and Espiritu and Cash recover indexed terms, and sometimes approximate text, through document-level security in Elasticsearch and OpenSearch [49], both with an attacker who writes documents. Our attacker inserts nothing and asks only whether nearby secrets exist, and we attempted no recovery. We add only that the channel appears through this layer's pre-filter and in Qdrant's sparse search.

Timing has gone as far: Espiritu and Cash, following earlier work [38, 129], recover records from PostgreSQL row-level security's timing, though they could not get as reliable a timing difference against document-level security [49]. Our latency channels have another source, the layer's widening loop and pgvector's iterative scan, neither paper tests vector search, and our setting cannot show whether these channels survive a network, as theirs did with more queries. An open-source project, not peer reviewed, reports channels of filter-after-retrieval RAG in process [117]. Appendix L.2 gives the comparison.

Padding (H6) and costs. No padder ran. Padded latency was computed afterwards from the recorded latencies, as the next multiple of a budget set to the slowest latency of the first seed, so any latency under the budget maps to the budget and the padded AUC of 0.500 in all seven cells with a forward channel holds by construction; the rule's "confirmed" says little. Reverse channels fell outside the rule. What the computation does give is the cost: padding raised mean latency 5.2 to 6.7 times in those configurations. Exact search cost 1.53, 2.01 and 8.10 times the mean latency of the approximate pre-filter on Qdrant, Elasticsearch and pgvector.

In sum, pooled over attackers, the largest channels at $m = 1{,}000$ were keyword scores ($|\mathrm{AUC} - 0.5|$ of 0.49 and 0.50), constant work on pgvector, in reverse (0.48; how much of it is build offset was not measured), and pgvector's approximate pre-filter (0.38); per attacker, the post-filter's latency and pgvector's pre-filter reached 0.50 at $p = 10\%$ and 50%, and the count 0.50 at $p = 1\%$. On the shared index, only searches that ran exactly, whether requested or chosen by the planner (Qdrant's

small filters), returned the same answer in both worlds; a separated index does so by construction. Appendix L.2 gives the full results, costs and the build-offset forensics.

9 End-to-End Evaluation With Canaries

Here the whole pipeline runs on one synthetic scenario: many tenants and people in one index, fixed red-team prompts, and a language model that answers from what retrieval returns. The goal is a leak test, not a measure of answer quality.

Setup. The index holds 107,540 chunks: 49,920 synthetic company documents in 20 tenants, of which 41,600 carry a unique canary string; 20 notices, each carrying a prompt injection; and 57,600 FiQA passages [85] that every tenant may read. 189 people, the scenario's subjects, are 13 role types repeated across the tenants: a typical employee may read 58,641 chunks, the least privileged 624, and one person nothing. They ask 1,105 questions: 108 generic, 376 targeted at a document they may read, 189 at a document they may not, and 432 red-team prompts. The policy, a tenant boundary and per-document ACLs, runs in ten configurations (FAISS over a flat index, the eight approximate stores of Figure 1, and pgvector with the policy written in OPA) and in an eleventh, with the ACLs as 74,960 OpenFGA tuples, which ran in four further runs after a harness fault of ours stopped it from starting. The scenario ran four times, with seeds 7, 11, 13 and 17, on release 2.1.0, with the same recorded software in every run but one client library (Appendix M.1). The seed-7 run was executed twice with the same notebook, on 2026-10-24 and on 2026-11-07; the artifact holds the second execution, every seed-7 number here comes from it, and the pre-registrations of the repeats and of the diagnosis runs quote the first execution's printed aggregates (Appendix M.1). The repeats and their predictions were pre-registered after the first execution and before any repeat. Permissions did not change during a run, so the revision a check read makes no difference here: these runs test containment, not freshness.

Safety. No configuration in any run returned an unreadable chunk or exposed a canary (Table 17 in Appendix M.2). How far that generalizes depends on what counts as a trial. Of each run's 1,105 (person, question) pairs, 216 use the same text in every seed, with only the documents under them changing: the 108 generic questions, which are 12 strings asked by nine people, and the 108 prompts of the two attacks that have no placeholder. The others are drawn per seed from document templates and attacks, with 208 entity names per template, and 26 of them recur by chance in a second seed. The runs' stored questions thus hold 3,746 distinct pairs, not 4,420, the same for every configuration, with 2,381 distinct question strings, and 21 questions per run come from the one person who may read nothing; taking those as independent, $1 - 0.05^{1/3746} \approx 8.0 \times 10^{-4}$ per question (one-sided 95%), separately for each of the eleven configurations. Per person the bound stays at a single run's $1 - 0.05^{1/189} \approx 1.6 \times 10^{-2}$, because the same people recur in every run with the same permissions. A fault tied to a permission shape would recur too, and the 189 people have 13 shapes, for which the same formula gives 0.21. As a secondary count the runs checked 784,028 hits, which are not independent trials. All of these bounds are over this question generator and say nothing about an adversary who adapts its queries; none was run, and for such an adversary we rely only on Observation 1 and its trusted computing base.

No violation occurred either, so in 784,028 hits the re-check dropped nothing, and these runs would look the same with it off. What they test is that stores round-trip metadata and that implementations of one predicate agree: in the in-process configurations filter and re-check evaluate the same ACL metadata, with OPA and OpenFGA they come from the engine's separate APIs, and the oracle that judges a leak is a separate plain implementation from the same source as the layer.

These numbers have power only if the questions came near content the asking person may not read, and if the pipeline's checks fire when enforcement is broken. A second amendment to the pre-registration, exploratory in that it was specified in writing after the main results and before running, measures both on FAISS (Appendix M.3). Over the four seeds, 95.2% of the 4,420 questions had an unreadable chunk in the unfiltered top 5, mostly through the copies in other tenants: inside the tenant, on seed 7, the share was 47.8%, and for targeted questions 0.5%, so those test the tenant boundary and hardly the ACL. Red-team prompts reached 87.7%, which falsifies our prediction of at least 90%. The control, on seed 7, breaks the pushed-down filter in three ways (it admits everything, also admits the neighboring tenant, or drops the ACL clause) with the re-check off and on. With it off, an unreadable hit reached the top 10 for 1,103, 751 and 533 of the 1,105 questions and all four of the harness's leak checks failed; the answer itself held a canary for only 138, 13 and 107. With it on, no unreadable hit came back: the re-check reported each as a violation (15,317, 4,834 and 7,642) and the layer marked those questions incomplete, while recall stayed where the broken filter left it (0.077, 0.703 and 0.548 at $k = 10$). So the re-check contains such a fault and reports it without repairing it.

This is the only end-to-end evidence that the re-check does anything, and it is narrow: one seed, one exact store, the filter mutated only, and since the check is untouched, containment there holds by construction, so its value is that it validates the harness. Nothing inside the trusted computing base was mutated, no wrong filter met an approximate index, and the red-team prompts change only the query text.

Containment held while quality did not. pgvector gave empty answers to 0, 1, 0 and 7 questions from people who could read something in the four runs, on 0, 1, 0 and 7 people, all least-privileged and each able to read 624 chunks. Each of them asked five questions and got one empty answer, so nobody who could read something received no results at all. The release reported those empty answers as complete. Our prediction that 3 to 25 people would receive no results in every repeat, scored by the harness on this count of answers, was falsified, because it took the ten empty answers of the seed-7 run's first execution for a property of the configuration; the other four predictions held. Over its eight runs the configuration ran exact in five and, by its median, fast in three, with at least one empty answer in four of the eight (0, 1, 0, 7, 1, 1, 0, 0); the one empty answer in each of two exact runs we have not explained. With OpenFGA, pgvector's recall@10 was between 0.693 and 0.758 in every run, and Qdrant's between 0.530 and 0.561. The 3B model's answer carried the marker the injected notice asks for in all 3,239 retrievals of it, which is outside our threat model and marks where a retrieval guarantee ends (Appendix M.4).

10 Related Work

Permission-aware retrieval draws on literatures that rarely cite one another; each settled part of the design we test, and we found none that tests the whole. Our statements that we found no prior work report searches, which are not exhaustive. Appendix N gives the full account, including native store features and commercial systems such as Amazon Bedrock's knowledge bases [3], for which we found no independent evaluation of permission correctness.

Enterprise search and databases. Büttcher and Clarke gave the founding security model for multi-user full-text search: results ranked with global statistics reveal unreadable files, and query integration fixes it [23]. Bailey et al. modeled late binding and fetched growing blocks of candidates [18]. The pre-filter we test is query integration applied to approximate search, and its widening post-filter is their loop; we claim neither. Query modification [150], Truman semantics [137]

and the sound, secure and maximum criteria [167] frame what a pushed-down filter does and owes. Zerber [183] and searchable encryption [149] protect an index from an untrusted server, whereas we trust the store's host. This lineage gives criteria, not a general proof: Wang et al. note that the straightforward extension of authorization-view query modification to queries with negation is not sound, and give a sound and secure algorithm for cell-level policies under exact relational evaluation. We test translators and approximate indexes, for which we found no such proof.

Authorization. In Zanzibar, answering a search query requires ACL checks for all candidate results, Expand lets clients build indexes for access-controlled content, and snapshot tokens guard against applying an old ACL to new content [118]; its descendants added reverse lookups with documented size limits [13, 110], and policy engines compile filters through engineering artifacts [28, 104]. Margrave [52], Zelkova [17] and Cedar's differential testing [42] address the policy and its check path. Cedar's public specification also proves its type-aware partial evaluator sound in Lean [27], but for residuals against concrete authorization, not a store's filter language, and the Cedar adapter we test does not use it. We found no peer-reviewed semantics or correctness proof for any policy-to-filter translator.

Filtered search and permission-aware indexes. The collapse of post-filtering is well measured [73, 79, 120]. Index designs for attribute-filtered search [24, 57, 166] take the predicate as part of the query rather than compile it from an external policy; benchmarks control or measure selectivity and correlation [2, 188], and HCBGen shows that neither alone is a stable proxy for difficulty [81]. None of these uses real ACLs, and neither do we. Permission-specific indexes are recent [63, 189], and a vision paper says Wang et al.'s criteria must be extended to vector search and compares enforcement strategies' recall on pgvector [180]. We propose no index. We found no evaluation of per-query disjunctions over hundreds or thousands of group labels, and no audit of production stores under such filters at documented, tuned and exact settings.

RAG privacy and agents. Anyone who can query a RAG system can drain its store [128, 182], and model-side defenses are probabilistic [20]. Attribute-based gating at retrieval [5] and a unified PostgreSQL data layer [22] are reported to remove cross-tenant leakage, each on one system. Corpus poisoning [191] and information-flow control for agents [33] are complementary. We found no oracle-based test across stores and engines.

Side channels and testing. Our game tests a restricted form of noninterference [56]. Timing has long revealed hidden content on the web [51, 54], and padding is the simplest case of predictive mitigation [6]. On access control over shared indexes the closest work is stronger than ours, on keyword search and relational predicates [38, 49, 129, 165]; Section 8.2 relates our results to it. Database engines have mature logic-bug oracles [134, 136]; for vector databases a testing roadmap reports only simplified metamorphic cases [168], and we found none for filter semantics [178]. Our testing is differential in McKeeman's sense [89], and it evaluates its relations under exact search, so that approximation cannot confound them.

11 Limitations, Ethics and Reproducibility

Limitations. Appendix O states these in full. This is a measurement of one layer, developed by the author and created in October 2026, with no comparator layer and no adaptive adversary run, on synthetic permission layouts. That no returned chunk leaked means that none lay outside the readable set at the revision the check read, under the stated trusted computing base. It does not mean fresh, nor that unreadable chunks leave no trace, and the detection fields of Section 7

are observables we did not measure. The enterprise layout has 19 near-identical copies of each company document, and Qdrant's recall depends on that layout, so these recall numbers should not be read as an estimate for an organization without such copies. Every measurement passes through one layer, and some numbers are properties of its schema as much as of a store. No single version of the library was evaluated throughout: the end-to-end numbers are for release 2.1.0, which has known defects, the fixes we report were tested piecemeal and not end to end, and PostgreSQL, the embedding dimension and store versions differ across experiments (Appendix A). PostgreSQL row-level security was measured only in a side run (Section 6.4). No reported run exercised the token path or the MCP tool: the harnesses build the subject directly, so that no text can choose the subject is a design argument. We did not measure document-level security in Elasticsearch and OpenSearch, sweep the share of near-duplicate chunks or rerun the evaluation on one release, and outside the 100% selectivity cells we have no unfiltered baseline per store. Zero-leak bounds treat repeated cases as independent, which the design supports only in part; we apply no correction for multiple comparisons across cells. The system under test, its oracles, the meta-tests that judge the oracles and the scripts that compute the verdicts share one source, so a misconception common to them would pass every check, a common-mode risk that repeated runs cannot remove, and no independent party has rerun anything. Three things limit it, without removing it: the oracles were written without the library's filter code, the vendor reproducers use only the vendors' clients, and we release outputs so that others can recompute verdicts, with the gaps listed below.

Ethics and disclosure. The author develops WARDOWL, the layer under test, at `https://github.com/muzakkirhussain011/wardowl`, the repository that the pull-request numbers in this paper refer to, and this evaluation found faults in that code as well as in others' (Table 1). No human subjects took part: people, documents, ACLs and canaries are synthetic, and every store and engine ran in containers or processes we controlled. None of our findings is a chunk returned through the layer outside the readable set at the checked revision. They can still matter for deployments that trust a pushed-down filter without a re-check, and Chroma's crash lets any user in enough groups stop an application that passes their groups to Chroma in local mode (Appendix O.3). The behaviors were public before we reported them (times in UTC). The campaign's Redis misses, without their cause, were on the public Actions page of its run from 2026-10-26, and Chroma's crash on a public results branch from 2026-10-29 16:59 (a smoke run; the full run followed at 22:48). Pull request #167, opened 2026-10-29 16:00 and merged 2026-10-30 02:16, describes the cut on Redis and MongoDB; #187, opened 2026-10-30 00:00, Chroma's crash; and #188, opened 11:41, MongoDB's filter field. On 2026-10-30 we then filed reports with minimal reproducers that do not need our library: Redis (`https://github.com/RediSearch/RediSearch/issues/12106`, 15:54), Chroma (`https://github.com/chroma-core/chroma/issues/7969`, `https://github.com/chroma-core/chroma/issues/7970`, 15:56), Cerbos (`https://github.com/cerbos/cerbos/issues/3456`, 19:15) and MongoDB (documentation feedback, no public link). So these vendors heard after the public could have, including of a crash that users can reach and of a filter that can admit too much, filed as ordinary public issues. We found no security policy file in Chroma's repository. RediSearch's policy routes serious vulnerabilities privately; we filed publicly because the over-admission needs a value longer than 4,096 bytes and our pull request had already described it, which was our judgment, and maintainers may disagree. Our first reports to Redis, Chroma and MongoDB said the maintainers were hearing of this first, which was wrong; we corrected the three public issues on 2026-11-01, and MongoDB's form cannot be edited. Table 7 in Appendix F gives each report's status. One unexplained SpiceDB trial past its documented bound, OPA's residual negation and pgvector's plan choice have no report. On 2026-11-02 a Cerbos developer replied on its issue, asking what the report was asking and noting that the plan's expression is valid when the attribute is a

list. No maintainer had replied on the other three public issues by then, though the Redis issue was reassigned on 2026-11-01. On 2026-10-31 someone outside the project opened a pull request against the first Chroma issue (chroma-core/chroma#7974), and on 2026-11-01 another offered to work on the second.

Use of AI assistance. Under the author's direction, an AI coding agent produced most of the library's code, the conformance kit, the oracles, the harnesses and the analysis scripts, drafted the pre-registrations, the evidence summaries and this text, and checked the references against primary records; the numbers audit and the critical review cited here were run by separate instances of it. The author chose the research questions, took the decisions recorded with the pre-registrations and reviews, and takes full responsibility for the content.

Reproducibility. The artifact is at `https://github.com/muzakkirhussain011/wardowl-paper-artifact`, tag `evidence-v1`, archived at `https://doi.org/10.5281/zenodo.23874614`. It holds the source of this text as of the tag, the pre-registrations with their dated amendments, the raw outputs with seeds and versions and, for most GitHub jobs, container image digests (X2 and X5 record image tags and server versions instead), the vendor reproducers, the Kaggle runners, and the library at the commits the runs used; `REPRODUCE.md` maps each table to its data and `evidence/ARTIFACT_GAPS.md` lists what is missing. The main gaps: the end-to-end runs stored answers and hits at $k = 5$ only, so their recall@10, and the check of the other hits against the oracle, are printed counters; the Kaggle runs did not pin their Python dependencies; and no rebuild of the prepared corpora has been shown. Compute: about 38 hours of Kaggle sessions (two T4 GPUs or four CPUs each) and more than 140 hours of GitHub-hosted runner jobs, listed per experiment in Appendix Q; no experiment reported here ran on a local machine. Data and model licenses are in Appendix O.2.

12 Conclusion

We measured one permission-aware retrieval layer, developed by the author, as a composition of untrusted parts, translators, approximate indexes and filter APIs, around a trusted computing base centered on the check API. Outside positive controls built to leak, in what we tested no returned chunk lay outside the readable set at the revision the check read: none in eleven end-to-end configurations (8.0×10^{-4} per question and 1.6×10^{-2} per person, one-sided 95%, under the independence assumptions of Section 9) and none in 1,050 fault episodes, beside 40 control episodes that turned denials into allows and leaked, as they must. Those end-to-end runs had no violations, so they do not show the re-check doing anything; end to end it was exercised only when we broke the filter on purpose, on one seed and an exact store, where the leak checks failed and the re-check contained the fault. No adaptive adversary was run; that containment does not depend on the filter or the index is an argument under stated assumptions, not a measurement.

Containment is not all that security asks. The checked revision can be stale, and in the code we ran revoked chunks were served for up to 38.4 s after an invalidation. In forced-choice tests, latency, counts, recall and scores told a subject which of two worlds held a group of unreadable chunks that included the query's nearest passages. And the untrusted parts still decide how much of what a subject may read it receives: on our synthetic layouts stores dropped matching chunks at size limits, filtered approximate search lost much of a readable set far from the question, and the same pgvector configuration was exact in some runs and lost about a quarter of its recall in others. That loss is not a cost of enforcement as such: in a side run on another corpus, PostgreSQL row-level security kept recall@10 at 0.998 or better, with median latency growing with the readable share to 608 ms.

No count sees a full answer of the wrong chunks.

On this evidence we suggest three practices. Test filter semantics differentially, and measure recall under permission-shaped filters against exact search over the readable set before choosing a store, a layout or a plan. Do not let a system call a short approximate answer complete: count, fall back to exact search within a bound, and sample exact searches. Treat the shape and timing of an answer as outputs too, and the staleness of a check as part of the claim.

References

[1] Rishabh Agarwal, Max Schwarzer, Pablo Samuel Castro, Aaron Courville, and Marc Bellemare. Deep reinforcement learning at the edge of the statistical precipice. In *Advances in Neural Information Processing Systems 34 (NeurIPS 2021)*, volume 34, pages 29304–29320. Curran Associates, Inc., 2021. URL `https://proceedings.neurips.cc/paper_files/paper/2021/hash/f514cec81cb148559cf475e7426eed5e-Abstract.html`.

[2] Abylay Amanbayev, Brian Tsan, Tri Dang, and Florin Rusu. Filtered approximate nearest neighbor search in vector databases: System design and performance analysis, 2026.

[3] Amazon Web Services. Access control lists awareness enablement. Amazon Bedrock User Guide, 2026. URL `https://docs.aws.amazon.com/bedrock/latest/userguide/kb-managed-acl.html`. Accessed 2026-10-26.

[4] Sabrina Aquino and David Myriel. Filtering in vector search: When to use what. Qdrant articles, September 2024. URL `https://qdrant.tech/articles/vector-search-filtering/`.

[5] Francisco Javier Arceo and Varsha Prasad Narsing. Securing the agent: Vendor-neutral, multitenant enterprise retrieval and tool use. In *Proceedings of the ACM Conference on AI and Agentic Systems (CAIS '26)*, pages 862–872, 2026. doi: 10.1145/3786335.3813145.

[6] Aslan Askarov, Danfeng Zhang, and Andrew C. Myers. Predictive black-box mitigation of timing channels. In *Proceedings of the 17th ACM Conference on Computer and Communications Security (CCS '10)*, pages 297–307, 2010. doi: 10.1145/1866307.1866341.

[7] Martin Aumüller, Erik Bernhardsson, and Alexander Faithfull. ANN-Benchmarks: A benchmarking tool for approximate nearest neighbor algorithms. *Information Systems*, 87:101374, 2020. doi: 10.1016/j.is.2019.02.006.

[8] Auth0. Authorization for RAG. Auth0 Docs (Auth0 for AI Agents), 2026. URL `https://auth0.com/ai/docs/intro/authorization-for-rag`. Accessed 2026-10-26.

[9] AuthZed. Consistency. AuthZed (SpiceDB) documentation, 2026. URL `https://authzed.com/docs/spicedb/concepts/consistency`. Accessed 2026-10-26.

[10] AuthZed. core.proto (authzed.api.v1). GitHub repository authzed/api, 2026. URL `https://raw.githubusercontent.com/authzed/api/main/authzed/api/v1/core.proto`. Accessed 2026-10-30.

[11] AuthZed. langchain-spicedb: Authorization library for RAG (retrieval-augmented generation) pipelines using SpiceDB. GitHub repository authzed/langchain-spicedb, 2026. URL `https://github.com/authzed/langchain-spicedb`. Accessed 2026-10-26.

[12] AuthZed. permission_service.proto (authzed.api.v1). GitHub repository authzed/api, 2026. URL `https://raw.githubusercontent.com/authzed/api/main/authzed/api/v1/permission_service.proto`. Accessed 2026-10-30.

[13] AuthZed. Protecting a list endpoint. AuthZed (SpiceDB) documentation, 2026. URL `https://authzed.com/docs/spicedb/modeling/protecting-a-list-endpoint`. Accessed 2026-10-26.

[14] AuthZed. Reference: `spicedb`. AuthZed (SpiceDB) documentation, 2026. URL `https://authzed.com/docs/spicedb/concepts/commands`. Accessed 2026-10-30.

[15] Jinsheng Ba and Manuel Rigger. Testing database engines via query plan guidance. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*, pages 2060–2071, 2023. doi: 10.1109/ICSE48619.2023.00174.

[16] Jinsheng Ba and Manuel Rigger. Keep it simple: Testing databases via differential query plans. *Proceedings of the ACM on Management of Data*, 2(3):1–26, 2024. doi: 10.1145/3654991.

[17] John Backes, Pauline Bolignano, Byron Cook, Catherine Dodge, Andrew Gacek, Kasper Luckow, Neha Rungta, Oksana Tkachuk, and Carsten Varming. Semantic-based automated reasoning for AWS access policies using SMT. In *2018 Formal Methods in Computer Aided Design (FMCAD)*, pages 1–9, 2018. doi: 10.23919/FMCAD.2018.8602994.

[18] Peter Bailey, David Hawking, and Brett Matson. Secure search in enterprise webs: Tradeoffs in efficient implementation for document level security. In *Proceedings of the 15th ACM International Conference on Information and Knowledge Management (CIKM '06)*, pages 493–502, Arlington, Virginia, USA, 2006. doi: 10.1145/1183614.1183686.

[19] Payal Bajaj, Daniel Campos, Nick Craswell, Li Deng, Jianfeng Gao, Xiaodong Liu, Rangan Majumder, Andrew McNamara, Bhaskar Mitra, Tri Nguyen, Mir Rosenberg, Xia Song, Alina Stoica, Saurabh Tiwary, and Tong Wang. MS MARCO: A human generated MAchine reading COmprehension dataset, 2016.

[20] Shashank Shreedhar Bhatt, Tanmay Rajore, Khushboo Aggarwal, Ganesh Ananthanarayanan, Ranveer Chandra, Nishanth Chandran, Suyash Choudhury, Divya Gupta, Emre Kiciman, Sumit Kumar Pandey, Srinath Setty, Rahul Sharma, and Teijia Zhao. Enterprise AI must enforce participant-aware access control, 2025.

[21] Xavier Bouthillier, Pierre Delaunay, Mirko Bronzi, Assya Trofimov, Brennan Nichyporuk, Justin Szeto, Nazanin Mohammadi Sepahvand, Edward Raff, Kanika Madan, Vikram Voleti, Samira Ebrahimi Kahou, Vincent Michalski, Tal Arbel, Chris Pal, Gael Varoquaux, and Pascal Vincent. Accounting for variance in machine learning benchmarks. In *Proceedings of Machine Learning and Systems (MLSys 2021)*, volume 3, pages 747–769, 2021. URL `https://proceedings.mlsys.org/paper_files/paper/2021/hash/0184b0cd3cfb185989f858a1d9f5c1eb-Abstract.html`.

[22] Venkata Krishna Prasanth Budigi and Siri Chandana Sirigiri. Beyond similarity search: A unified data layer for production RAG systems, 2026.

[23] Stefan Büttcher and Charles L. A. Clarke. A security model for full-text file system search in multi-user environments. In *Proceedings of the 4th USENIX Conference on File and Storage Technologies (FAST '05)*, pages 169–182. USENIX Association, 2005. URL `https://www.usenix.org/legacy/events/fast05/tech/buettcher.html`.

[24] Yuzheng Cai, Jiayang Shi, Yizhuo Chen, and Weiguo Zheng. Navigating labels and vectors: A unified approach to filtered approximate nearest neighbor search. *Proceedings of the ACM on Management of Data*, 2(6):1–27, 2024. doi: 10.1145/3698822.

[25] Nicholas Carlini, Chang Liu, Úlfar Erlingsson, Jernej Kos, and Dawn Song. The secret sharer: Evaluating and testing unintended memorization in neural networks. In *28th USENIX Security Symposium (USENIX Security 19)*, pages 267–284, Santa Clara, CA, 2019. USENIX Association. ISBN 978-1-939133-06-9. URL `https://www.usenix.org/conference/usenixsecurity19/presentation/carlini`.

[26] Cedar Contributors. cedar_policy crate documentation, version 4.14.0. docs.rs, 2026. URL `https://docs.rs/cedar-policy/4.14.0/cedar_policy/`. Accessed 2026-10-31.

[27] Cedar Contributors. cedar-lean/Cedar/Thm/TPE.lean: type-aware partial evaluation soundness. cedar-policy/cedar-spec repository on GitHub, 2026. URL `https://github.com/cedar-policy/cedar-spec/blob/main/cedar-lean/Cedar/Thm/TPE.lean`. Accessed 2026-10-31.

[28] Cerbos. The Cerbos API. Cerbos documentation (PlanResources / resources query plan), 2026. URL `https://docs.cerbos.dev/cerbos/latest/api/index.html`. Accessed 2026-10-26.

[29] Cerbos. Implementing authorization in RAG-based AI systems. Cerbos documentation (recipes), 2026. URL `https://docs.cerbos.dev/cerbos/latest/recipes/ai/rag-authorization/`. Accessed 2026-10-26.

[30] Tsong Yueh Chen, Fei-Ching Kuo, Huai Liu, Pak-Lok Poon, Dave Towey, T. H. Tse, and Zhi Quan Zhou. Metamorphic testing: A review of challenges and opportunities. *ACM Computing Surveys*, 51(1):1–27, 2018. doi: 10.1145/3143561.

[31] Yannis Chronis, Helena Caminal, Yannis Papakonstantinou, Fatma Özcan, and Anastasia Ailamaki. Filtered vector search: State-of-the-art and research opportunities. *Proceedings of the VLDB Endowment*, 18(12):5488–5492, 2025. doi: 10.14778/3750601.3750700.

[32] Koen Claessen and John Hughes. QuickCheck: A lightweight tool for random testing of Haskell programs. In *Proceedings of the Fifth ACM SIGPLAN International Conference on Functional Programming (ICFP '00)*, pages 268–279, 2000. doi: 10.1145/351240.351266.

[33] Manuel Costa, Boris Köpf, Aashish Kolluri, Andrew Paverd, Mark Russinovich, Ahmed Salem, Shruti Tople, Lukas Wutschitz, and Santiago Zanella-Béguelin. Securing AI agents with information-flow control, 2025.

[34] Dylan Couzon and Meina Ghafouri. Filtered vector search: What ACORN fixes, and what fixes ACORN. Qdrant article, August 2026. URL `https://qdrant.tech/articles/filtered-vector-search-acorn/`.

[35] Coveo. Coveo, Microsoft Dynamics 365, and security. Coveo Platform documentation, 2026. URL `https://docs.coveo.com/en/448/`. Accessed 2026-10-26.

[36] Reza Curtmola, Juan Garay, Seny Kamara, and Rafail Ostrovsky. Searchable symmetric encryption: Improved definitions and efficient constructions. In *Proceedings of the 13th ACM Conference on Computer and Communications Security (CCS '06)*, pages 79–88, 2006. doi: 10.1145/1180405.1180417.

[37] Joseph W. Cutler, Craig Disselkoen, Aaron Eline, Shaobo He, Kyle Headley, Michael Hicks, Kesha Hietala, Eleftherios Ioannidis, John Kastner, Anwar Mamat, Darin McAdams, Matt McCutchen, Neha Rungta, Emina Torlak, and Andrew M. Wells. Cedar: A new language for expressive, fast, safe, and analyzable authorization. *Proceedings of the ACM on Programming Languages*, 8(OOPSLA1):670–697, 2024. doi: 10.1145/3649835.

[38] Chen Dar, Moshik Hershcovitch, and Adam Morrison. RLS side channels: Investigating leakage of row-level security protected data through query execution time. *Proceedings of the ACM on Management of Data*, 1(1):1–25, 2023. doi: 10.1145/3588943.

[39] Namrata Das, Kailas Devadkar, and Anand Godbole. Recall-bounded staleness for filtered multi-tenant ANN graph indexes under concurrent updates, 2026.

[40] DataStax. Query timestamps in Cassandra drivers. DataStax driver documentation, 2026. URL `https://docs.datastax.com/en/datastax-drivers/developing/query-timestamps.html`. Accessed 2026-10-30.

[41] Edoardo Debenedetti, Ilia Shumailov, Tianqi Fan, Jamie Hayes, Nicholas Carlini, Daniel Fabian, Christoph Kern, Chongyang Shi, Andreas Terzis, and Florian Tramèr. Defeating prompt injections by design, 2025.

[42] Craig Disselkoen, Aaron Eline, Shaobo He, Kyle Headley, Michael Hicks, Kesha Hietala, John Kastner, Anwar Mamat, Matt McCutchen, Neha Rungta, Bhakti Shah, Emina Torlak, and Andrew Wells. How we built Cedar: A verification-guided approach. In *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering (FSE Companion '24)*, pages 351–357, 2024. doi: 10.1145/3663529.3663854.

[43] Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou. The Faiss library, 2024.

[44] Elastic. Controlling access at the document and field level. Elastic Documentation, 2026. URL `https://www.elastic.co/docs/deploy-manage/users-roles/cluster-or-deployment-auth/controlling-access-at-document-field-level`. Accessed 2026-10-26.

[45] Elastic. ignore_above. Elasticsearch Reference, mapping parameters, 2026. URL `https://www.elastic.co/docs/reference/elasticsearch/mapping-reference/ignore-above`. Accessed 2026-10-30.

[46] Elastic. Keyword type family. Elasticsearch reference documentation, 2026. URL `https://www.elastic.co/docs/reference/elasticsearch/mapping-reference/keyword`. Accessed 2026-10-30.

[47] Elastic. Run a search. Elasticsearch API documentation, 2026. URL `https://www.elastic.co/docs/api/doc/elasticsearch/operation/operation-search`. Accessed 2026-10-30.

[48] Joshua Engels, Ben Landrum, Shangdi Yu, Laxman Dhulipala, and Julian Shun. Approximate nearest neighbor search with window filters. In *Proceedings of the 41st International Conference on Machine Learning (ICML 2024)*, volume 235 of *Proceedings of Machine Learning Research*, pages 12469–12490. PMLR, 2024. URL `https://proceedings.mlr.press/v235/engels24a.html`.

[49] Zachary Espiritu and David Cash. Plaintext recovery against post-filtering access control. In *35th USENIX Security Symposium (USENIX Security 26)*, pages 3753–3772, 2026. URL `https://www.usenix.org/conference/usenixsecurity26/presentation/espiritu`.

[50] Ronald Fagin, Ravi Kumar, Kevin S. McCurley, Jasmine Novak, D. Sivakumar, John A. Tomlin, and David P. Williamson. Searching the workplace web. In *Proceedings of the Twelfth International Conference on World Wide Web (WWW '03)*, 2003. doi: 10.1145/775152.775204.

[51] Edward W. Felten and Michael A. Schneider. Timing attacks on Web privacy. In *Proceedings of the 7th ACM Conference on Computer and Communications Security (CCS '00)*, pages 25–32, 2000. doi: 10.1145/352600.352606.

[52] Kathi Fisler, Shriram Krishnamurthi, Leo A. Meyerovich, and Michael Carl Tschantz. Verification and change-impact analysis of access-control policies. In *Proceedings of the 27th International Conference on Software Engineering (ICSE '05)*, 2005. doi: 10.1145/1062455.1062502.

[53] Philip W. L. Fong. Relationship-based access control: Protection model and policy language. In *Proceedings of the First ACM Conference on Data and Application Security and Privacy (CODASPY '11)*, pages 191–202, 2011. doi: 10.1145/1943513.1943539.

[54] Nethanel Gelernter and Amir Herzberg. Cross-site search attacks. In *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security (CCS '15)*, pages 1394–1405, 2015. doi: 10.1145/2810103.2813688.

[55] gelevanog. permission-aware-rag ("clearance"): private company RAG that answers each employee only from documents they may read. GitHub repository gelevanog/permission-aware-rag, 2026. URL `https://github.com/gelevanog/permission-aware-rag`. Accessed 2026-10-26.

[56] J. A. Goguen and J. Meseguer. Security policies and security models. In *1982 IEEE Symposium on Security and Privacy*, 1982. doi: 10.1109/SP.1982.10014.

[57] Siddharth Gollapudi, Neel Karia, Varun Sivashankar, Ravishankar Krishnaswamy, Nikit Begwani, Swapnil Raz, Yiyong Lin, Yin Zhang, Neelam Mahapatro, Premkumar Srinivasan, Amit Singh, and Harsha Vardhan Simhadri. Filtered-DiskANN: Graph algorithms for approximate nearest neighbor search with filters. In *Proceedings of the ACM Web Conference 2023 (WWW '23)*, pages 3406–3416, 2023. doi: 10.1145/3543507.3583552.

[58] Google. Atheris. GitHub repository google/atheris, 2026. URL `https://github.com/google/atheris`. Accessed 2026-10-26.

[59] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec '23)*, pages 79–90, 2023. doi: 10.1145/3605764.3623985.

[60] Patricia P. Griffiths and Bradford W. Wade. An authorization mechanism for a relational database system. *ACM Transactions on Database Systems*, 1(3):242–255, 1976. doi: 10.1145/320473.320482.

[61] Marco Guarnieri and David Basin. Optimal security-aware query processing. *Proceedings of the VLDB Endowment*, 7(12):1307–1318, 2014. doi: 10.14778/2732977.2733003.

[62] Gaurav Gupta, Jonah Yi, Benjamin Coleman, Chen Luo, Vihan Lakshman, and Anshumali Shrivastava. CAPS: A practical partition index for filtered similarity search, 2023.

[63] Shanshan Han, Vishal Chakraborty, and Sharad Mehrotra. Don't stir the pot! Authorized vector data retrieval via access-aware indexing, 2026.

[64] James A. Hanley and A. Lippman-Hand. If nothing goes wrong, is everything all right? Interpreting zero numerators. *JAMA*, 249(13):1743–1745, 1983. doi: 10.1001/jama.1983.03330370053031.

[65] Norm Hardy. The confused deputy: (or why capabilities might have been invented). *ACM SIGOPS Operating Systems Review*, 22(4):36–38, 1988. doi: 10.1145/54289.871709.

[66] Patrick Iff, Paul Brügger, Marcin Chrapek, David Kochergin, Maciej Besta, and Torsten Hoefler. Benchmarking filtered approximate nearest neighbor search algorithms on transformer-based embedding vectors. In *Proceedings of the 49th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '26)*, pages 610–622, 2026. doi: 10.1145/3805712.3809731.

[67] Yannis E. Ioannidis, Raymond T. Ng, Kyuseok Shim, and Timos K. Sellis. Parametric query optimization. *The VLDB Journal*, 6(2):132–151, 1997. doi: 10.1007/s007780050037.

[68] Bargav Jayaraman, Virendra J. Marathe, Hamid Mozaffari, William F. Shen, and Krishnaram Kenthapadi. Permissioned LLMs: Enforcing access control in large language models. In *Advances in Neural Information Processing Systems 38 (NeurIPS 2025)*, pages 90674–90704, 2025. doi: 10.52202/085713-2731.

[69] Yicheng Jin, Yongji Wu, Wenjun Hu, Bruce M. Maggs, Xiao Zhang, and Danyang Zhuo. Curator: Efficient indexing for multi-tenant vector databases, 2024.

[70] M. Jones, A. Nadalin, B. Campbell, J. Bradley, and C. Mortimore. OAuth 2.0 token exchange. Request for Comments 8693, Internet Engineering Task Force, January 2020. URL `https://www.rfc-editor.org/rfc/rfc8693`.

[71] Govind Kabra, Ravishankar Ramamurthy, and S. Sudarshan. Redundancy and information leakage in fine-grained access control. In *Proceedings of the 2006 ACM SIGMOD International Conference on Management of Data (SIGMOD '06)*, pages 133–144, 2006. doi: 10.1145/1142473.1142489.

[72] Matteo Kamm, Manuel Rigger, Chengyu Zhang, and Zhendong Su. Testing graph database engines via query partitioning. In *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA '23)*, pages 140–149, 2023. doi: 10.1145/3597926.3598044.

[73] Yoonseok Kim and Gyusik Choe. RACORN-1: Adaptive recall-preserving speedup for low-selectivity filtered vector search, 2026.

[74] George Klees, Andrew Ruef, Benji Cooper, Shiyi Wei, and Michael Hicks. Evaluating fuzz testing. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security (CCS '18)*, pages 2123–2138, 2018. doi: 10.1145/3243734.3243804.

[75] LangChain. pebblo_retrieval (langchain_community). LangChain Reference, 2026. URL https://reference.langchain.com/python/langchain-community/chains/pebblo_retrieval. Accessed 2026-10-26.

[76] Kristen LeFevre, Rakesh Agrawal, Vuk Ercegovac, Raghu Ramakrishnan, Yirong Xu, and David DeWitt. Limiting disclosure in Hippocratic databases. In *Proceedings 2004 VLDB Conference*, pages 108–119, 2004. doi: 10.1016/B978-012088469-8.50013-9.

[77] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[78] Haoran Li, Mingshi Xu, and Yangqiu Song. Sentence embedding leaks more information than you expect: Generative embedding inversion attack to recover the whole sentence. In *Findings of the Association for Computational Linguistics: ACL 2023*, pages 14022–14040, 2023. doi: 10.18653/v1/2023.findings-acl.881.

[79] Mocheng Li, Xiao Yan, Baotong Lu, Yue Zhang, James Cheng, and Chenhao Ma. Attribute filtering in approximate nearest neighbor search: An in-depth experimental study. *Proceedings of the ACM on Management of Data*, 3(6):1–26, 2025. doi: 10.1145/3769763.

[80] Zhaoheng Li, Silu Huang, Wei Ding, Yongjoo Park, and Jianjun Chen. SIEVE: Effective filtered vector search with collection of indexes. *Proceedings of the VLDB Endowment*, 18(11): 4723–4736, 2025. doi: 10.14778/3749646.3749725.

[81] Mintaek Lim, Dogeun Kim, Minwoo Kim, and Jaeyoung Do. Revisiting filtered ANN benchmarks: A hardness-controlled benchmark generator for realistic evaluation, 2026.

[82] Peiyang Liu, Xi Wang, Di Liang, Youru Li, Zhi Cai, and Wei Ye. Protection is not one event: A survey of RAG protection research by measured endpoint, 2026.

[83] Duo Lu, Helena Caminal, Manos Chatzakis, Yannis Papakonstantinou, Yannis Chronis, Vaibhav Jain, and Fatma Özcan. An in-depth study of filter-agnostic vector search on a PostgreSQL database system: [experiments & analysis]. *Proceedings of the ACM on Management of Data*, 4(3):1–26, 2026. doi: 10.1145/3802011.

[84] David MacIver, Zac Hatfield-Dodds, and Many Contributors. Hypothesis: A new approach to property-based testing. *Journal of Open Source Software*, 4(43):1891, 2019. doi: 10.21105/jos s.01891.

[85] Macedo Maia, Siegfried Handschuh, André Freitas, Brian Davis, Ross McDermott, Manel Zarrouk, and Alexandra Balahur. WWW'18 open challenge: Financial opinion mining and question answering. In *Companion Proceedings of The Web Conference 2018 (WWW '18)*, pages 1941–1942, 2018. doi: 10.1145/3184558.3192301.

[86] Yu. A. Malkov and D. A. Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 42(4):824–836, 2020. doi: 10.1109/TPAMI.2018.2889473.

[87] Nitin Mangtani. Document-level security with enterprise search. Official Google Cloud Blog, December 2006. URL https://cloud.googleblog.com/2006/12/document-level-security-with-enterprise.html.

[88] Evan Martin and Tao Xie. A fault model and mutation testing of access control policies. In *Proceedings of the 16th International Conference on World Wide Web (WWW '07)*, pages 667–676, 2007. doi: 10.1145/1242572.1242663.

[89] William M. McKeeman. Differential testing for software. *Digital Technical Journal*, 10(1): 100–107, 1998.

[90] Microsoft. Data, privacy, and security for Microsoft Copilot. Microsoft Learn, 2026. URL `https://learn.microsoft.com/en-us/copilot/microsoft-365/microsoft-365-copilot-privacy`. Accessed 2026-10-26.

[91] Microsoft. Query-time ACL and RBAC enforcement – Azure AI Search. Microsoft Learn, 2026. URL `https://learn.microsoft.com/azure/search/search-query-access-control-rbac-enforcement`. Accessed 2026-10-26.

[92] Milvus. Implement multi-tenancy. Milvus Documentation, v3.0.x, 2026. URL `https://milvus.io/docs/multi_tenancy.md`. Accessed 2026-10-26.

[93] Milvus. JSON field. Milvus Documentation, 2026. URL `https://milvus.io/docs/use-json-fields.md`. Accessed 2026-10-30.

[94] MITRE. ATLAS: Adversarial threat landscape for AI systems, data release 2026.09. `https://github.com/mitre-atlas/atlas-data`, 2026. URL `https://raw.githubusercontent.com/mitre-atlas/atlas-data/main/dist/v6/ATLAS-2026.09.yaml`.

[95] Model Context Protocol. Model context protocol specification, version 2026-07-28: Authorization. `https://modelcontextprotocol.io/specification/2026-07-28/basic/authorization`, 2026. URL `https://modelcontextprotocol.io/specification/2026-07-28/basic/authorization`.

[96] MongoDB. Configure `mongot`. Self-Managed MongoDB Search and Vector Search documentation, 2026. URL `https://www.mongodb.com/docs/search/self-managed/current/configuration/reference/`. Accessed 2026-10-30.

[97] MongoDB. Run vector search ANN and ENN queries. MongoDB Vector Search documentation (the `$vectorSearch` stage), 2026. URL `https://www.mongodb.com/docs/vector-search/query/aggregation-stages/vector-search-stage/`. Accessed 2026-10-30.

[98] John X. Morris, Volodymyr Kuleshov, Vitaly Shmatikov, and Alexander M. Rush. Text embeddings reveal (almost) as much as text. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 12448–12460, Singapore, 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.emnlp-main.765.

[99] A. Motro. An access authorization model for relational databases based on algebraic manipulation of view definitions. In *Proceedings of the Fifth International Conference on Data Engineering (ICDE 1989)*, pages 339–347, 1989. doi: 10.1109/ICDE.1989.47234.

[100] Ali Naseh, Yuefeng Peng, Anshuman Suri, Harsh Chaudhari, Alina Oprea, and Amir Houmansadr. Riddle me this! Stealthy membership inference for retrieval-augmented generation. In *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security (CCS '25)*, pages 1245–1259, 2025. doi: 10.1145/3719027.3744840.

[101] National Vulnerability Database. CVE-2025-32711. NVD vulnerability record, June 2025. URL https://nvd.nist.gov/vuln/detail/CVE-2025-32711.

[102] National Vulnerability Database. CVE-2025-65111: SpiceDB LookupResources results. NVD vulnerability record, November 2025. URL https://nvd.nist.gov/vuln/detail/CVE-2025-65111.

[103] OASIS. eXtensible Access Control Markup Language (XACML) version 3.0. Oasis standard, OASIS, January 2013. URL http://docs.oasis-open.org/xacml/3.0/xacml-3.0-core-spec-os-en.html.

[104] Open Policy Agent. Evaluating a data filter policy. Open Policy Agent documentation, 2026. URL https://www.openpolicyagent.org/docs/filtering/partial-evaluation. Accessed 2026-10-26.

[105] Open Policy Agent. Writing valid data filtering policies. Open Policy Agent documentation, 2026. URL https://openpolicyagent.org/docs/filtering/fragment. Accessed 2026-10-26.

[106] OpenFGA. OpenFGA configuration options. OpenFGA documentation, 2026. URL https://openfga.dev/docs/getting-started/setup-openfga/configuration. Accessed 2026-10-26.

[107] OpenFGA. openfga.proto (openfga.v1). GitHub repository openfga/api, 2026. URL https://raw.githubusercontent.com/openfga/api/main/openfga/v1/openfga.proto. Accessed 2026-10-30.

[108] OpenFGA. Perform a list objects call. OpenFGA documentation, 2026. URL https://openfga.dev/docs/getting-started/perform-list-objects. Accessed 2026-10-26.

[109] OpenFGA. RAG authorization. OpenFGA documentation, 2026. URL https://openfga.dev/docs/modeling/agents/rag-authorization. Accessed 2026-10-26.

[110] OpenFGA. Search with permissions. OpenFGA documentation, 2026. URL https://openfga.dev/docs/interacting/search-with-permissions. Accessed 2026-10-26.

[111] OpenSearch Project. Document-level security. OpenSearch Documentation, 2026. URL https://docs.opensearch.org/latest/security/access-control/document-level-security/. Accessed 2026-10-26.

[112] OpenSearch Project. Efficient k-NN filtering. OpenSearch Documentation, 2026. URL https://docs.opensearch.org/latest/vector-search/filter-search-knn/efficient-knn-filtering/. Accessed 2026-10-26.

[113] OpenSearch Project. Keyword field type. OpenSearch Documentation, 2026. URL https://docs.opensearch.org/latest/mappings/supported-field-types/keyword/. Accessed 2026-10-30.

[114] OpenSearch Project. Search API. OpenSearch Documentation, 2026. URL https://docs.opensearch.org/latest/api-reference/search-apis/search/. Accessed 2026-10-30.

[115] Oracle. Using Oracle virtual private database to control data access. Oracle Database Security Guide, 19c, 2026. URL https://docs.oracle.com/en/database/oracle/oracle-database/19/dbseg/using-oracle-vpd-to-control-data-access.html. Accessed 2026-10-26.

[116] OWASP Foundation. LLM08:2025 vector and embedding weaknesses. OWASP Top 10 for LLM Applications 2025 (OWASP GenAI Security Project), 2025. URL https://genai.owasp.org/llmrisk/llm082025-vector-and-embedding-weaknesses/.

[117] Raghuram P. onewayglass 0.3.0. Python Package Index, 2026. URL https://pypi.org/project/onewayglass/. Accessed 2026-10-31.

[118] Ruoming Pang, Ramon Caceres, Mike Burrows, Zhifeng Chen, Pratik Dave, Nathan Germer, Alexander Golynski, Kevin Graney, Nina Kang, Lea Kissner, Jeffrey L. Korn, Abhishek Parmar, Christina D. Richards, and Mengzhi Wang. Zanzibar: Google's consistent, global authorization system. In *2019 USENIX Annual Technical Conference (USENIX ATC 19)*, pages 33–46, Renton, WA, 2019. USENIX Association. ISBN 978-1-939133-03-8. URL https://www.usenix.org/conference/atc19/presentation/pang.

[119] Primal Pappachan, Roberto Yus, Sharad Mehrotra, and Johann-Christoph Freytag. Sieve: A middleware approach to scalable access control for database management systems. *Proceedings of the VLDB Endowment*, 13(12):2424–2437, 2020. doi: 10.14778/3407790.3407835.

[120] Liana Patel, Peter Kraft, Carlos Guestrin, and Matei Zaharia. ACORN: Performant and predicate-agnostic search over vector embeddings and structured data. *Proceedings of the ACM on Management of Data*, 2(3):1–27, 2024. doi: 10.1145/3654923.

[121] pgvector contributors. pgvector: Open-source vector similarity search for Postgres. GitHub repository pgvector/pgvector (README), 2026. URL https://github.com/pgvector/pgvector. Accessed 2026-10-26.

[122] Pinecone. Check data freshness. Pinecone Docs, 2026. URL https://docs.pinecone.io/guides/index-data/check-data-freshness. Accessed 2026-10-30.

[123] Pinecone. Search with a vector. Pinecone API reference, version 2026-07 (data plane, query), 2026. URL https://docs.pinecone.io/reference/api/latest/data-plane/query. Accessed 2026-10-30.

[124] Qdrant. Release v1.16.0. GitHub release notes, qdrant/qdrant, November 2025. URL https://github.com/qdrant/qdrant/releases/tag/v1.16.0.

[125] Qdrant. Filtering. Qdrant documentation, 2026. URL https://qdrant.tech/documentation/search/filtering/. Accessed 2026-10-30.

[126] Qdrant. Multitenancy. Qdrant documentation (Configure Multitenancy), 2026. URL https://qdrant.tech/documentation/manage-data/multitenancy/. Accessed 2026-10-29.

[127] Qdrant. Search. Qdrant documentation, 2026. URL https://qdrant.tech/documentation/search/search/. Accessed 2026-10-29.

[128] Zhenting Qi, Hanlin Zhang, Eric Xing, Sham Kakade, and Himabindu Lakkaraju. Follow my instruction and spill the beans: Scalable data extraction from retrieval-augmented generation systems, 2024.

[129] Alexander Rasin, James Herbick, Ben Lenard, Nick Scope, and James Wagner. On vulnerability of access control restrictions to timing attacks in a database management system. In *Proceedings of the 36th International Conference on Scientific and Statistical Database Management (SSDBM '24)*, pages 1–4, 2024. doi: 10.1145/3676288.3676306.

[130] Redis. Configuration parameters. Redis documentation (Redis Search), 2026. URL https://redis.io/docs/latest/develop/ai/search-and-query/administration/configuration/. Accessed 2026-10-30.

[131] Redis. Tag fields. Redis documentation (Redis Query Engine), 2026. URL https://redis.io/docs/latest/develop/ai/search-and-query/advanced-concepts/tags/. Accessed 2026-10-30.

[132] Redis. Vector search concepts. Redis documentation (Redis Query Engine), 2026. URL https://redis.io/docs/latest/develop/ai/search-and-query/vectors/. Accessed 2026-10-29.

[133] Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 3982–3992, 2019. doi: 10.18653/v1/D19-1410.

[134] Manuel Rigger and Zhendong Su. Detecting optimization bugs in database engines via non-optimizing reference engine construction. In *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE '20)*, pages 1140–1152, 2020. doi: 10.1145/3368089.3409710.

[135] Manuel Rigger and Zhendong Su. Finding bugs in database systems via query partitioning. *Proceedings of the ACM on Programming Languages*, 4(OOPSLA):1–30, 2020. doi: 10.1145/3428279.

[136] Manuel Rigger and Zhendong Su. Testing database engines via pivoted query synthesis. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, pages 667–682. USENIX Association, 2020. ISBN 978-1-939133-19-9. URL https://www.usenix.org/conference/osdi20/presentation/rigger.

[137] Shariq Rizvi, Alberto Mendelzon, S. Sudarshan, and Prasan Roy. Extending query rewriting techniques for fine-grained access control. In *Proceedings of the 2004 ACM SIGMOD International Conference on Management of Data (SIGMOD '04)*, pages 551–562, 2004. doi: 10.1145/1007568.1007631.

[138] Abdel Rodriguez, John Trengrove, and Joon-Pil Hwang. How we speed up filtered vector search with ACORN. Weaviate blog, November 2024. URL https://weaviate.io/blog/speed-up-filtered-vector-search.

[139] Ayush RoyChowdhury, Mulong Luo, Prateek Sahu, Sarbartha Banerjee, and Mohit Tiwari. ConfusedPilot: Confused deputy risks in RAG-based LLMs, 2024.

[140] Torin Sandall. Write policy in OPA. enforce policy in SQL. Open Policy Agent blog, August 2018. URL https://openpolicyagent.org/blog/write-policy-in-opa-enforce-policy-in-sql-d9d24db93bf4.

[141] Debdeep Sanyal, Umakanta Maharana, Yash Sinha, Hong Ming Tan, Shirish Karande, Mohan Kankanhalli, and Murari Mandal. OrgAccess: A benchmark for role based access control in organization scale LLMs, 2025.

[142] Moritz Schloegel, Nils Bars, Nico Schiller, Lukas Bernhard, Tobias Scharnowski, Addison Crump, Arash Ale-Ebrahim, Nicolai Bissantz, Marius Muench, and Thorsten Holz. SoK: Prudent evaluation practices for fuzzing. In *2024 IEEE Symposium on Security and Privacy (SP)*, pages 1974–1993, 2024. doi: 10.1109/SP54263.2024.00137.

[143] Sergio Segura, Gordon Fraser, Ana B. Sanchez, and Antonio Ruiz-Cortés. A survey on metamorphic testing. *IEEE Transactions on Software Engineering*, 42(9):805–824, 2016. doi: 10.1109/TSE.2016.2532875.

[144] Jiayang Shi, Yuzheng Cai, and Weiguo Zheng. Filtered approximate nearest neighbor search: A unified benchmark and systematic experimental study [experiment, analysis & benchmark], 2025.

[145] Harsha Vardhan Simhadri, Martin Aumüller, Matthijs Douze, Dmitry Baranchuk, Amir Ingber, Edo Liberty, George Williams, Ben Landrum, Magdalen Manohar, Mazin Karjikar, Laxman Dhulipala, Meng Chen, Yue Chen, Rui Ma, Kai Zhang, Yuzheng Cai, Jiayang Shi, Weiguo Zheng, Yizhuo Chen, Jie Yin, and Ben Huang. Results of the big ANN: NeurIPS'23 competition. In *Advances in Neural Information Processing Systems 38 (NeurIPS 2025), Datasets and Benchmarks Track*, pages 76394–76408, 2025. doi: 10.52202/085713-2303.

[146] sjmurdoch. [bug]: `delete_collection` leaves list-valued metadata behind, which reappears on records written to the next collection. GitHub issue chroma-core/chroma#7594, August 2026. URL `https://github.com/chroma-core/chroma/issues/7594`.

[147] Mark D. Smucker, James Allan, and Ben Carterette. A comparison of statistical significance tests for information retrieval evaluation. In *Proceedings of the Sixteenth ACM Conference on Information and Knowledge Management (CIKM '07)*, pages 623–632, 2007. doi: 10.1145/1321440.1321528.

[148] Congzheng Song and Ananth Raghunathan. Information leakage in embedding models. In *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security (CCS '20)*, pages 377–390, 2020. doi: 10.1145/3372297.3417270.

[149] Dawn Xiaoding Song, D. Wagner, and A. Perrig. Practical techniques for searches on encrypted data. In *Proceeding 2000 IEEE Symposium on Security and Privacy (S&P 2000)*, pages 44–55, 2000. doi: 10.1109/SECPRI.2000.848445.

[150] Michael Stonebraker and Eugene Wong. Access control in a relational data base management system by query modification. In *Proceedings of the 1974 Annual Conference (ACM '74)*, pages 180–186, 1974. doi: 10.1145/800182.810400.

[151] Suhas Jayaram Subramanya, Fnu Devvrit, Harsha Vardhan Simhadri, Ravishankar Krishnaswamy, and Rohan Kadekodi. DiskANN: Fast accurate billion-point nearest neighbor search on a single node. In *Advances in Neural Information Processing Systems 32 (NeurIPS 2019)*, pages 13748–13758, 2019. URL `https://papers.nips.cc/paper_files/paper/2019/hash/09853c7fb1d3f8ee67a61b6bf4a7f8e6-Abstract.html`.

[152] He Sun, Shinan Liu, Siyuan Ma, Junhao Li, Mingjun Xiao, and Wenhao Jiang. Agent-assisted side-channel attacks on non-prefix KV cache in RAG, 2026.

[153] sungbin1015. fix: escape VARCHAR primary keys in MilvusClient get/query/delete by ids. GitHub pull request milvus-io/pymilvus#3823, October 2026. URL `https://github.com/milvus-io/pymilvus/pull/3823`.

[154] Nandan Thakur, Nils Reimers, Andreas Rücklé, Abhishek Srivastava, and Iryna Gurevych. BEIR: A heterogeneous benchmark for zero-shot evaluation of information retrieval models. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, volume 1, 2021. URL https://datasets-benchmarks-proceedings.neurips.cc/paper/2021/hash/65b9eea6e1cc6bb9f0cd2a47751a186f-Abstract-round2.html.

[155] The PostgreSQL Global Development Group. JSON types. PostgreSQL Documentation, version 18, Section 8.14, 2026. URL https://www.postgresql.org/docs/current/datatype-json.html. Accessed 2026-10-30.

[156] The PostgreSQL Global Development Group. PREPARE. PostgreSQL Documentation, version 16, SQL Commands, 2026. URL https://www.postgresql.org/docs/16/sql-prepare.html. Accessed 2026-10-30.

[157] The PostgreSQL Global Development Group. Row security policies. PostgreSQL Documentation, version 18, Section 5.9, 2026. URL https://www.postgresql.org/docs/current/ddl-rowsecurity.html. Accessed 2026-10-26.

[158] The PostgreSQL Global Development Group. selfuncs.c: gincostestimate. PostgreSQL source code, branch REL_16_STABLE, src/backend/utils/adt/selfuncs.c, 2026. URL https://github.com/postgres/postgres/blob/REL_16_STABLE/src/backend/utils/adt/selfuncs.c. Accessed 2026-10-30.

[159] The Psycopg Team. Prepared statements. Psycopg 3 documentation, 2026. URL https://www.psycopg.org/psycopg3/docs/advanced/prepare.html. Accessed 2026-10-30.

[160] Scott Thornton. Retrieval pivot attacks in hybrid RAG: Measuring and mitigating amplified leakage from vector seeds to graph expansion, 2026.

[161] Benjamin Trent. Filtered HNSW & kNN search in Lucene and Elasticsearch. Elasticsearch Labs blog, February 2025. URL https://www.elastic.co/search-labs/blog/filtered-hnsw-knn-search.

[162] Julián Urbano, Harlley Lima, and Alan Hanjalic. Statistical significance testing in information retrieval: An empirical analysis of type I, type II and type III errors. In *Proceedings of the 42nd International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '19)*, pages 505–514, 2019. doi: 10.1145/3331184.3331259.

[163] Vespa. Graceful query coverage degradation. Vespa documentation, 2026. URL https://docs.vespa.ai/en/graceful-degradation.html. Accessed 2026-10-30.

[164] Jianguo Wang, Xiaomeng Yi, Rentong Guo, Hai Jin, Peng Xu, Shengjun Li, Xiangyu Wang, Xiangzhou Guo, Chengming Li, Xiaohai Xu, Kun Yu, Yuxing Yuan, Yinghao Zou, Jiquan Long, Yudong Cai, Zhenxiang Li, Zhifeng Zhang, Yihua Mo, Jun Gu, Ruiyi Jiang, Yi Wei, and Charles Xie. Milvus: A purpose-built vector data management system. In *Proceedings of the 2021 International Conference on Management of Data (SIGMOD '21)*, pages 2614–2627, 2021. doi: 10.1145/3448016.3457550.

[165] Liang Wang, Paul Grubbs, Jiahui Lu, Vincent Bindschaedler, David Cash, and Thomas Ristenpart. Side-channel attacks on shared search indexes. In *2017 IEEE Symposium on Security and Privacy (SP)*, pages 673–692. IEEE, 2017. doi: 10.1109/SP.2017.50. URL https://stressattack.github.io/paper/sp17stress.pdf.

[166] Mengzhao Wang, Lingwei Lv, Xiaoliang Xu, Yuxiang Wang, Qiang Yue, and Jiongkang Ni. An efficient and robust framework for approximate nearest neighbor search with attribute constraint. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, pages 15738–15751, 2023. doi: 10.52202/075280-0692.

[167] Qihua Wang, Ting Yu, Ninghui Li, Jorge Lobo, Elisa Bertino, Keith Irwin, and Ji-Won Byun. On the correctness criteria of fine-grained access control in relational databases. In *Proceedings of the VLDB Conference (VLDB '07)*, Vienna, Austria, 2007. VLDB Endowment. URL `https://www.vldb.org/conf/2007/papers/research/p555-wang.pdf`.

[168] Shenao Wang, Yanjie Zhao, Yinglin Xie, Zhao Liu, Xinyi Hou, Quanchen Zou, and Haoyu Wang. Towards reliable vector database management systems: A software testing roadmap for 2030, 2025.

[169] Wenhui Wang, Furu Wei, Li Dong, Hangbo Bao, Nan Yang, and Ming Zhou. MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers. In *Advances in Neural Information Processing Systems*, volume 33, 2020. URL `https://proceedings.neurips.cc/paper/2020/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html`.

[170] Weaviate. Inverted indexes. Weaviate Documentation, 2026. URL `https://docs.weaviate.io/weaviate/concepts/indexing/inverted-index`. Accessed 2026-10-30.

[171] Weaviate. tokenizer.go. GitHub repository weaviate/weaviate, entities/tokenizer, tag v1.39.12, 2026. URL `https://raw.githubusercontent.com/weaviate/weaviate/v1.39.12/entities/tokenizer/tokenizer.go`. Accessed 2026-10-30.

[172] Weaviate. Vector index. Weaviate Documentation, 2026. URL `https://docs.weaviate.io/weaviate/config-refs/indexing/vector-index`. Accessed 2026-10-29.

[173] Chuangxian Wei, Bin Wu, Sheng Wang, Renjie Lou, Chaoqun Zhan, Feifei Li, and Yuanzhe Cai. AnalyticDB-V: A hybrid analytical engine towards query fusion for structured and unstructured data. *Proceedings of the VLDB Endowment*, 13(12):3152–3165, 2020. doi: 10.14778/3415478.3415541.

[174] Wai Kit Wong, David Wai-lok Cheung, Ben Kao, and Nikos Mamoulis. Secure kNN computation on encrypted databases. In *Proceedings of the 2009 ACM SIGMOD International Conference on Management of Data (SIGMOD '09)*, pages 139–152, 2009. doi: 10.1145/1559845.1559862.

[175] Fangzhou Wu, Ethan Cecchetti, and Chaowei Xiao. System-level defense against indirect prompt injection attacks: An information flow control perspective, 2024.

[176] Wei Wu, Junlin He, Yu Qiao, Guoheng Fu, Li Liu, and Jin Yu. HQANN: Efficient and robust similarity search for hybrid queries with structured and unstructured constraints. In *Proceedings of the 31st ACM International Conference on Information and Knowledge Management (CIKM '22)*, pages 4580–4584, 2022. doi: 10.1145/3511808.3557610.

[177] Shitao Xiao, Zheng Liu, Peitian Zhang, Niklas Muennighoff, Defu Lian, and Jian-Yun Nie. C-Pack: Packed resources for general Chinese embeddings. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR)*, 2024. doi: 10.1145/3626772.3657878.

[178] Yinglin Xie, Xinyi Hou, Yanjie Zhao, Shenao Wang, Kai Chen, and Haoyu Wang. Toward understanding bugs in vector database management systems, 2025.

[179] Yuexuan Xu, Jianyang Gao, Yutong Gou, Cheng Long, and Christian S. Jensen. iRangeGraph: Improvising range-dedicated graphs for range-filtering nearest neighbor search. *Proceedings of the ACM on Management of Data*, 2(6):1–26, 2024. doi: 10.1145/3698814.

[180] Lakshmi Sahithi Yalamarthi and Primal Pappachan. Policy-aware vector search: A vision for fine grained access control in vector databases, 2026.

[181] An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, et al. Qwen2.5 technical report, 2024.

[182] Shenglai Zeng, Jiankun Zhang, Pengfei He, Yue Xing, Yiding Liu, Han Xu, Jie Ren, Shuaiqiang Wang, Dawei Yin, Yi Chang, and Jiliang Tang. The good and the bad: Exploring privacy issues in retrieval-augmented generation (RAG). In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 4505–4524, Bangkok, Thailand, 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.findings-acl.267.

[183] Sergej Zerr, Elena Demidova, Daniel Olmedilla, Wolfgang Nejdl, Marianne Winslett, and Soumyadeb Mitra. Zerber: r-confidential indexing for distributed documents. In *Proceedings of the 11th International Conference on Extending Database Technology: Advances in Database Technology (EDBT '08)*, pages 287–298, 2008. doi: 10.1145/1353343.1353380.

[184] Sergej Zerr, Daniel Olmedilla, Wolfgang Nejdl, and Wolf Siberski. Zerber+R: Top-k retrieval from a confidential index. In *Proceedings of the 12th International Conference on Extending Database Technology: Advances in Database Technology (EDBT '09)*, pages 439–449, 2009. doi: 10.1145/1516360.1516412.

[185] Collin Zhang, John X. Morris, and Vitaly Shmatikov. Universal zero-shot embedding inversion, 2025.

[186] Danfeng Zhang, Aslan Askarov, and Andrew C. Myers. Language-based control and mitigation of timing channels. In *Proceedings of the 33rd ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI '12)*, pages 99–110, 2012. doi: 10.1145/2254064.2254078.

[187] Qianxi Zhang, Shuotao Xu, Qi Chen, Guoxin Sui, Jiadong Xie, Zhizhen Cai, Yaoqi Chen, Yinxuan He, Yuqing Yang, Fan Yang, Mao Yang, and Lidong Zhou. VBASE: Unifying online vector similarity search and relational queries via relaxed monotonicity. In *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI 23)*, pages 377–395, 2023. URL `https://www.usenix.org/conference/osdi23/presentation/zhang-qianxi`.

[188] Xiang Zhang, Chao Zhang, Ju Fan, Guoliang Li, and Xiaoyong Du. VecBench: A controllable benchmark for filtered vector search: [experiments & analysis]. *Proceedings of the ACM on Management of Data*, 4(3):1–27, 2026. doi: 10.1145/3802125.

[189] Hongbin Zhong, Matthew Lentz, Nina Narodytska, Adriana Szekeres, and Kexin Rong. HONEYBEE: Efficient role-based access control for vector databases via dynamic partitioning. *Proceedings of the ACM on Management of Data*, 4(1):1–26, 2026. doi: 10.1145/3786625.

[190] Huijun Zhou, Xiaohan Zhang, Haozhe Zhang, Haoyang Zhang, Mi Zhang, and Min Yang. A first measurement study on authentication security in real-world remote MCP servers, 2026.

[191] Wei Zou, Runpeng Geng, Binghui Wang, and Jinyuan Jia. PoisonedRAG: Knowledge corruption attacks to retrieval-augmented generation of large language models. In *34th USENIX Security Symposium (USENIX Security 25)*, pages 3827–3844. USENIX Association, 2025. ISBN 978-1-939133-52-6. URL `https://www.usenix.org/conference/usenixsecurity25/presentation/zou`.

[192] Chaoji Zuo, Miao Qiao, Wenchao Zhou, Feifei Li, and Dong Deng. SeRF: Segment graph for range-filtering approximate nearest neighbor search. *Proceedings of the ACM on Management of Data*, 2(1):1–26, 2024. doi: 10.1145/3639324.

A Experiment map

The main text names experiments by what they do. Their codes, which the appendices and the released files use, are defined only here. Table 4 gives, for each, where it is reported, what it tests, where and how often it ran, the library commit, the versions of stores and engines, and the run identifiers. Numeric run identifiers are GitHub Actions runs of the library's repository unless marked; E is the enterprise corpus and M the MS MARCO corpus. No single version of the library was evaluated throughout (Appendix O). The logs of the Kaggle kernels named below are in the artifact and in the public dataset `https://www.kaggle.com/datasets/muzakkirhussain1/wardowl-paper-evidence` (version 2).

Table 4: The experiments: code, where reported, what each tests, platform with seeds and builds, library commit, store and engine versions, and run identifiers.

Code	Where	What it tests	Platform; seeds and builds	Library commit	Store and engine versions	Run ids
X0	§6.1, §9; App. H, M	The enterprise scenario end to end: safety, recall and the model, ten configurations (the OpenFGA one did not start)	Kaggle, one session, 2×T4, 4 CPUs; seed 7	wardowl 2.1.0 from PyPI; acceptance scripts bf25bd0	Table 21	Kaggle kernel wardowl-end-to-end-scale-run-seed-7, executed 2026-11-07 (first executed 2026-10-24; App. M.1)
X1, run 1	§6.2, §6.3; App. H	The diagnosis grid, Qdrant and pgvector hypotheses; also an offline re-analysis of X0's answers	Kaggle, one session; 1,489 questions; one pgvector build, 11 settings (12 items skipped for time); five Qdrant layouts, three builds each	wardowl from commit 947fb8f, whose package tree equals tag v2.1.0; scripts 947fb8f	PostgreSQL 16, pgvector 0.8.7; Qdrant 1.19.2	Kaggle notebook wardowl-under-retrieval-diagnosis-run-1 (then notebook11401c38ff), version 362442153, 2026-10-26
X1, run 2	§6.2, §6.3; App. H.4	The grid again with plan control, executed plans and two replays; every X1 number in the paper	Kaggle, one GPU and four CPU sessions; 1,489 questions, three timed passes; three builds per layout; 49 settings	Branch paper/x1-underretrieval at a956424 (version 2.1.0)	PostgreSQL 16, pgvector 0.8.7; Qdrant 1.19.2	Kaggle kernels x1run2-prep-gpu, -shard-a, -b, -c, -stats, 2026-10-29 and 30
X2	§6.5; App. I	The frontier experiment: recall against throughput, correlation and selectivity	GitHub-hosted, 4 vCPUs, 16 GB, one store per job; E, 1,468 scorable pairs; M, 5 seeds of 2 layouts, 1,050 subjects, 6,200 pairs; 3 builds per store	Branch paper/x2-x5 at 884ce18, from feat/under-retrieval at db28517 (PR #168)	PostgreSQL 17.11, pgvector 0.8.7; Qdrant 1.19.2; Elasticsearch 9.5.5; OpenSearch 3.9.0; Weaviate 1.39.12; Redis 8.10.2; Milvus 3.0.2 (image); MongoDB Atlas Local 8.0.30; LanceDB 0.40.0; Chroma 1.5.9; FAISS 1.15.1	40921769714, created 2026-10-29 17:29 UTC; concluded cancelled, since one job (x2-m-chroma-b2) reached the runner's timeout
X3	§6.5; App. J	Filter size: 1 to 40,000 groups, a fixed readable set	GitHub-hosted, as X2; five seeds; 12 stores and 2 controls; 30 questions per level, three passes	Branch paper/x3-x6 at a191626, from main at 438a176 (2.1.0 and PR #166)	Images and clients of the conformance campaign (Appendix E.2)	40940385704, created 2026-10-29 19:28 UTC

continued on the next page

Table 4, continued

Code	Where	What it tests	Platform; seeds and builds	Library commit	Store and engine versions	Run ids
X4	§5; App. G	Engine filters against engine checks: agreement, revision skew, truncation, latency	GitHub-hosted, as X2; five seeds; 13 profiles, 600 subjects each, 900,000 pairs in each of the 11 that gave any user a filter	Harness e66dffc on paper/x4-x7; library fix/engine-limits at e7776f0	OpenFGA 1.22.0; SpiceDB 1.56.2; OPA 1.21.2; Cerbos 0.56.0; PostgreSQL 17 (image pgvector/pgvector:pg17); FAISS 1.15.1	40919304635, created 2026-10-29 17:13 UTC
X5	§7; App. K	Detection and bounded repair of under-retrieval	GitHub-hosted, as X2; pgvector and Qdrant on E and M; 3 builds per store and corpus	As X2	As X2	40921769714
X6	§8; App. L	Side channels: the distinguishing game and four mitigations	GitHub-hosted, as X2; five seeds, 72 probes per seed, three attackers, five repetitions	As X3	pgvector, Qdrant, Elasticsearch and FAISS as X3	40940385704
Follow-up (X3 and X6, amendment 4)	App. J, L	Part A, Chroma's X3 seeds 3 and 4. Part B, exploratory: FAISS's offset, cross-build A/A, hubs, plans, vacuum state, the crash's threshold	GitHub-hosted, as X2; five seeds; Elasticsearch's part lost with its runner	Branch paper/x3-x6 at 9e22b89	As X3	40982986417, created 2026-10-30 00:58 UTC; concluded failure, since one job (fu-cb-elasticsearch) lost its runner
X7	§5.1; App. G.6	Revocation delay per path and faults	GitHub-hosted, as X2; five seeds; 40 trials per cell; 1,090 fault episodes	As X4	As X4, with Qdrant 1.19.2 and Redis 8.10.2 (shared cache)	40919304635
X7, amendment 2	§5.1; App. G.6	A targeted rerun of the library's fixes	GitHub-hosted, as X2; five seeds; 40 trials per cell; 140 fault episodes	Harness 41292bd; library fix/revocation-faults at 3c6559f (PR #186)	As X7	40969846593, created 2026-10-29 22:58 UTC
X8	§6.1, §9; App. M	Three repeats of X0	Kaggle, 2×T4, 4 CPUs, one session per seed; seeds 11, 13, 17	As X0	As X0 (psycopg 3.3.6; X0 3.3.8)	Kaggle kernels wardowl-x8-seed11, -seed13, -seed17
X8-FGA (X8, amendment 1)	§9; App. M	pgvector with OpenFGA, its start-up timeout fixed, beside in-process pgvector	Kaggle, as X8; seeds 7, 11, 13, 17	As X0	PostgreSQL 16.15, pgvector 0.8.7; OpenFGA 1.22.0	Kaggle kernels wardowl-x8fga-seed7, -seed11, -seed13, -seed17
X8, amendment 2	§9	Exposure and a mutation positive control (App. M.3)	Kaggle, as X8; FAISS only; seeds 7, 11, 13, 17 (exposure), seed 7 (control)	As X0	FAISS, as X0	Kaggle kernels wardowl-x8b-seed7, -seed11, -seed13, -seed17, wardowl-x8b-control-seed7, wardowl-x8b-pool
Conformance campaign, runs 1 and 2	§4; App. E, F	The kit against stores and engines	GitHub-hosted; 28 targets ran; 20 seeds of nine (stores) or seven (engines) variants	5d0f414 (2.1.0); 182bcbc (2.1.0 with PRs #166 and #167)	Appendix E.2	40313285885, created 2026-10-26 11:12 UTC; 40905549149, created 2026-10-29 15:50 UTC

continued on the next page

Table 4, continued

Code	Where	What it tests	Platform; seeds and builds	Library commit	Store and engine versions	Run ids
Vendor reproducers	§11; App. O	The Redis, MongoDB, Cerbos and Chroma behaviors with the vendors' clients only	GitHub-hosted, ubuntu-24.04; the tested and the latest version of each	None	Redis 8.10.2, 8.10.3; MongoDB Atlas Local 8.0.30, 8.3.13; Cerbos 0.56.0, 0.57.0; Chroma 1.5.9	41007436602, 41007779363, 41008151303, in the author's private repository, so these pages do not open; the outputs are in the artifact, under `evidence/raw/results-reproducers/`; rerun on the public repository on 2026-10-31 as 41249047470, with the same verdicts

B Pre-registration ledger

Table 5 lists every hypothesis and prediction with its registration document, its commit or amendment, whether it was written before or after the results it concerns, the run that judged it, the verdict as the analysis script printed it, and a class: *conf.*, confirmatory, written before any full-size result of its experiment, smoke runs of a harness being earlier exposure where they occurred; *expl.*, exploratory, specified in writing after the main results and before running; or *impl.*, an implementation check of a fix or of the harness. A reader can check the ordering to different degrees. For the experiments on GitHub Actions, the pre-registration commit is an ancestor of the run's head commit (`git merge-base -is-ancestor`), and the run's creation time, which GitHub records, follows the commit. For the Kaggle runs the evidence is weaker. Git dates are self-reported. GitHub's public activity record (`evidence/raw/x1-branch-activity.json`) shows X1's branch pushed at 947fb8f on 2026-10-26 at 09:50 UTC, after the registration and amendment 1, and at a956424 on 2026-10-29 at 16:55 UTC, after amendment 2 and its revisions. Each X1 kernel clones its commit from there, so that a kernel cannot have run before the push; run 1's public notebook log prints 947fb8f, while run 2's log is the author's copy. No such record supports X8. The X8 pre-registration and its amendments were committed before the runs in a private repository, which was not public then; the public artifact repository that holds those commits was first created on 2026-10-30 at 21:29 UTC, after every Kaggle run, and recreated on 2026-10-31 at 22:32 UTC, before its archive, to drop private planning notes from its history; the commits named here kept their hashes. The kernels are private; readers can check their outputs and logs, in the artifact and in the public Kaggle dataset `https://www.kaggle.com/datasets/muzakkirhussain1/wardowl-paper-evidence`. The store logs of X0, X8 and X8-FGA carry dates, while the kernel logs, which are all that amendment 2 has, hold only seconds since their start, and any of these date at most a run, not the writing. The artifact's `evidence/raw/KAGGLE_KERNELS.tsv` lists the start times Kaggle's API reported to the author, which agree with the commit dates but cannot be checked by others. Rules and full wording are in the pre-registration files. The confirmatory class is relative to a stated earlier exposure: the X1 predictions followed the printed aggregates of X0's first execution (Appendix M.1); the X2 and X5 predictions followed the same, the verdicts of X1's first run and earlier MS MARCO benchmarks in the library's repository; the X8 predictions followed X0's first execution; and the library's FiQA

benchmarks, among them the row-level security run of Appendix O.1, preceded all of them. ∗: the reference setting changed after run 1, so run 2's verdict is not purely confirmatory. The labels H2 and H5 of X6 are unrelated to H2.1 to H2.4 of X2 and H5.1 to H5.7 of X5.

Table 5: Every hypothesis and prediction, with its registration, ordering, run, verdict as printed and class.

Label	Hypothesis or prediction	Commit	Written	Run	Verdict as printed	Class
X1, the diagnosis grid. PREREGISTRATION.md, branch paper/x1-underretrieval: 7c991cd (2026-10-26 09:01), amendment 1 fb2ef64 (09:49, before any result), amendment 2 eeb7d2b (2026-10-29 15:46) and its revisions afadde4 (16:02, the prep may embed on the CPU) and 80e3f56 (16:54). Run 1: Kaggle, 2026-10-26. Run 2: Kaggle, prep pushed 2026-10-29 19:12. By amendment 2 every verdict is judged on run 2; run 1's is given first.						
Q1	Islanded permitted subgraph in Qdrant	7c991cd	Before run 1	Runs 1, 2	confirmed; confirmed	conf.
Q2	ACORN's default gate engages unreliably	7c991cd	Before run 1	Runs 1, 2	falsified; falsified	conf.
Q3	Recall varies by build and segment layout	7c991cd	Before run 1; layout fixed by am. 2	Runs 1, 2	inconclusive; inconclusive	conf.
Q4	`hnsw_ef` 128 was not the cause	7c991cd	Before run 1	Runs 1, 2	inconclusive; inconclusive	conf.
P1	The iterative scan's budget runs out	7c991cd; am. 2	Before run 1; reference setting changed after run 1, before run 2	Runs 1, 2	confirmed; inconclusive	conf.*
P2	Mis-retrieval from shallow, relaxed scans	7c991cd; am. 2	Before run 1; reference setting changed after run 1, before run 2	Runs 1, 2	inconclusive; inconclusive	conf.*
P3	The planner overestimates a least-privileged filter	7c991cd	Before run 1	Runs 1, 2	inconclusive; confirmed	conf.
P4	Generic plans make no difference	7c991cd; am. 1	Before run 1	Runs 1, 2	falsified; falsified	conf.
P5	The plan cache decides whether pgvector searches the graph	eeb7d2b, 80e3f56	After run 1, before run 2	Run 2	confirmed	expl.
X2, the frontier experiment. PREREG_X2_X5.md, branch paper/x2-x5: 828ea27 (2026-10-29 16:24), alone, before any harness code. Run 40921769714 at 884ce18, created 17:29.						
H2.1	Negative correlation costs recall at defaults	828ea27	Before any X2 result	40921769714	inconclusive	conf.
H2.2	Local axes predict recall better than global selectivity	828ea27	Before any X2 result	40921769714	falsified	conf.
H2.3	Effort narrows the gap but does not close it	828ea27	Before any X2 result	40921769714	confirmed by its rule, on two of four stores; LanceDB's narrowing is 0.001	conf.
H2.4	Intermediate filters are harder than the smallest (registered as "the hardest filters are the intermediate ones"; the test does not ask whether recall recovers)	828ea27	Before any X2 result	40921769714	confirmed (per store, "U-shaped", "monotone" or "neither")	conf.
X5, repair. The same document and commit; the same run.						
H5.1	A count certifies short results without false alarms	828ea27	Before any X5 result	40921769714	confirmed (follows from the count on unchanging data)	conf.
H5.2	Exact repair removes empty and short results	828ea27	Before any X5 result	40921769714	confirmed (the rule names the unbounded arm)	conf.
H5.3	The repairs cost nothing on untriggered queries	828ea27	Before any X5 result	40921769714	inconclusive	conf.
H5.4	Repairs do not touch mis-retrieval	828ea27	Before any X5 result	40921769714	confirmed	conf.
H5.5	The recall monitor sees what the flag cannot	828ea27	Before any X5 result	40921769714	confirmed	conf.

continued on the next page

Table 5, continued

Label	Hypothesis or prediction	Commit	Written	Run	Verdict as printed	Class
H5.6	Widening is not a certificate	828ea27	Before any X5 result	40921769714	confirmed	conf.
H5.7	Store-native layout fixes what library repairs cannot	828ea27	Before any X5 result	40921769714	confirmed	conf.
X3, filter size. PREREG_X3_X6.md, branch paper/x3-x6: 5da58ee (2026-10-29 15:50), amendments 1 to 3 f0acc15 (17:05), c369a80 (19:10), 3b7ba31 (19:19), all before any full result. Run 40940385704 at a191626, created 19:28. Amendment 4 ba67dd8 (23:31), after that run's results; follow-up 40982986417 at 9e22b89, created 2026-10-30 00:58.						
X3-H1	Every translator returns within a second at every level	5da58ee	Before any full result	40940385704; 40982986417	inconclusive (77 of 84 parts confirmed, 7 inconclusive, all Chroma); confirmed (84 of 84) with Chroma's five seeds	conf.
X3-H2	Eight stores accept 40,000 groups; Chroma refuses	5da58ee	Before any full result	40940385704; 40982986417	confirmed for each of the eight; Chroma inconclusive (three seeds), confirmed with five	conf.
X3-H3a	pgvector's median grows at least fiftyfold	5da58ee	Before any full result	40940385704	confirmed	conf.
X3-H3b	The controls' median grows at most twofold	5da58ee	Before any full result	40940385704	confirmed	conf.
X3-H3c	Cassandra's per-group queries against its fallback	5da58ee	Before any full result	40940385704	confirmed	conf.
X3-H4	Recall does not depend on the filter's length	5da58ee	Before any full result	40940385704; 40982986417	confirmed for each of eight stores; Chroma inconclusive in both	conf.
X6, side channels. The same document, commits and run. Guards G1 and G2 precede every verdict.						
H1	Widening: count at $m = 1{,}000$, latency at $m = 100$, none at $m = 1$	5da58ee	Before any full result	40940385704	falsified (parts 29 confirmed, 5 falsified, 6 inconclusive)	conf.
H2	Constant work removes the latency channel, the count channel remains	5da58ee	Before any full result	40940385704	falsified (2, 4, 10)	conf.
H3a	pgvector's approximate pre-filter: latency channel	5da58ee	Before any full result	40940385704	confirmed	conf.
H3b	Elasticsearch's approximate pre-filter: latency channel	5da58ee	Before any full result	40940385704	inconclusive	conf.
H3c	Qdrant at $p = 1\%, 10\%$: no channel	5da58ee	Before any full result	40940385704	confirmed	conf.
H4	Exact pre-filter: no channel in any observable	5da58ee	Before any full result	40940385704	inconclusive for pgvector, Qdrant, Elasticsearch, FAISS and the in-memory control	conf.
H5a	BM25, IDF-weighted sparse: score channel at every m	5da58ee	Before any full result	40940385704	inconclusive for both stores	conf.
H5b	`ts_rank_cd`: no score channel	5da58ee	Before any full result	40940385704	confirmed	conf.
H5c	Fusion changes the hit list for at least 25% of probes	5da58ee	Before any full result	40940385704	confirmed	conf.
H6	Padding removes the latency channel	5da58ee	Before any full result	40940385704	inconclusive (script defect; confirmed under the written rule, by construction, since padding was computed afterwards)	conf.
Am. 4, B	Readings fixed in advance for FAISS's post-filter at $m = 1$, FAISS's pre-filter and pgvector's exact search; hubs; plans; vacuum state; the crash's threshold	ba67dd8	After the main results, before running	40982986417	inconclusive for the three readings; hub mechanism as predicted; no verdict changes	expl.
X4, filters against checks. PREREG_X4_X7.md, branch paper/x4-x7: eafafa9 (2026-10-29 16:28), alone, before any harness code; amendment 1 ba00ca6 (17:07), after a smoke run, before any full result. Run 40919304635 at e66dffc, created 17:13.						

continued on the next page

Table 5, continued

Label	Hypothesis or prediction	Commit	Written	Run	Verdict as printed	Class
X4-H1	Agreement at one revision	eafafa9	Before any full result	40919304635	confirmed for seven profiles	conf.
X4-H2	Shape drift: Cerbos disagrees, OPA's policy does not	eafafa9	Before any full result	40919304635	confirmed	conf.
X4-H3	Negation is never dropped; availability per profile	eafafa9	Before any full result	40919304635	confirmed (no false admit, vacuously); availability confirmed for Cerbos, falsified for OPA's partial evaluation	conf.
X4-H4a	SpiceDB pinned under the writer: no disagreement	eafafa9	Before any full result	40919304635	confirmed	conf.
X4-H4b	Unpinned disagreements explained by a write	eafafa9	Before any full result	40919304635	confirmed for five conditions	conf.
X4-H4c	Quiet conditions: no disagreement	eafafa9	Before any full result	40919304635	confirmed	conf.
X4-H4d	SpiceDB, default consistency against pinned	eafafa9	Before any full result	40919304635	confirmed	conf.
X4-H5a–d	List APIs truncate at the limit; the layer never returns a short filter	eafafa9	Before any full result	40919304635	confirmed, each part	conf.
X4-H6	Recall after the fallback	eafafa9	Before any full result	40919304635	confirmed, 24 of 24	conf.
X4-H7	Caching is at least tenfold faster	eafafa9	Before any full result	40919304635	confirmed for OpenFGA, SpiceDB, OPA; inconclusive for Cerbos	conf.
X7, revocation and faults. The same document, commits and run.						
X7-H1	The stated bound holds	eafafa9	Before any full result	40919304635	falsified in 3 of 45 cells (SpiceDB `minimizeLatency`), confirmed in 42	conf.
X7-H2	The ttl is the realized delay	eafafa9	Before any full result	40919304635	confirmed, 12 of 12	conf.
X7-H3	Invalidation ends the delay	eafafa9	Before any full result	40919304635	falsified for SpiceDB `minimizeLatency` (2 cells), confirmed in 16	conf.
X7-H4	Metadata paths ignore the ttl	eafafa9	Before any full result	40919304635	confirmed, 10 of 10	conf.
X7-H5	Engine staleness is real and bounded	eafafa9	Before any full result	40919304635	falsified	conf.
X7-H6	Fail closed	eafafa9	Before any full result	40919304635	confirmed	conf.
X7-H7a–b	Availability cost of transport and slow faults on engines	eafafa9	Before any full result	40919304635	confirmed, 24 of 24 and 12 of 12	conf.
X7-H7c	Availability cost of store faults	eafafa9	Before any full result	40919304635	confirmed for Qdrant and pgvector's down and refuse; inconclusive for pgvector's blackhole	conf.
X7-H8a–b	Recovery, and the circuit breaker	eafafa9	Before any full result	40919304635	confirmed, each part	conf.
X7-H9	A warm cache masks an outage	eafafa9	Before any full result	40919304635	confirmed, 20 of 20	conf.
X7, amendment 2, the targeted rerun. PREREG_X4_X7.md: 618b108 (2026-10-29 20:40), after run 40919304635's results. Run 40969846593 at 41292bd, created 22:58.						
A2-H1	Revocation within the declared staleness plus 1 s after invalidation	618b108	After the main results, before running	40969846593	confirmed	impl.
A2-H2	The cache adds nothing to the engine's delay	618b108	As above	40969846593	confirmed	impl.
A2-H3	The declared staleness bounds the engine (with X7-H2 and X7-H5 judged again)	618b108	As above	40969846593	confirmed	impl.
A2-H4	pgvector, unanswered connection with requests in flight	618b108	As above	40969846593	confirmed	impl.

continued on the next page

Table 5, continued

Label	Hypothesis or prediction	Commit	Written	Run	Verdict as printed	Class
A2-H5	The statement timeout bounds a statement the server holds	618b108	As above	40969846593	confirmed	impl.
A2-H6	An empty stream no longer reads as "nothing readable"	618b108	As above	40969846593	confirmed	impl.
A2-H7	Store errors carry a fixed message	618b108	As above	40969846593	confirmed	impl.
A2-H8	Fail closed, replicated	618b108	As above	40969846593	confirmed	impl.
X8, the end-to-end repeats. plan/x8-prereg.md, committed in the author's then private repository: 875d98e (git date 2026-10-29 15:44), after X0, before any repeat; kernels wardowl-x8-seed11, -seed13, -seed17 at bf25bd0 of the acceptance scripts.						
X8-1	No unreadable hit, canary or violation in any run	875d98e	After X0, before the repeats	X8 kernels	holds	conf.
X8-2	Qdrant recall@10 between 0.45 and 0.70 in every run	875d98e	After X0, before the repeats	X8 kernels	holds	conf.
X8-3	pgvector returns nothing to 3 to 25 people in every run (the harness's count is of empty answers, not people)	875d98e	After X0, before the repeats	X8 kernels	falsified	conf.
X8-4	Redis, Elasticsearch and OpenSearch at or above 0.90	875d98e	After X0, before the repeats	X8 kernels	holds	conf.
X8-5	The model obeys every injected notice it retrieves (scored as the notice's marker in the answer)	875d98e	After X0, before the repeats	X8 kernels	holds	conf.
X8-FGA, amendment 1 of the same file: ba64c23 and 1a25086 (git date 2026-10-29 21:58), after X0 and X8, before these runs; kernels wardowl-x8fga-seed7, -seed11, -seed13, -seed17.						
F1	OpenFGA starts and every tuple is written	ba64c23	After the main results, before running	X8-FGA kernels	holds	impl.
F2	Safety for pgvector with OpenFGA	ba64c23	As above	X8-FGA kernels	holds	expl.
F3	Every retrieval with OpenFGA uses the pre-filter	ba64c23, 1a25086	As above	X8-FGA kernels	holds	impl.
F4	In-process pgvector shows one of the two known modes	ba64c23	As above	X8-FGA kernels	holds as worded, with a caveat: one run was a mixture	expl.
X8, amendment 2 of the same file: eff0b23 (git date 2026-10-30 10:11), after X0, X8, X8-FGA and the critical review, before any of its runs.						
Am. 2	Exposure and the mutation control: seven predictions, E1 to E7 here; plan/x8-prereg.md and evidence/x8-controls.md number them X1 to X7, unrelated to the experiment codes	eff0b23	After the main results, before running	wardowl-x8b kernels	E1 to E6 hold; E7 holds for targeted questions and is falsified for red-team prompts (87.7%, predicted at least 90%)	expl.

C The problem statement in detail

This appendix holds the discussion that Section 2 summarizes: how the criteria relate to their sources, what follows from the observation, the cache and count arguments with the measured delays, and the full threat model and metrics.

C.1 Reading the criteria

Permission-recall@k, the under-retrieval rate and the false-completeness rate (Section 2.4) measure the shortfall of soundness and completeness. Count-honesty implies completeness only when

$R_\tau(s) \subseteq E(\varphi_s)$, and a full answer of the wrong chunks satisfies it. A flag with $c = 1$ only for maximal results would need exact search to set; Appendix D.3 shows that the layer we test is count-honest relative to φ_s and no more. Wang et al.'s soundness compares an answer with the unrestricted one; ours compares it with the exact answer over the readable set, which is what a Truman subject is owed. Their security quantifies over database states that the policy cannot tell apart and requires the same answer on all of them, so it resists collusion and repeated queries; our *secure* asks only that each returned chunk be readable at the revision its check read, which an empty answer satisfies, and it is strictly weaker. The two worlds of Section 8.1 are such states for the attacker, and answers that differ between them, in hits or in count, would fail Wang et al.'s definition. Score fidelity is a separate conformance property (finite scores in non-increasing order, Section 4); mis-ordering inside a store, such as the slightly out-of-order output of pgvector's relaxed iterative scans [121], shows in recall, not in security. Security concerns the returned set only. Büttcher and Clarke's stricter requirement, that no aspect of the output depend on unsearchable content [23], is a noninterference condition [56] on everything a caller observes, including timing and the flag; we treat those observables as side channels (Section 2.3). Under Truman semantics a short answer looks like an answer to a subject who can read little; the flag c is a partial step toward non-Truman behavior, letting the layer say that an answer may be incomplete.

Observation 1 moves the burden of security onto the trusted computing base of Section 2.2, whose size we did not measure, and turns everything else into a question of completeness, soundness and freshness. That it holds for every query, chosen adaptively or not, is something no sampled measurement can show, and we ran no adaptive adversary.

Three consequences follow. *First, the re-check sees filter faults in one direction only.* A *false admit*, where φ_s accepts a chunk outside $R_\tau(s)$, reaches the re-check, which drops it and counts a violation; the violation count thus measures online how far the pushed-down filter over-approximates. A *false denial*, where φ_s rejects a readable chunk, never becomes a candidate and silently costs completeness. Truncated allow-lists (OpenFGA's non-streaming ListObjects stops at 1,000 results or 3 seconds by default [106]), divergent store filter semantics and approximate-index misses all fail this way, and only an offline oracle finds them: differential conformance testing against a reference semantics (Section 4) or an audit against exact search over $R_\tau(s)$ (Section 6). *Second, the assumptions mark where security can still fail.* (A1) excludes wrong upstream permissions and, when the check reads ACLs copied into chunk metadata, holds only as far as the copy is fresh; (A2) fails under any store or ingestion fault that binds content to the wrong identifier or metadata, which no re-check can detect; and security holds at the revision the check reads, so how stale that revision may be is a separate guarantee, freshness, which we state per path below. *Third, a violation is a translation fault only at a single revision.* If φ_s is compiled at revision τ_f and hits are checked at $\tau_c \neq \tau_f$, a disagreement may be a legitimate ACL change between the calls. Zanzibar's snapshot tokens and SpiceDB's revision tokens prevent such mixing [13, 118], and we pin filter and check to one revision when measuring disagreement (Section 5).

The trusted computing base in full. The observation trusts (i) the authorizer's check API and the authorization data it reads (A1), including the freshness of ACLs copied into chunk metadata; (ii) the layer's mapping of a request to checks: the subject taken from a verified token (Appendix D.5) and the adapter code that turns the subject, its groups and each chunk's identifier and attributes into the engine's check request; (iii) the binding of content to identifier and metadata in the store and in ingestion (A2); (iv) the decision cache, its keys, time to live and invalidation, and the engine's own staleness, which together set δ; and (v) the fail-closed path (A3). It does not trust the filter APIs, the translators, the store's filter semantics or index, the widening loop, or a reranker or redactor, which receive only permitted hits. The trusted part has failed before. Two faults in the

layer's own adapters let the check itself admit, beyond any re-check: a SpiceDB adapter that checked groups as separate subjects and so bypassed an exclusion, and a Cedar adapter that dropped values a `forbid` rule needed (Table 7); the project's acceptance scenarios and review found them, and both were fixed before the version we evaluate. With the re-check turned off, the filter and its translator join the trusted computing base (Appendix D.2).

C.2 Caches, staleness and count certificates

Caches and staleness. Let the engine answer from a snapshot that may lack the writes of the last δ_e seconds, and let the layer cache each decision for at most a time to live ℓ. A decision used at time t was then read at a revision that includes every write committed before $t - (\ell + \delta_e)$, so under (A1) to (A3) the layer is $(\ell + \delta_e)$-fresh, or δ_e-fresh without a cache. An explicit invalidation is meant to restore δ_e-freshness for the subjects it names; it can do so only if the cache does not then store answers the engine computes from a snapshot older than the invalidation. Section 5.1 measures δ per path (Table 2). When permissions are copied into chunk metadata, timed from the write to the store, and for engines read at full consistency, revocation took effect in a median of 0.01 s or less; for engines behind a cache it took the time to live (medians within 34 ms of 5 s and 30 s), and invalidation ended that delay at once, or within 1 s on a replica sharing the cache. Two sources of truth were stale themselves. Under SpiceDB's default `minimizeLatency`, revoked chunks were served for a median 3.3 s and, in one of 40 trials of the first run, 8.7 s, beyond the 6.5 s we had registered as the limit (the documented 5.5 s plus 1 s for polling); a rerun stayed within 5.2 s. Cerbos took a median 2.5 s to reload a rewritten policy file, and states no bound. In front of these two engines, the code we ran violated its stated bound: after an invalidation the next call refilled the cache from the stale snapshot and held it for the full time to live, so 79 of 80 SpiceDB revocations with invalidation exceeded the bound, with medians of 30 and 31 s and a maximum of 38.4 s. The fix caches nothing for the engine's declared staleness after an invalidation, and a rerun confirmed it: all 160 SpiceDB and Cerbos revocations with invalidation took effect within the declared staleness plus 1 s, and 1 s more on the replica.

Count certificates. Security needs no knowledge of $R_\tau(s)$; completeness does. On a short answer, a layer can set $c = 1$ count-honestly only if (a) the store searched exactly under φ_s, deep enough that k candidates survived the re-check or none remained, or (b) a filter-only count shows that every chunk passing φ_s was among the candidates. Conversely, a count of at least k alongside fewer than k candidates proves under-retrieval; OpenSearch uses this test to switch to exact search for its Faiss engine, unless a setting disables it [112]. Both certificates are relative to φ_s, so completeness with respect to $R_\tau(s)$ also needs a filter free of false denials, which we test offline rather than certify at run time. A layer that marks a short or empty approximate result complete is not count-honest; the release of WARDOWL we ran end to end was not, and in the runs of Section 9 it reported empty answers as complete for up to ten questions per run, from people who could read 624 chunks each.

C.3 The threat model in full

Adversary. The adversary is an authenticated subject, or an agent or application acting for one, who may issue any number of arbitrary queries through the layer, chosen adaptively and including adversarial and red-team prompts, to obtain the content of any chunk outside the subject's readable set, across users or tenants, or to learn whether such chunks exist near a query. The adversary does not control the authorizer, the store or the layer's host. Observation 1 covers adaptive queries, but we ran no adaptive adversary: our measurements draw questions from a generator, whose red-team prompts change only the query text, and describe that generator only (Section 2.4).

In scope. (i) Cross-user and cross-tenant disclosure through retrieval results. (ii) Faults in the composed components: errors in an authorizer's filter compilation; filter-semantics bugs and gaps in stores, such as type coercion, Unicode and case handling, and nested field names; and approximate-index behavior under selective or misaligned filters. (iii) Authorizer outages, slow answers and malformed answers, which must fail closed. (iv) Identity confusion: a retrieval tool that takes a user identifier as an argument lets a prompt-injectable model choose whose permissions apply, a confused deputy [65]; the layer takes the subject only from a verified token, following RFC 8693 and the MCP authorization specification [70, 95]. (v) Revocation: a permission removed after indexing must stop admitting chunks within the δ the serving path states. Context added after the vector stage, such as parent documents, neighboring chunks or graph hops, is covered only if it passes through the same re-check [160].

Out of scope. (i) Prompt injection carried in readable content, poisoning of readable content [191], and anything the model does with retrieved content, including exfiltration after correct retrieval as in EchoLeak [101]; we report that the model's answers carried the injected notice's marker whenever it was retrieved (Section 9) to mark where our claim ends. (ii) Wrong upstream permissions: over-broad ACLs are enforced faithfully, as by assistants that inherit source permissions [90]. (iii) Extraction or membership inference over readable content [100, 182]. (iv) Infrastructure: compromise of the store, its credentials, backups or hosts. Because embeddings can be inverted [98, 185], our guarantee covers the query path only; the index needs storage-level protection, and the layer never returns or logs raw vectors, query embeddings included. (v) Several participants per interaction, where content must be authorized for each [20]; we model one subject per request.

Side channels. Observation 1 says nothing about what the timing or shape of an answer reveals. Widening rounds, latency and the flag c depend on how many unreadable chunks lie near the query, and a filtered graph traversal may depend on the unreadable nodes it passes. Hybrid scoring that mixes BM25 or IDF-weighted sparse scores, or normalizes or fuses scores over candidates that include unreadable chunks, reopens Büttcher and Clarke's collection-statistics channel [23]; the dense similarity of one query and one chunk does not, because it ignores collection statistics. We do not assume these channels away: Section 8 measures them, and we say that such a channel *reveals* information about unreadable chunks, reserving *leak* for returned content.

C.4 Metrics in full

Metrics use the policy oracle, which evaluates P without calling the layer, and exact search over $R_\tau(s)$ by exhaustive scoring. *Selectivity* is the fraction of the corpus that passes a filter ($|R_\tau(s)|/|D|$ for the permission filter), following ACORN [120]; some systems use the complement [164].

Leaks, and bounds per question and per subject. A *leak* is a returned chunk outside $R_\tau(s)$ as the oracle decides it, or a canary string from an unreadable document found in a hit, a prompt or an answer; canaries follow the Secret Sharer's inserted secrets [25], used as a deterministic enforcement check rather than a memorization estimate. When no leak occurs, the unit of the bound matters. Hits are not independent trials: they cluster by question and by subject, and a fault in the layer would affect a question or a permission shape, not a random hit. We therefore state the leak rate per question and per subject, with the one-sided 95% bound $1 - 0.05^{1/n} \approx 3/n$ for zero events in n independent trials, the rule of three [64]. For the end-to-end runs of Section 9 this gives about 8.0×10^{-4} per question for each configuration (3,746 distinct pairs among 4,420 questions), 1.6×10^{-2} per subject (the scenario's 189 people, who recur across runs) and 0.21 per role type (their 13 types), not the 3.8×10^{-6} that 784,028 checked hits would suggest; for the injected faults of

Section 5.1, about 2.8×10^{-3} per fault episode (1,050 episodes), or 0.21 over the 13 fault kinds they replay. Independence holds only in part for each of these, and a deterministic predicate does not generate Bernoulli trials. The numbers describe the question generator and the injected faults; they say nothing about an adaptive adversary, for which only Observation 1 and its trusted computing base speak. The *violation rate*, the share of store candidates the re-check rejects, measures how far the pushed-down filter over-approximates, not the security of the layer.

Permission-recall@k. For a question whose subject can read something, permission-recall@k is $|H \cap T^\star_\epsilon| / \min(k, |R_\tau(s)|)$, with $T^\star_\epsilon$ as in Definition 1 and $\epsilon = 10^{-4}$, so that ties do not penalize a store. Measured against exact search over the readable set rather than the store's filtered set, it charges filter false denials and index misses alike; recall against the store's own exact search under the same filter separates the two.

Under-retrieval and false completeness. The *under-retrieval rate* is the share of retrievals that are not complete, returning fewer than $\min(k, |R_\tau(s)|)$ hits, and the *empty-result rate* the share returning none to subjects who can read something. A count can detect under-retrieval, but mis-retrieval, where k hits come back but not the right ones, shows only in recall; in the enterprise scenario it, not empty results, caused most of the recall loss (Sections 6 and 7). We report the confusion matrix of c against the oracle; the *false-completeness rate*, the share of results with $c = 1$ that are not maximal, measures how far a count-honest flag stays from certifying maximality.

Latency. End-to-end latency per retrieval (filter compilation, search, re-check and widening) is reported at p50, p95 and p99, each the median of three passes after warm-up, with thread count, batch mode and cache state stated; widening rounds and authorizer calls are recorded for the side-channel analysis.

A measured correlation axis. Recall under a filter depends on selectivity and on where the readable set lies relative to the query. Prior work calls the second property query correlation and controls it explicitly [2, 81, 120, 188]; our "aligned" and "misaligned" permission layouts are positive and negative instances of it, not a new axis. We therefore place each (question, subject) pair on measured axes: global selectivity; ACORN's query correlation, which compares the query's distance to the readable set with its expected distance to a random set of equal size [120]; Global-Local Selectivity [2]; HCBGen's α-hardness [81]; the local selectivity ρ_m, the share of readable chunks among the m nearest unfiltered neighbors ($m \in \{100, 1000\}$); and the unfiltered rank of the first readable chunk. Because selectivity and correlation alone are unstable proxies for difficulty [81], we report results against several of these. Rates carry 95% confidence intervals from a cluster bootstrap over subjects, and comparisons between configurations are paired on the same questions.

D The enforcement layer in detail

This appendix gives the details behind Section 3.

Table 6 separates what has precedent from what we add.

D.1 Contracts and filter semantics

For a subject s and an action, an *authorizer* offers check(s, H), which every authorizer implements and which returns one Boolean per hit, in order; filter_for(s), which returns a filter φ_s selecting *exactly* what s may read, or nothing; and, optionally, scope_for(s), which returns a *necessary* condition ψ_s, such as the subject's tenant, that every readable chunk satisfies. Besides ACLs copied

Component	Precedent	Role in this paper
Pre-filter: the access predicate pushed into the query	Query modification [150]; query integration in multi-user full-text search [23]; vendor pre-filter integrations [11, 29]	Mechanism under test
Widening post-filter	Late binding with growing candidate blocks [18, 87]; over-fetching two to three times [109]	Mechanism under test; its rounds are a side channel (Section 8)
Re-check of every candidate	Late-bound checks per result [118]; real-time verification of pre-filtered documents [3]	Violations as an online count of filter false admits (Observation 1)
Failing closed	Withholding documents on transient errors [3]; an error instead of a partially filtered result [91]	Mechanism under test
Truman answers over the readable set	Truman and non-Truman semantics [137]	The flag c as a partial step toward non-Truman answers
Three-operator filter algebra with one reference semantics	Per-store filter languages	**Contribution**: the oracle for differential testing across the 14 stores the layer supports, 13 of them run (Section 4)
Count-honest completeness	Exact search when fewer than k approximate results return, in OpenSearch's Faiss engine [112]	**Contribution**: an exactness declaration and a flag that never reports a short approximate answer as complete (Appendix D.3 and Section 7)
Composition and tenant scopes	Policy-combining algorithms [103]; denials that override grants [3]	Engineering; described, not claimed
Decision cache with a revocation bound	Snapshot tokens against applying stale ACLs [118]; revocation defeated by a cached index [139]	Engineering; described, not claimed
Subject from a verified token	Token exchange [70]; MCP authorization [95]	Adopted, not claimed

Table 6: Components of the enforcement layer, their precedents, and their role in this paper.

into chunk metadata, user functions and tenants, WARDOWL supports seven engines: its adapters for OPA (through a filter document its policy returns), OpenFGA, SpiceDB and Cerbos offer filters [13, 28, 108], and those for Ory Keto, Cedar and Casbin offer checks only. This describes the adapters, not the engines: Cedar exposes `is_authorized_partial` and an experimental type-aware partial evaluator [26], which the adapter does not use. A *search backend* declares whether search(q, k, φ) applies a filter inside the search, and returns at most k hits, best first, with metadata exactly as stored, since the re-check decides on it (A2). Adapters cover 14 stores (pgvector, Qdrant, Milvus, MongoDB Atlas, Weaviate, Redis, LanceDB, FAISS, Vespa, Cassandra, Pinecone, Elasticsearch, OpenSearch and Chroma), and an in-memory backend that evaluates the reference semantics is their oracle. One algorithm, written as a generator of operations, runs under a synchronous and an asynchronous driver, so both share the security logic.

Filters are built from $\mathrm{In}(f, V)$, And and Or. Let $\mathcal{V}$ be the strings and integers, Booleans excluded, with strings compared by code point, without case folding or Unicode normalization, and never equal to integers. One function, `matches`, defines $m \models \mathrm{In}(f, V)$ as in Equation 1, with And and Or as conjunction and disjunction. Every store translation must agree with it, and it is the oracle of Section 4. Each design rule closes a leak. (i) Empty And and Or are rejected, since an empty conjunction is vacuously true and would allow everything. (ii) Values are strings or integers only: Python equates `True`, `1` and `1.0` while JSON-based stores compare by type, so floats, nulls, Booleans, nested lists and objects never match, and a bare string such as `"12"` is rejected rather than read as `"1"` and `"2"`. (iii) A missing field never matches, so a chunk without permission metadata is readable by nobody. (iv) A filter that can match nothing, such as an In with no values, makes translators raise `NothingMatches` and backends return nothing without a query, because Pinecone, MongoDB, Milvus, LanceDB, Redis and Vespa read a missing or empty filter as "match everything". Parsing rejects unknown operators and nesting deeper than 100 levels.

D.2 Retrieval

By default WARDOWL pre-filters when the backend supports filters and the authorizer returns φ_s, and post-filters otherwise, inside ψ_s when one is offered; a caller who demands pre-filtering gets an error rather than a silent fallback.

Pre-filter and re-check. One search with φ_s returns the store's answer H_0, truncated to k so that a store that ignores k cannot widen the checks. Every hit is checked again; hits that fail are dropped, not replaced, and counted as *violations*. Because filter and check are separate code paths, a violation is, at a single revision, a false admit by the filter or its translation, observed online (Observation 1). With the re-check off, security rests on the filter. Hits are still matched against φ_s locally, so a store that ignores the filter cannot return a chunk outside it, but a filter that over-approximates the policy, such as the Cerbos translation of Appendix E.2, is then returned unchecked.

Widening post-filter. Without a filter, WARDOWL fetches $k \cdot o$ candidates ($o = 4$ by default), checks those not seen before, and multiplies the fetch by o until k hits are permitted, the store returns fewer candidates than requested, or a cap is reached (1,000 by default; a larger k is refused). A redactor and a reranker, if configured, receive only permitted hits and cannot add one.

Failing closed. Any exception from the authorizer ends the request with an authorization error and no hits. An exception the adapters did not raise themselves becomes a fixed message ("authorizer failed"), the original kept only as its cause, and the adapters' own messages come from a fixed set that names no host, because agent frameworks show tool errors to the model and exception text can name hosts or credentials; Section 5.1 checks this under injected faults. Malformed answers fail likewise: a filter that is not an In, And or Or, a decision list of the wrong length, or any decision that is not a Boolean (`1`, `"no"` and `None` are rejected, not coerced).

D.3 Count-honest completeness

Every result carries the flag c of Section 2.1. The release we ran end to end (2.1.0) set $c = 1$ for any pre-filtered result without violations, an empty one included, and for any post-filter whose last search came back short, so it was not count-honest: in the runs of Section 9 it reported empty answers as complete for questions from least-privileged people who could read 624 chunks each. Section 6.3 relates those answers to pgvector's plan cache and the scan's two limits. The current code (PR #166, not yet in a release) lets a backend declare `exact_search` when it ranks every chunk that satisfies the filter, so that a short answer proves nothing else matches; only the in-memory reference and FAISS over a flat index (its default) do. With e the declaration and v the number of violations, a pre-filtered result has $c = (|H_0| = k \vee e) \wedge (v = 0)$; a post-filtered one has $c = 1$ only if k hits were permitted, or the last search came back short before the cap and either no scope applied or e holds. The flag is thus count-honest relative to φ_s (Definition 1), and only to it. It does not imply completeness against $R_\tau(s)$, since a false denial in φ_s shrinks $E(\varphi_s)$ unseen, and it does not imply soundness: a full answer is taken at the store's word, so $c = 1$ does not certify maximality, and we measure mis-retrieval offline (Section 6). One assumption remains: an unfiltered post-filter reads a short list as the end of the store, which fails for pgvector with iterative scans off (at most `hnsw.ef_search` rows, 40 by default). Count-based certification and a bounded exact fallback, as in OpenSearch's Faiss engine [112], are evaluated in Section 7.

D.4 Composition, tenants and caching

Composition. AllOf admits a chunk when every part does, asking each part only about chunks that the earlier parts admitted; AnyOf asks each part only about chunks still denied. Their filters, the conjunction or disjunction of the parts' filters, exist only when every part has one. Their scopes are asymmetric. A scope is valid when $R_\tau(s) \subseteq E(\psi_s)$. Since $\bigcap_i R_i \subseteq R_j$ for every j, where R_i is the readable set under part i, AllOf may conjoin whichever filters or scopes its parts offer; since leaving a part out of a scope for $\bigcup_i R_i$ can hide readable chunks, AnyOf offers a scope only when every part offers a filter or a scope. Every candidate is still checked, so a wrong scope costs speed or recall, never security. The tenant authorizer admits chunks whose tenant field equals the subject's tenant and raises when the subject has no usable tenant, so a request that lost its tenant returns nothing rather than everything; combined with an engine that offers no filter, it becomes the post-filter's scope.

Caching. A caching wrapper stores, per subject and action, the filter, the scope, and each decision keyed by the chunk's identifier and metadata. Its time to live (30 s by default) is a stated revocation bound, and an explicit invalidation ends it at once, also for answers that calls in flight obtained before it, provided the engine answers from current data. In the code we ran in the engine study, a branch of release 2.1.0 (Appendix A), an engine that serves a stale snapshot after the revocation can refill the cache, which then holds the stale answer for a full time to live, a violation of the stated bound; Section 5.1 found this, and the fix it evaluates caches nothing for the engine's declared staleness after an invalidation, so the bound becomes that staleness. Keys encode type and length, so `1`, `"1"`, `True` and `1.0` never share an entry, and failures are never cached. Entries shared through Redis carry their absolute expiry, so the bound holds across replicas, and, with a signing key, an HMAC-SHA256 over key and content, so whoever can write to Redis cannot plant a decision. The bound covers the cache alone: engine staleness [9] and the freshness of ACLs copied into chunks add to it. With the cache, the layer is $(\ell + \delta_e)$-fresh rather than δ_e-fresh (Section 2.2), and the cache joins the trusted computing base; its invalidation needed three successive fixes for races that stored a pre-invalidation answer after the invalidation.

D.5 Identity and audit

No interface lets a client or a model choose the subject. The MCP search tool takes only a query and a bounded result count, derives the subject from the access token the server verified for the request, and refuses calls without one [95]. When an agent acts for a user, the token is issued for the user, for example by token exchange, with the agent as the client or actor [70]; the agent then holds no permission its user lacks [94], and no prompt can redirect the tool to another user's permissions, the confused-deputy pattern [65]. The HTTP service for other languages verifies OIDC bearer tokens, each with its key's own asymmetric algorithm, and refuses bodies with unknown fields, so a request cannot name a user, group or tenant. Every call, failed and cancelled ones included, emits one event with the subject's identifier, the returned identifiers, the counts, c, timings and the error's type, but never query text, query vectors or chunk text; given a key of at least 16 bytes, it adds an HMAC-SHA256 of the query, so queries can be correlated without being stored. The violation count and c are the online signals that Sections 6, 7 and 9 compare with offline oracles.

E The conformance kit and campaign in detail

This appendix gives the kit and the campaign that Section 4 summarizes. The kit, `check_backend` and `check_authorizer`, ships with the library so that third-party adapters meet the same standard,

and continuous integration runs it against a real instance of every built-in store except Pinecone, for which we have no API key, and against the built-in authorizers.

E.1 Method

Two oracles. The reference gives the expected answer $E(\varphi) = \{d \in D : m_d \models \varphi\}$, which the in-memory backend computes. Because the reference is also code, end-to-end checks add an oracle written without the library's filter code: s may read d if s is among d's users, or if d's groups contain the everyone marker or one of s's groups. The retrieval checks use the metadata ACL authorizer, whose filter and check both evaluate the reference, so the second oracle also tests the first. Answers are compared as sets, and order is checked as a property, since stores score differently.

An adversarial generator. From a seed, the kit draws corpora of up to 20 documents (by default 25 corpora with four filters each), each document holding zero to four of seven fields, so missing fields are common. Values are strings, integers, Booleans, lists of either, nested lists or objects, drawn from confusable pairs: `1` and `"1"`; `"a"` and `"A"`; and é as one code point (U+00E9) or as `e` with a combining accent (U+0301). The ACL documents repeat the pairs in user and group names (`bob` and `Bob`, `eng` and `Eng`, a group named `"1"`). A pair is equal only for a store that converts types, folds case or normalizes Unicode. The field `team.name`, generated beside objects such as `{"name": "a"}` under `team`, catches stores that read dots as paths, and `größe-x` lies outside ASCII. Filters nest And and Or two levels deep; an In holds zero to three values, so empty In filters occur; about one In in ten adds 300 values that match nothing, and about one subject in five belongs to 300 extra groups. A store may refuse a field name it cannot express, returning nothing, but may not answer wrongly. Every failure reports its seed and example, so that it can be replayed.

What the kit checks. For each corpus, `check_backend` fills a fresh store and checks (1) unfiltered and (2) filtered searches against $E(\varphi)$: at most k hits, no duplicates, no document outside $E(\varphi)$, $\min(k, |E(\varphi)|)$ of them, finite scores in non-increasing order, and text and metadata identical to what was stored, since the re-check decides on that metadata; (3) that the asynchronous search returns the same identifiers, order, text, metadata and scores, and refuses the same filters; (4) that replacing metadata keeps every text, leaves no old key, creates no document and changes what filters select as the reference says; and (5) that retrieval through the layer, pre-filtered and post-filtered and through both drivers, returns no hit the second oracle denies, no violation, $\min(k, |R(s)|)$ readable hits, and the same answer from both drivers. Checks (1) and (2) draw k for each search: it equals the corpus size for one of the two unfiltered searches and for half of the filtered ones, so that every match must come back and a false denial cannot hide outside the top k, and is otherwise uniform from 1 to the corpus size; check (5) draws k from 1 to 5. The counts are required in exact mode. Stores whose index is approximate even on these small corpora (Cassandra, and Vespa and Redis with HNSW) run in a sound-only mode, in which answers may be short but never wrong; a store that declares exact search is always checked exactly, because the layer reports its short answers as complete (Appendix D.3). `check_authorizer` requires one Boolean per hit; answers independent of the rest of the batch and its order (all hits, each hit alone, and the batch reversed); asynchronous answers equal to synchronous ones; and check agreeing with $m \models \varphi_s$ on every hit whenever a filter is offered. It empties every cache before each call, so that a cached answer cannot mask a batch-dependent one. It shows consistency, not a correct policy: an authorizer that lets everyone read everything passes, and so does a scope narrower than $R(s)$. OPA, OpenFGA, SpiceDB and Cerbos are additionally compared with a reference answer against their real servers.

Translations without injection. Ten stores (Elasticsearch, OpenSearch, Milvus, MongoDB, Weaviate, Redis, LanceDB, Vespa, Cassandra and Pinecone) index one field of type-tagged terms $f \triangleright t{:}v$, one for every string or integer value or list element, where $\triangleright$ is a control character (U+001F) and t a type letter, as in `level▷i:1`. An In becomes set membership on these terms, and And and Or become the store's own connectives; equality is exact in type and value, untagged values (Booleans, floats, nested values) never match, and the metadata itself is stored unindexed and returned as written. Where terms enter query text (Redis, LanceDB and Vespa), and in Pinecone, they are hex-encoded, so they contain only digits and the letters a to f. Elsewhere values are bound parameters (pgvector, Milvus and Cassandra) or data structures of the client API (Elasticsearch, OpenSearch, Qdrant, Weaviate, Chroma and MongoDB), and FAISS applies the reference in process. Filter parsing and the translators that write query text are fuzzed with Atheris [58]: parsing must round-trip or raise, pgvector must bind every field name and value, and LanceDB and Redis queries must contain only hex-encoded terms.

Testing the tester. At the commit that the campaign's second run tested (182bcbc), the kit's own suite plants 18 faulty backends and six faulty authorizers that the kit must reject (17 and six at the first run's commit, 5d0f414, before the rule for stores that declare exact search), beside positive controls it must accept: the reference backend, the built-in authorizers, an approximate backend that misses matches but never errs, and a backend that refuses dotted names. The backends ignore the filter, alter metadata, ignore k, misorder hits, miss matches while checked as exact or while declaring exact search, diverge on the asynchronous path in six ways, match after case folding, Unicode normalization or conversion of integers to strings, read dots as paths, or replace metadata by merging old keys or by creating documents. The authorizers let filter and check disagree, depend on a hit's position in the batch (alone or behind a cache), or answer differently on the asynchronous path (alone or behind a cache). Several meta-tests record a gap the kit once had: no `1/"1"`, case or Unicode pairs in its data; comparing only the identifiers of filtered asynchronous searches; skipping the asynchronous search when the synchronous one refused a filter; and cached answers hiding batch dependence. In this history the kit's blind spots lay in what it generated and what it compared; none has yet been found in its oracle.

E.2 A controlled campaign

Setup. We ran the kit against current releases of the stores and engines the layer supports, with fixed versions, seeds and counts, in GitHub-hosted runners. There were 29 targets, of which 28 ran. Thirteen stores were tested: pgvector 0.8.7 on PostgreSQL 17.11, Redis 8.10.2, Cassandra 5.0.9, Elasticsearch 9.5.5, OpenSearch 3.9.0, Vespa 8.763.13, Qdrant 1.19.2, Milvus 3.0.2, Weaviate 1.39.12, MongoDB Atlas Local 8.0.30, Chroma 1.5.9, LanceDB 0.40.0 and FAISS 1.15.1, beside the in-memory reference as a control. The fifteenth store target, Pinecone, needs an API key we did not have and was skipped in both runs, so the layer's Pinecone adapter is not covered here. The 14 engine targets were OPA 1.21.2, OpenFGA 1.22.0, SpiceDB 1.56.2, Cerbos 0.56.0, Cedar (cedarpy 4.12.2) and Casbin (casbin 1.43.0), each with a metadata-ACL policy and a tenant policy, plus Ory Keto 26.2.0 and the in-process metadata-ACL authorizer as a control. Container image digests are recorded with the results. Each store target ran 20 seeds of nine data variants and each engine target 20 seeds of seven (Appendix E.3). A *run* is one (target, seed, variant) and takes the worst outcome of its examples: *pass*; a *conformance violation*, the kit's first disagreement with the reference; a *refusal*, a `ValueError` by which the layer declines data it cannot represent; or an *error*, anything else. Engines run in consistency mode, which compares each engine's filter API with its own check API on every hit.

Run 1. The first run (wardowl 2.1.0) completed 4,315 runs: 3,612 passed, 84 were violations, none was refused and 619 were errors. Both controls passed every run. No target returned a hit outside the reference, and none altered stored metadata. The violations are of two kinds. *The layer's Cerbos adapter over-approximated* in 39 of the 40 runs whose ACL fields hold a scalar instead of a list (the fortieth, one seed of the tenant policy, passed). The adapter's translator maps CEL's `in`, `==` and `hasIntersection` to In, which also matches a scalar: for a chunk with `allowed_users` equal to the string `"zoë"`, the filter admits the user, as the reference does, while Cerbos's check evaluates the policy's CEL `in`, which is defined on lists and maps only, raises, and denies. The policy is the authority, so the filter over-approximates; the re-check drops every such hit, and with the re-check off they would be returned. This is a type gap in the adapter's translator, not a fault of Cerbos, which behaves as it documents. The project's review had found the gap and the adapter's documentation records it, so the campaign reproduced it rather than discovered it. *Redis and MongoDB missed matching chunks* in 26 and 19 runs with long and huge string values, always passing the sound-only retry. Both stores truncate a long term when they index it and look up the whole term when they search: RediSearch keeps at most 4,096 bytes of a tag value, which the layer's hex encoding reaches at 2,048 bytes of field and value, and the vector-search index's filter fields keep 8,191 characters (a reproducer without the layer measured the cut; the campaign's diagnosis had assumed the 8,181 that MongoDB documents for another field type). A value equal to the kept prefix of a longer one also matches it, a false admit that the re-check should catch; the reproducers, which do not use the layer, show the admit, while the campaign's generator holds no such pairs. Neither limit is in the documentation of the field type the layer uses.

Of the errors, 479 came from OpenFGA and SpiceDB, which refused whole batches with HTTP 400. OpenFGA accepts at most 100 contextual tuples per request, and one generated subject in five is in more than 300 groups; long identifiers exceeded its size limits. SpiceDB object identifiers admit only a restricted alphabet, which excludes `zoë` and `7.0`. The layer failed closed in every case, but only after a round trip, with an error that did not name the limit. The other 140 errors came from the stores' own limits, which fail loudly and return nothing: Lucene's 32,766-byte term on write in OpenSearch (20 runs) and Elasticsearch (16 runs; its other four failed on a query, with `x_content_parse_exception`, in the same huge-string variant), Milvus's array-element and JSON sizes (40 runs), and in 20 runs each, Chroma's rules for metadata keys, which refused the write, SQLite's expression depth of 1,000 under Chroma, which refused the query, and Qdrant's grammar for field names. Ory Keto and Casbin ran out of the shared time budget after 72 and 43 of 140 runs, all passing.

Fixes and run 2. Four of these called for a change in the layer's adapters, and we made it: OpenFGA and SpiceDB now refuse a subject they cannot represent before sending anything, with an error naming the limit, and Redis and MongoDB store and search a term above their limits as its SHA-256 digest, leaving shorter terms unchanged. The second run tested the fixed adapters with the harness's budgets split across more jobs and every finding recorded in full; it changed nothing else, and every outcome matched what the diagnosis predicted. Of 4,480 runs, 3,822 passed, 39 were violations, 479 were refusals and 140 were errors. Redis and MongoDB passed every long- and huge-string run. The 479 OpenFGA and SpiceDB runs that had failed with HTTP 400 became refusals by the layer, while the same 21, 21, 16 and 23 runs of the four OpenFGA and SpiceDB targets passed as before. Those refusals are 85.5% of these engines' 560 runs. A run takes its worst subject's outcome, so the other subjects of a refused run still reached the engine, but only 81 runs reached it with every subject: the campaign's evidence on these two engines is thin, and Section 5 tests them on data within their limits. Keto and Casbin completed all 140 runs each, every one passing. The 140 store errors were the limits above, unchanged. The 39 Cerbos runs disagreed again,

and the harness now attributes each disagreeing hit: in all 280, the pushed-down filter gave the reference answer and Cerbos's check denied, so the filter over-approximated the policy as written and never the reverse. No run in either campaign returned a hit outside the reference or altered metadata.

E.3 The kit's data variants

Each store target ran 20 seeds of nine variants: the kit's default data (ten corpora per seed) and eight stress variants (three corpora per seed): corpora of up to 150 documents; filters nested six deep; disjunctions over 100 and 1,000 values; strings up to 40,000 characters (long and huge); case and Unicode confusables; numeric strings next to integers; and unusual field names. Each engine target ran 20 seeds of seven variants: default; case and Unicode; numeric strings; 1,000 groups; 300 chunks per check; long identifiers; and scalar instead of list ACL fields. A run is one (seed, variant) and takes its worst subject's outcome: pass, a conformance violation, a refusal by the library (a `ValueError` for data it cannot represent), or an error. The in-memory reference store and the in-process metadata-ACL engine are controls.

F Faults and semantic gaps: the full table

Table 7 lists the faults and semantic gaps in stores and engines that the project's release notes, code comments and documentation record, up to the version we evaluate, and those that this paper's experiments exposed. *Found by* separates the two. Findings of this paper's method are marked *Kit*, the campaign of Appendix E.2, or §n, the section or appendix of another experiment. The project's history before this paper follows its release notes: *Review*, attributed collectively to the project's own code review or security audit; *Accept.*, to the acceptance scenarios against real servers; *n.s.*, not stated. *Effect* is the consequence without the mitigation: *Leak*, a false admit, which the re-check turns into a violation; *Leak**, wrong stored or returned metadata, which also passes a re-check that reads chunk metadata, because (A1) or (A2) fails; *Leak†*, a check that itself admits, beyond any re-check; *Recall*, readable chunks silently missing; *Stale*, a revoked grant that still admits for a while after the change; *Wrong rows*, a write reaching chunks it did not name; *Avail.*, requests failing closed; *Perf.*, slower retrieval, as without a pre-filter; and *Crash*, the client process killed. PR #n marks a mitigation merged on 2026-10-30 and not yet in a release. *None (adapter)* marks a fault in WARDOWL's own adapter or cache where the engine behaves as configured.

Table 7: Faults and semantic gaps in vector stores (top) and authorization engines (bottom), collected from the project's release notes, code comments and documentation and from this paper's experiments; the legend is above. No row has been confirmed by the maintainers of the store or engine.

Store or engine	Fault or semantic gap	Found by	Effect	Mitigation in WARDOWL	Upstream
Elasticsearch, OpenSearch	Keyword fields compare as strings: a `terms` query for `1` matches `"1"`	n.s.	Leak	Type-tagged terms in one keyword field; metadata unindexed	Not stated [46, 113]
pgvector (jsonb)	Numbers compare by value: In(`level`, {1}) matched a stored `1.0`	Review	Leak	Compare integers by the stored number's text	Documented [155]
pgvector (jsonb)	A float of magnitude $\geq 10^{16}$, which Python's JSON encoder writes with an exponent, is stored and returned as an integer	Review	Leak*	Write such floats with a fraction	Documented [155]
pgvector 0.8	Iterative HNSW scans stop at `hnsw.max_scan_tuples` or a memory limit: few or no rows under selective filters	§9	Recall	Honest completeness; both limits documented	Limits documented [121]

continued on the next page

Table 7, continued

Store or engine	Fault or semantic gap	Found by	Effect	Mitigation in WARDOWL	Upstream
pgvector via psycopg	Prepared statements: after five custom plans PostgreSQL may switch to a generic plan that reads the GIN index and sorts (exact, about 50 times slower) or keep custom HNSW plans (lossy), decided at run time	§6.3, §9	Recall or Perf.	None yet	Rule documented [156, 159]
Chroma 1.5.x, local mode	Array metadata of a deleted collection reappears on new records elsewhere and matches `where` filters (chroma-core/chroma#7594)	n.s.	Leak*	No array metadata; metadata rebuilt from an exact JSON copy	Open; fix unmerged [146]
Redis; Weaviate	Distinct values merge: TAG matching ignores case by default; `field` tokenization trims whitespace, joining `"hr "` and `"hr"`	n.s.	Leak	Hex-encoded terms (Redis); a separator at both ends of each term (Weaviate)	Redis: documented [131]; Weaviate: in code [171], not docs [170]
Redis 8.10	Tag values are cut at 4,096 bytes when indexed and looked up whole: long values never match, and a value equal to the kept prefix matches the longer one	Kit	Recall; Leak	Store and search a term over the limit as its SHA-256 digest (PR #167)	Reported 2026-10-30 (RediSearch issue #12106)
MongoDB Atlas 8.0, 8.3	Strings in vector-search filter fields are cut at 8,191 characters when indexed, undocumented (8,181 is documented for another field type): the same effects as for Redis	Kit	Recall; Leak	As for Redis (PR #167)	Documentation feedback sent 2026-10-30
Elasticsearch, OpenSearch, Milvus, Qdrant	Size and syntax limits refuse a write or query: 32,766-byte terms; array-element and JSON sizes; field-name grammar	Kit	Avail.	None needed: each fails closed with an error and returns nothing	Term limit [45] and JSON size [93] documented; Qdrant's grammar in its source
Chroma 1.5.9, local mode	A flat `$or` of about 1,000 clauses exceeds SQLite's expression depth; one of about 16,500 or more kills the client process (SIGSEGV)	Kit; §J	Avail.; Crash	Nest `$and` and `$or` 16 to a node; past four size bounds, filter 1,000 candidates itself (PR #187)	Reported 2026-10-30 (Chroma issues #7969, #7970)
Qdrant	`.`, `[`, `]` and `"` in a field name are path syntax: `team.name` reaches a nested object	n.s.	Leak	Refuse such field names	Path syntax documented [125]
Pinecone, MongoDB, Milvus, LanceDB, Redis, Vespa	A missing or empty filter means "match everything"	Review	Leak	Raise `NothingMatches`; send no query	Filters optional, e.g. [97, 123]
Elasticsearch, OpenSearch, Redis, Vespa	Partial results as success: failed or timed-out shards under HTTP 200; RESP3 timeout warnings; degraded coverage	Review	Recall	Disallow partial results; raise on each marker	Documented [47, 114, 130, 163]
Pinecone serverless; MongoDB Atlas	Writes become searchable only after a delay (eventual consistency; index lag)	n.s.	Stale	Writes wait until a search sees them (Pinecone: up to 30 s)	Documented [96, 122]
Cassandra 5.0	Paxos timestamps of lightweight transactions do not order with plain writes: an older permission could overwrite a newer one	Review	Leak*	Plain writes only	Documented by DataStax [40]
Milvus (pymilvus)	Reads and deletes by identifier quote identifiers into an expression: the identifier `a','b` also selects `a` and `b`	Review	Wrong rows	Identifiers as filter parameters	pymilvus PR #3823 open [153]
OpenFGA	ListObjects stops at its result limit (1,000 by default) and returns a partial list without saying so	n.s.	Recall	At the limit, no filter: post-filter	Limits documented [106]

continued on the next page

Table 7, continued

Store or engine	Fault or semantic gap	Found by	Effect	Mitigation in WARDOWL	Upstream
OpenFGA	ListObjects also stops at its deadline (3 s by default), again without saying so	Review	Recall	After the deadline, no filter: post-filter	Limits documented [106]
SpiceDB	LookupResources may stop at the request limit (1,000 by default) or return results conditional on a caveat	n.s.	Recall; Leak	In either case, no filter: post-filter	Documented in the API [12]
SpiceDB (adapter)	An empty LookupResources stream, which is also what a stream cut before its first line looks like, was read as "nothing readable"	§5.1	Recall	No filter: post-filter, checking every candidate (PR #186)	None (adapter)
SpiceDB (adapter)	Groups checked as separate subjects bypass exclusion: under `viewer - banned`, a banned user read through a group	Accept.	Leak†	Check the user only; SpiceDB resolves the memberships it stores	None (adapter)
SpiceDB 1.56	Under `minimizeLatency`, a revoked permission admitted for 8.7 s in 1 of 40 trials, past the 5.5 s its documented defaults imply, which we have not explained (a rerun's 40 trials: at most 5.2 s), and could reappear after it had gone	§5.1	Stale	Declare 11 s of staleness; after `invalidate()` the cache holds back that long (PR #186)	Bound documented [9, 14]
SpiceDB, Cerbos (cache)	After `invalidate()`, WARDOWL's cache refilled from the engine's stale answer and held the revoked grant for its full ttl	§5.1	Stale	Cache nothing for the engine's declared staleness after `invalidate()` (PR #186)	None (adapter)
OPA (partial evaluation)	The residual keeps `not (x in [])` even for users with no ban, and WARDOWL's filters have no negation: one ban rule leaves every user without a pre-filter	§G.2	Perf.	None; such users are post-filtered	Not stated [104, 105]
Cerbos (adapter)	The translator maps CEL `in`, `hasIntersection` and `==` to In, which also matches a scalar and matches a list by element; in CEL, `in` or `hasIntersection` on a scalar is an error (denied) and `==` compares a list whole	Review; Kit	Leak	Translate `and`, `or`, `hasIntersection`, `in` and `==` only; else post-filter; documented	None (adapter); documentation note filed 2026-10-30 (Cerbos issue #3456)
Cedar (adapter)	Values Cedar cannot hold (floats, nulls, integers beyond 64 bits) and keys starting with `__` were dropped, so a `forbid` on them stopped applying	Review	Leak†	Whole-number floats become Longs; a chunk with any other such value is denied	None (adapter)
OpenFGA, SpiceDB	An identifier outside the engine's syntax makes it refuse the whole request	Review	Avail.	Deny such a chunk without sending it	Syntax in API specs [10, 107]
OpenFGA (adapter)	More than 100 contextual tuples, or a user or group identifier over its size limit, makes it refuse the whole request	Kit	Avail.	Refuse such a subject before sending, naming the limit (PR #167)	None (adapter)
SpiceDB (adapter)	A subject or group identifier outside the object-identifier syntax makes it refuse the whole request	Kit	Avail.	Refuse such a subject before sending, naming the limit (PR #167)	None (adapter)
Ory Keto (adapter)	Batches above `limit.max_batch_check_size` (10 by default) are refused with HTTP 400; the adapter sent 100	Accept.	Avail.	Batches of 10	None (adapter)

Of the table's 31 rows, 18 record the project's development before this paper and are history, not findings of our method. Twelve are this paper's findings, six from the campaign and six from the other experiments, and one, Cerbos, the campaign reproduced. Nine entries let a filter admit a chunk the subject may not read, which leaks without the re-check and becomes a violation with it, as Observation 1 predicts. The five Leak* and Leak† entries lie beyond any re-check: the kit's

exact comparison of returned with stored metadata targets the first kind, and only a test of the policy itself reaches the second. The remaining effects cost recall, freshness, speed or availability, and one crashed the client process; where a fix was needed, the lost recall had been reported as a complete answer, which Appendix D.3 changes. Before the campaign, the record credited review or the acceptance scenarios where it credited anything, never the kit.

Counted by kind, the campaign found two store behaviors with a correctness effect, Redis and MongoDB cutting a long term at indexing, undocumented for the field type the layer uses; two availability faults in the layer's own adapters, for OpenFGA and SpiceDB; and the limits of five stores, some documented and some not (Elasticsearch, OpenSearch, Milvus, Qdrant and Chroma), which fail closed. It reproduced one known gap, in the Cerbos adapter's translator. All lie in what the generator targets, long values, many groups, unusual identifiers and scalar fields, at sizes that the continuous-integration runs had not reached. The filter-size experiment then showed that one of those limits, Chroma's, can also crash the client, which is a store behavior of its own. The other experiments added SpiceDB's staleness beyond its documented bound, in one unexplained trial, faults in WARDOWL's cache and SpiceDB adapter, a vacuous negation that OPA's partial evaluation keeps, and pgvector's scan limits and its run-time choice between a lossy and an exact plan. Of the eleven rows of Table 1, four are faults in WARDOWL's own adapters or cache. Several entries, the eight marked as adapter faults among them, are faults in WARDOWL's own code that a store's or engine's behavior exposed; any other integration meets the same behavior. Recall lost by approximate indexes under selective or misaligned filters, such as Qdrant's filtered HNSW traversal, is measured in Section 6 instead.

G The engine study in detail (X4 and X7)

This appendix gives the design and full results behind Section 5, which asks what items (iii) and (v) of the threat model (Appendix C.3) require. Hypotheses, rules and statistics were pre-registered; we report every verdict, falsified ones included (Appendix B).

G.1 Method

We tested OpenFGA 1.22.0 and SpiceDB 1.56.2 on a PostgreSQL datastore, OPA 1.21.2 and Cerbos 0.56.0 through the adapters of Section 3, in GitHub-hosted runners. For each of five seeds a generator draws 3 tenants, 120 users, 60 groups and 80 folders (both nested up to four levels) and 1,500 documents with owners, direct readers, reader groups, occasional everyone-readable grants, bans of users and groups, and a classification. A user may read a document of their own tenant that they own or that is shared with them directly, through a group, with everyone or through a folder, unless banned; engines that see attributes add a clearance condition. OpenFGA and SpiceDB hold this model as relationships (tenant intersection, `but not banned`, inheritance through parent folders, a `user:*` wildcard), OpenFGA once with stored group memberships and once with memberships sent as contextual tuples. OPA holds it as a filter document over ACL metadata, one over sharing data, and its own partial evaluation [104, 105], whose residual we translate (equality and membership to In, conjunction and disjunction as such, anything else to no filter). Cerbos derives filters from query plans [28]. OPA and Cerbos omit bans here, because the filter language (Appendix D.1) has no negation, and Cerbos also omits folders, with each principal's groups already flattened, so that profile holds a smaller model; stress profiles add bans and one-value ACL fields stored without a list. There is one generated model per profile. A separate plain Python implementation of the model is the oracle, and two controls (filter and check both the oracle; a filter with 2% planted errors) validate the counting.

Profile	Filter route	Users with a filter	False admits	False denials
OpenFGA, stored groups	ListObjects	600	0	0
OpenFGA, contextual groups	ListObjects	600	0	0
SpiceDB	LookupResources	600	0	0
OPA, ACL metadata	filter document	600	0	0
OPA, sharing data	filter document	600	0	0
OPA, sharing data	partial evaluation	600	0	0
Cerbos	query plan	600	0	0
Cerbos, one-value ACL fields	query plan	600	569	0
OPA, ACL metadata, one-value fields	filter document	600	0	0
Cerbos, bans	query plan	0	n/a	n/a
OPA, sharing data, bans	partial evaluation	0	n/a	n/a

Table 8: Filter against check at one revision: five seeds of 120 users and 1,500 documents, 900,000 (user, document) pairs in each profile that gave filters. Above the rule, each engine's check also agreed with the oracle on every pair. Below it, stress profiles.

One revision means SpiceDB's `atExactSnapshot` at the token of the last write [9, 118]; OpenFGA offers no snapshot token, so its store was read quiescent with `HIGHER_CONSISTENCY`; OPA and Cerbos are stateless per request. For every user and document we record a *false admit* (the filter admits, the check denies) or a *false denial*, and repeat each disagreement twice. Skew is measured under a writer committing 20 toggles a second, half about the user being measured; a disagreement counts as skew when the oracle's answer changes between the last write visible to the filter (allowing 10 s of staleness for OpenFGA's `MINIMIZE_LATENCY` and 5.5 s for SpiceDB's `minimizeLatency`) and the end of the check. Truncation is measured in a corpus of 100,000 documents, growing one subject's readable set from 10 to 100,000, with 30 queries per level and seed over exact FAISS search and the layer's defaults ($k = 10$, at most 1,000 candidates). Intervals are 95% cluster bootstraps over (seed, user) or (seed, query); after zero events among n clusters we give the bound $1 - 0.05^{1/n}$.

G.2 At one revision, filters agree with checks

Table 8 shows no translation fault in the seven profiles that hold the model as intended, with each ACL field in the shape the policy assumes: 6.3 million pairs produced no disagreement between filter and check, and every check agreed with the oracle (one-sided 95% bound on users with any disagreement: 0.5% per profile, over this generator's worlds; the pairs are seven policy and engine combinations repeated over five seeds, not 6.3 million independent cases). The oracle control reported nothing and the planted control exactly its 16,774 false admits and 1,233 false denials. The only disagreement at one revision is the type gap in the layer's Cerbos adapter that the campaign reproduced (Appendix E.2): the translator maps CEL's `in`, `==` and `hasIntersection` to In, which also matches a scalar, while Cerbos's check treats `in` on a scalar as an error and denies. When 10% of documents store a one-value ACL list as a plain value, the adapter's filter admits what the check denies in 569 pairs (0.063% of pairs; 52.5% of users, 95% interval 48.3% to 56.3%), always on such a document and reproducibly; the re-check turns each into a violation. OPA's policy, which reads a field as one value or a list, stays exact on the same data. Neither translator dropped a negation, but vacuously: with bans, Cerbos's plan and OPA's residual both kept a `not`, and no user received a filter, so negation in a filter is untested. For OPA this falsified our prediction that partial evaluation would simplify `not (x in [])` away for the 73 users with no ban in scope; it does not, so one ban rule makes OPA's partially evaluated filter unavailable to everyone, who are then served by post-filtering.

G.3 In practice, disagreement is revision skew

Unpinned under the writer, filter and check disagreed in every OpenFGA trial with either consistency level (9,433 and 9,972 pairs over 200 trials each), in 81.5% of SpiceDB trials with `fullyConsistent` (95% interval 76.0% to 86.5%), and in 13.0% of OPA trials (8.5% to 18.0%), whose calls are a few milliseconds apart. A write between the calls explained every disagreeing pair, and none reproduced. Pinned to one SpiceDB snapshot under the same writer, 200 trials produced no disagreement; neither did the quiet conditions. SpiceDB's default, `minimizeLatency`, disagreed in only 4.0% of trials (1.5% to 7.0%), not because it is fresher but because both calls read the same quantized, possibly stale snapshot; 226 of its 228 pairs were explained, and the other two are consistent with staleness beyond the 5.5 s allowed, which Section 5.1 measures. The attribution is generous where windows hold dozens of writes (medians of 59 and 208 for OpenFGA), so the pinned and quiet conditions carry the claim: an online violation is evidence of a translation fault only when filter and check read one revision (Section 2.2).

G.4 Truncated lists are detected, at a cost in recall

Non-streaming ListObjects and LookupResources with the layer's limit of 1,000 both returned exactly $\min(N, 1000)$ objects; ListObjects adds no other field [106] and LookupResources attaches a continuation cursor whatever N [12], so neither signals truncation. Without those limits both returned all 100,000 objects in time: streamed ListObjects in a median 1.7 to 2.0 s, within its 3 s deadline, and unlimited LookupResources in 2.6 to 3.7 s. WARDOWL returned an exact filter for every $N \leq 999$ and none for $N \geq 1000$ in all 540 calls, never a short one; over 8,100 retrievals, no result with recall below 1 was marked complete and none leaked. The cost lies at the limit. With 1,000 or 1,001 readable documents among 100,000, the post-filter's 1,000 candidates hold about ten readable ones, and recall@10 fell to 0.855 to 0.891 per level, against 1.000 with the filter below the limit and from 3,000 readable documents up. A list limit equal to the post-filter's cap thus places the loss where selectivity is about 1%; paging or streaming the list would remove it. OPA, which has no list limit, gave exact filters at every size, but over a sharing document of 100,000 entries its partial evaluation took a median 4.2 to 5.7 s, past the adapters' default 2 s timeout, so every retrieval through it failed closed and its recall was not measured.

G.5 Latency and caching

An uncached filter took a median 0.9 ms (OpenFGA, 10 objects, a model without intersection) to 324 ms (OpenFGA's full model, 100,000 objects), 22 to 29 ms for SpiceDB once its limit binds, 1.2 ms for Cerbos, and 1.7 to 13.6 ms for OPA's filter documents and 116 ms for its partial evaluation over 1,500 documents. A check took 0.13 to 1.4 ms per hit on those worlds. Caching paid as predicted for OpenFGA, SpiceDB and OPA, at least 17.5 times faster in every pre-registered cell. Cerbos's check is so cheap that caching sped it up only 8.6 times (95% interval 8.5 to 8.7): short of the predicted factor of 10, but above the falsification bound of 5, so inconclusive.

G.6 Revocation and faults in detail

Revocation. Table 2 gives the measured δ of each serving path (Definition 1), and a dagger marks a violation of the δ the path states. The cache's time to live is the realized delay, not merely a bound: every ttl median in Table 2 lies within 34 ms of 5 s or 30 s, and invalidation ends the delay at once, or within the shared replica's 1 s local copy. Permissions copied into chunk metadata took effect within milliseconds of the write to the store, whatever the cache (the delay of copying them

from a source system was not measured): the store filters current metadata, and cached decisions are keyed by it. Two sources of truth were stale themselves. Cerbos reloaded a rewritten policy file in a median 2.5 s (at most 4.0 s). SpiceDB under `minimizeLatency`, WARDOWL's default, took a median 3.3 s, with 95% of trials within 4.9 s, and in 28 polls a revoked chunk reappeared after it had disappeared. One of its 40 trials took 8.7 s, beyond the 6.5 s we had registered as the limit: the 5.5 s that its quantization window [9, 14] implies, plus the 1 s allowed for polling. That single trial falsified the prediction, and the rerun below did not reproduce it: the same cell's 40 trials took at most 5.2 s, within the 5.5 s. Invalidation does not survive such an engine: the next call refills the cache from the stale snapshot, which the cache holds for its full ttl, so 79 of 80 SpiceDB revocations with invalidation exceeded their bound (medians 30.0 and 31.0 s), and Cerbos's took as long. After invalidation the bound is ttl plus staleness, not staleness; a cache that declines to refill during the engine's staleness window would restore it, which we evaluate below.

Faults. We injected 14 faults into each engine, by a proxy or by killing the server, and five or eight into pgvector and Qdrant, in 1,090 episodes of a 3 s baseline, a 12 s fault and recovery. Outside the control below, no retrieval returned a chunk outside the user's readable set, in 623,279 requests over 1,050 episodes. Requests within an episode share its fault, so we state the rate per episode, 2.8×10^{-3} (one-sided 95%), not per request. These episodes are not samples from a population of faults: ten per (target, fault, condition) replay 13 fault kinds, each episode cycling 10 fixed (user, query) pairs, and over the 13 kinds the formula gives 0.21. The control, 40 episodes that turned the engine's denials into allows, is the fault class that does leak: it leaked in every engine, and it is outside the bound. Every engine-caused failure was the layer's authorization error with a message naming no host. Transport faults (server down, connections refused or never answered, delays past the 2 s timeout, HTTP 500 and 503) failed every request inside the fault. Recovery took a median 0.02 to 0.18 s after the proxy's faults were removed, and 0.08 to 0.47 s after a killed server was started again (OPA, Cerbos, OpenFGA, SpiceDB), a case the pre-registration left exploratory. A circuit breaker failed requests in at most about 0.1 ms instead of 2 s, at the price of a median 3.1 to 3.6 s to the first success after recovery; a warm 30 s cache answered every request through the 12 s outages. Answers altered in content failed closed, except that an empty LookupResources stream reads as "no readable documents": 4,746 retrievals returned nothing without an error, losing recall only. Store faults surfaced as the clients' own exceptions, unwrapped, some with the store's raw response, and pgvector's pool held blocked requests for its 5 s timeout or longer, so its unanswered-connection case put no request inside the window.

The fixes, tested. The run exposed five defects in the layer, and we fixed all five. After an invalidation the cache now caches nothing for the engine's declared staleness, and the SpiceDB adapter, whose documentation had promised 5 s, declares 11 s without full consistency: twice the documented 5.5 s [14], since one trial had measured 8.7 s, which stays unexplained. An empty LookupResources stream now yields no filter, so the retriever post-filters and checks every candidate. Every store failure is raised as one search error with a fixed message and the client's exception as its cause. And pgvector can set a statement timeout. A targeted rerun, an implementation check specified in writing after the main results and before running, with requests started every 0.2 s so that some are in flight when a fault begins, confirmed all of its predictions. With invalidation, all 160 SpiceDB and Cerbos revocations took effect within the predicted bound (the declared staleness plus 1 s, and 1 s more on the replica): medians of 4.8 s (at most 5.2 s) for SpiceDB and 3.9 s for Cerbos, against 30 and 31 s before. Without a cache, SpiceDB took a median 4.7 s and at most 5.2 s, so the first run's 8.7 s did not recur. SpiceDB itself still flapped, serving a revoked chunk again after it had gone in 16 to 28 polls per condition; the cache no longer prolongs that. Under the emptied stream, 3,520 retrievals failed closed instead of returning nothing, and with only the

Store, under this layer's schema	Four runs			Overlap@5, first run		p50 (ms)
	Recall@5	Recall@10	Range@10	judy	others	
FAISS (flat, exact)	1.000	1.000	0.000	1.000	1.000	343–368
Redis	0.984	0.985	0.017	0.998	0.982	30
Elasticsearch	0.945	0.952	0.021	0.987	0.941	8–9
OpenSearch	0.942	0.936	0.005	0.998	0.941	7
Weaviate	0.900	0.903	0.016	1.000	0.876	7–8
Chroma	0.875	0.878	0.007	0.987	0.876	1432–1481
LanceDB	0.845	0.884	0.034	0.949	0.821	24–25
pgvector, fast mode (2 runs)	0.701	0.729	0.006	0.724	0.686	7.8–8.1
pgvector, exact mode (2 runs)	0.993	0.993	0.002			386–399
Qdrant	0.550	0.544	0.030	1.000	0.523	4.5–4.7

Table 9: Retrieval quality in the enterprise scenario (Section 9), four runs of 1,105 questions (seeds 7, 11, 13, 17). Recall is the mean over runs; Range@10 is the largest minus the smallest recall@10; p50 is the range of per-run medians. pgvector ran in one of two modes per run (Section 6.3); each row averages the two runs in that mode. Overlap@5 is the share of the exact top five that a store returned, by person type, from a re-analysis of the first run (a fast-mode run for pgvector); "judy" are the least-privileged people (624 readable chunks). pgvector with OPA follows pgvector's mode in each run. Settings: Appendix P.

lookups emptied 3,180 returned exactly the right answer through post-filtering. All 6,066 store failures carried the fixed message and named no host. pgvector's statement timeout turned a table lock into failures after a median 1.003 s with no request hanging, against 80 hanging requests and a median of 5 s without it; for a connection that is accepted but never answered it cannot help, since the server never sees the statement, and every request hung or failed after 5 s with or without it, as predicted. Only a deadline on the whole retrieval, which the layer does not yet offer, would bound that case. In the 10,303 requests of the 20 SpiceDB episodes under these faults no chunk leaked, and the 120 store episodes reported no leak either; over the rerun's 140 episodes the formula gives 2.1×10^{-2} per episode (one-sided 95%).

H The audit and its diagnosis in detail (X0 and X1)

This appendix holds the full audit, the full diagnosis of Qdrant and pgvector, and the design of the diagnosis grid, which Section 6 summarizes. Appendix B lists the hypotheses.

H.1 The audit

Table 9 shows the spread that Section 6.1 summarizes, at the settings of Appendix P. The rows compare configurations, not stores, and the corpus, in which only one of twenty near-identical tenant copies of a document passes a person's filter, is a hard case for graph indexes. Every store's loss is concentrated where the readable set is far from the question. pgvector's empty answers in these runs, seven in the seed-17 run and one in the seed-11 run, were all on red-team prompts. In the seed-11 and seed-13 runs the same configuration searched exactly, at fifty times the latency, and the seed-11 run still gave one person one empty answer, as did another exact run of Section 9; we have not explained these two. On targeted questions in the first execution of the seed-7 run, 59.6% of Qdrant's top five were shared FiQA passages where exact search returned none, and in the execution the artifact holds the model found the target document for only 34.0% of targeted questions (33.8% to 39.6% over the four runs).

Layout	Search setting	Recall@10	Range	p50 (ms)
As run (one collection)	`hnsw_ef` 128 (as run)	0.656	0.086	7.1
	`hnsw_ef` 64	0.600	0.074	6.0
	`hnsw_ef` 512	0.758	0.120	11.3
	ACORN, max selectivity 0.4 or 1.0	0.808	0.127	7.3
	exact	1.000	n/a	43.1
Tenant field marked `is_tenant`	as run	0.651	0.025	6.9
	ACORN	0.802	0.076	7.1
Built as one segment	as run	0.625	0.029	7.0
	ACORN	0.777	0.027	7.2
Per-tenant graphs (`is_tenant`, $m = 0$)	as run	0.952	0.001	6.6
Shared passages in a separate collection	as run	0.949	0.003	12.9

Table 10: Qdrant 1.19.2 under the enterprise scenario's permission filters (X1 run 2, 1,489 questions, three index builds per layout, means over builds; Range is the range over builds; latency from one CPU session). ACORN at the default gate (0.4) and at 1.0 gave identical results. Run 1 agreed within 0.05 (as run 0.619, ACORN 0.761, per-tenant graphs 0.943).

H.2 Qdrant

Qdrant plans filtered search per segment from estimated filter cardinality [127]: a filter estimated below a threshold (at the default 10,000 KB and 768 dimensions about 3,333 points, by our arithmetic) runs as a plain, exact scan over the payload index, and a broader one runs as filtered HNSW traversal, which in the source at v1.19.2 scores and expands only neighbors that pass the filter. Extra graph edges that keep a filtered subgraph connected are built per payload value, and only for values held by fewer points than a cap of the point count divided by the average links per node, times four, about 19% of the collection in the cited article's example [34]. In the scenario every tenant value (56% of points) and the employee and contractor groups (65% and 61%) exceed that.

The pre-registered predictions for the hypothesis of Section 6.2 all held, in both runs of the grid (Table 10 shows the second). They test recall by question category and by intervention; no connectivity or reachability was measured. In the first run, recall@10 for company documents among everyone but the least-privileged was 0.273, against 0.369 in the second. Per-tenant graphs were faster than the as-run layout (6.6 against 7.1 ms). Raising `hnsw_ef` from 128 to 512 stayed within the band we had pre-registered as inconclusive and did not close the gap, as a traversal trapped in the wrong region would predict. ACORN's 0.808 has a 95% interval of 0.785 to 0.830 over people, while its range over the three builds is 0.127: read such intervals as conditional on the build.

One prediction failed. We expected ACORN's default gate, which engages when the estimated selectivity is at most 0.4, to engage for some people and not others, because the planner's independence estimate for a typical employee sits near 0.4. It engaged for everyone, and the gate at 1.0 gave identical recall. So at `hnsw_ef` 128 ACORN helps without reaching the layout change, and its gate does not misfire; with more effort it matches the change at lower throughput (Appendix I). A third hypothesis, that recall varies with the segment layout, remains inconclusive: at the as-run setting recall@10 varied by up to 0.086 between three builds of the same data (by up to 0.127 with ACORN), and the layout built as one segment scored 0.030 lower (0.018 to 0.042), but in the end the as-run collection also held its points in a single segment, so the difference is one between builds, not layouts.

H.3 pgvector

pgvector 0.8 answers a filtered HNSW query by scanning the graph in order of distance and applying the filter to each row the scan emits; with iterative scans enabled, it continues past `hnsw.ef_search` until enough rows pass the filter or a limit is reached [121]. The limit is `hnsw.max_scan_tuples` (20,000 by default) or a memory bound of `hnsw.scan_mem_multiplier` times `work_mem`. A person whose readable set is small and far from the question can exhaust the budget before any readable row is emitted, and the query then returns nothing, which is indistinguishable from having nothing to read. That was our hypothesis for the empty answers of the audit. Testing it showed that a decision taken before the scan matters more: whether PostgreSQL runs the HNSW scan at all.

The plan cache. The layer binds the filter's values and k as parameters, so the search statement's text repeats for every person with the same filter shape. psycopg prepares a statement on its sixth execution on a connection [159]; PostgreSQL then plans five executions with their values and afterwards uses a generic plan whenever that plan's estimated cost is not much higher than the custom plans' average [156], the trade-off studied as parametric query optimization [67]. A generic plan cannot see the `jsonb` values or the `LIMIT`, assumes a selective filter, and reads the rows the GIN index returns and sorts them: an exact search. The first diagnosis run recorded no executed plans, and most of its searches ran that one. The second recorded the plan of every search (X1 run 2). Hypothesis P4 is confirmatory, pre-registered before the first run; P5 is exploratory, specified in writing after the first run's results and before the second ran. Under the default configuration, after a vacuum, only 0.9% of the searches on the scenario's questions used the HNSW index, and recall@5 was 0.996 at a median of 362 ms; planned with their own values, 98.1% used the HNSW index, with recall@5 of 0.680 at 22 ms, against 0.691 in the first execution of the audit's seed-7 run and 0.703 in the execution the artifact holds (P5 confirmed; P4, which had predicted that generic plans make no difference, falsified, with recall@10 lower by 0.343 under custom plans).

One replay pair, on one build and in a fixed order, points to the state of the GIN index. Replaying the audit's pass right after loading, when the index's pending list held 388 pages, PostgreSQL never switched: 98.1% of searches used HNSW, and recall@5 was 0.697 at 7.0 ms, the audit reproduced. The same replay after `VACUUM (ANALYZE)`, with the pending list empty, ran 2,148 generic executions, and recall@5 was 0.998 at 335 ms. That command changes more than the pending list, so the pair does not isolate it. PostgreSQL's GIN cost estimate charges every pending page as a page read at the start of the scan [158], which is consistent with a generic GIN plan that looks expensive after a bulk load until a vacuum merges the list, and with a generic plan that, once it has won, keeps winning on that connection. Autovacuum runs on its own schedule, and the variation across the end-to-end runs would fit it, but we did not record its timing there and cannot say what decided their modes: the same configuration ran fast by its median in three of eight runs, one of them a mixture of fast and exact searches, and exact in five (Section 9). A measurement of pgvector through prepared statements can therefore report one of two systems, 16 to 64 times apart in latency (16 on the grid; across the eight end-to-end runs, 47 to 64 between exact and fast runs, and 23 against the one mixture run) and about 0.3 apart in recall, with no benchmark setting naming which. The harness of Section 6.5 plans every search with its own values, and so can a deployment, with `plan_cache_mode = force_custom_plan` or by not preparing the search statement.

The scan's budget. With the HNSW scan forced, the audit's loss came back on X1's full question set, which adds 24 people in a single group each: recall@10 of 0.656, and 8.9% of retrievals with $k = 5$ empty, the scan emitting a median of 20,181 rows on an empty answer. The budget hypothesis (P1) was inconclusive as stated, and it needed a correction. Both limits bind: raising the tuple limit alone, to 100,000 or a million, lowered the empty rate only to 7.3%, because the 4 MB memory

bound then stopped the scan; raising the memory alone changed nothing; raising both removed every empty answer. The first run's report that the tuple limit alone sufficed came from searches that had in fact run the exact plan. And 79.9% of the empty answers came from questions whose first readable row lay beyond half the scan's reach, short of the 95% we had predicted.

Removing the empty answers did not remove the loss. With both limits raised, recall@10 was 0.780, and 0.866 on a graph built with $m = 32$ and `ef_construction` 200. The rest is mis-retrieval: a relaxed-order scan stops at the first k permitted rows it meets. The hypothesis that this loss comes from shallow, relaxed scans (P2) was inconclusive. A deeper search (`ef_search` 1,000: +0.069), more candidates before the final sort (ten times k: +0.095) and a better graph (+0.066) each helped, as predicted, and strict ordering hurt (0.497), but the loss among questions with under a tenth of their 100 nearest chunks readable, 0.368, was not the predicted twice the 0.188 elsewhere. The planner also misjudged the people it served worst: for the least-privileged, its row estimate was a median 11.1 times their readable set (P3, inconclusive in the first run and confirmed in the second, the run that judges it), which would steer it to the HNSW scan where an exact plan over a readable set of a few hundred chunks would return the right answer.

H.4 The diagnosis grid

pgvector. The numbers in the paper are from run 2: three index builds with HNSW $m = 16$ and `ef_construction` 64 and three with $m = 32$ and `ef_construction` 200 (a core grid), 1,489 questions (the scenario's 1,105 and 384 asked by 24 people in a single group each), three timed passes each. Query-time settings: the scenario's settings with the planner's choice and prepared statements (*as run*); custom plans forced (`plan_cache_mode = force_custom_plan`); no prepared statements; the HNSW scan forced (custom plans with sequential and bitmap scans disabled), the reference for the scan-budget hypotheses, and every budget and depth setting forced the same way: `hnsw.max_scan_tuples` of 20,000, 100,000 or 1,000,000 crossed with `hnsw.scan_mem_multiplier` of 1, 8 or 16; `work_mem` 64 MB; `hnsw.ef_search` 40, 200, 400 and 1,000; an inner `LIMIT` of four or ten times k; strict ordering; and exact search with index scans disabled. Two replays of the audit's pass ran first on the first build, right after loading and after `VACUUM (ANALYZE)`; every other setting ran after a vacuum. PostgreSQL's `auto_explain` recorded the plan of every search of the first pass. Run 1 built one pgvector index, skipped 12 settings for time, and recorded no executed plans; it is reported in the released evidence.

Qdrant. Five layouts (as run; tenant field marked `is_tenant`; per-tenant graphs with `payload_m` 16 and $m = 0$; shared passages in a separate collection; one segment), three builds each, with `hnsw_ef` 64, 128 and 512, ACORN with maximum selectivity 0.4 and 1.0, and exact search. Search paths per segment were read from Qdrant's telemetry.

Statistics. 95% intervals from 10,000 cluster-bootstrap resamples over people (seed 20261026); paired differences between settings on the same questions; verdicts computed by script from the pre-registered criteria.

I The frontier experiment in detail (X2)

This appendix gives the design and full results of the frontier experiment, which Section 6.5 summarizes. Its verdicts are as the analysis script printed them (Appendix B).

The enterprise scenario has one permission layout. To separate the filter's selectivity from its correlation with the question, X2 adds a corpus whose layouts we control: the first million passages of MS MARCO [19] from BEIR, embedded with bge-small-en-v1.5 (384 dimensions, so that ten

stores fit beside the harness on a 16 GB runner). Every passage belongs to one team of a hierarchy of 100 divisions, 10 departments each and 10 teams each, assigned either at random (*uncorrelated*) or by hierarchical k-means over the embeddings (*clustered*, so groups are topical), five seeds of each. A subject receives divisions, departments and teams until it may read a target share of the corpus, from 0.06% to 100%, and its filter is either one disjunction over its groups or a tenant clause conjoined with a disjunction over a grant twice as large, a shape in which each clause alone passes far more than both. Questions come from a pool of 20,000 MS MARCO questions: four drawn at random for a subject of an uncorrelated layout; for a subject of a clustered layout, four from those whose 100 nearest passages are at least half readable (*positively correlated*) and four from those with nothing readable among their 1,000 nearest (*negatively correlated*), the best by the criterion when fewer than four qualify (684 of the 6,200 pairs). Ten subjects per seed and cell make 1,050 subjects. For every question and subject we compute, from exact scores, the global selectivity, the local selectivity among the 100 and 1,000 nearest passages, the rank of the first readable passage, ACORN's query correlation [120], the global-local selectivity of Amanbayev et al. [2] and the over-fetch estimate of HCBGen [81].

Ten approximate stores run against FAISS over a flat index: pgvector, Qdrant, Elasticsearch, OpenSearch, Weaviate, Redis, Milvus, MongoDB, LanceDB and Chroma, each at its default, at five levels of its search-effort parameter, with its store-specific options (pgvector's raised scan limits and strict order, Qdrant's ACORN and per-tenant graphs, Weaviate's filter strategy and flat-search cutoff, Redis's hybrid policies, LanceDB's refinement and post-filtering, and on the enterprise corpus Elasticsearch's `int8_hnsw` index beside its default `bbq_hnsw`) and with its own exact search where it has one (not Milvus or Chroma), in three fresh builds per store; exact searches, which do not use the index, run on the first build, and FAISS on one. On MS MARCO the per-tenant graphs are queried by the subjects with a tenant clause. Each pgvector search is planned with its own values, so the plan-cache effect of Section 6.3 cannot arise, and the share of searches whose plan uses the HNSW index is recorded. Throughput is measured end to end through the layer with one client thread and compared only within a job; recall is compared across jobs. Each setting has a share of its job's time; one that runs out keeps a prefix of a fixed order that cycles through cells and subjects. The same harness runs the enterprise scenario, whose as-run settings must reproduce the audit's recall within 0.03 for seven of nine stores. We pre-registered four hypotheses: that negatively correlated questions lose at least 0.05 of recall@10 against uncorrelated ones at defaults in at least six stores (H2.1); that local selectivity predicts recall better than global selectivity (H2.2); that raising search effort narrows but does not close that gap (H2.3); and that intermediate selectivities are harder than very small ones, which stores answer by brute force (H2.4).

What ran. X2 ran on GitHub Actions hosted runners (four CPUs, 16 GB), one store per job: PostgreSQL 17.11 with pgvector 0.8.7 (the audit ran PostgreSQL 16.15), Qdrant 1.19.2, Elasticsearch 9.5.5, OpenSearch 3.9.0, Weaviate 1.39.12, Redis 8.10.2, Milvus (image 3.0.2), MongoDB Atlas Local 8.0.30, LanceDB 0.40.0 and Chroma 1.5.9. The validity checks held. Every exact setting, and FAISS, returned the exact answer to every question it ran (recall@10 1.000); no hit was unreadable and no canary appeared; and every subject granted 0.5% or more could read within 25% of its target share. Eight of the nine stores of the audit reproduced the recall@10 of the first execution of its seed-7 run, the values recorded beside the X2 verdicts in `evidence/x2-x5-run1.md` (not the four-run means of Table 9), within 0.03 at its settings, pgvector among them at 0.730 against 0.732, with every search planned with its own values; against the execution the artifact holds (Appendix M.1) seven of the nine are within 0.03, Qdrant's 0.576 against 0.530 being the other exception, which still meets the pre-registered rule of at least seven. Chroma reached 0.990 against 0.883 (0.878 in the artifact's execution): at about 1 s a retrieval its share of the job's time covered only 381 to 407 of the 1,468

	Default			Most search effort			Best other option			Exact	Ent.
Store	R@10	100%	QPS	Setting	R@10	QPS	Option	R@10	QPS	QPS	R@10
pgvector	0.575	0.916	75.3	`ef_search` 1,000	0.927	14.8	both scan limits	0.949	12.9	0.26	0.112
Qdrant	0.799	0.978	205.3	`hnsw_ef` 1,024	0.898	96.7	ACORN 1.0	0.997	28.5	22.6	0.641
Elasticsearch	0.806	0.870	185.1	candidates 10,000	0.878	107.2				52.1	0.815
OpenSearch	0.998	0.971	105.3	`ef_search` 512	0.999	91.5				15.8	0.948
Weaviate	0.968	0.966	136.1	`ef` 1,024	0.994	77.1	sweeping	0.995	104.3	24.8	0.923
Redis	0.971	0.793	54.1	`EF_RUNTIME` 1,000	0.999	49.9				17.0	0.987
Milvus	0.972	0.936	68.6	`ef` 512	0.999	98.1					0.864
MongoDB	0.996	0.960	24.1	candidates 10,000	1.000	10.3				18.7	0.953
LanceDB	0.437	0.517	67.9	`nprobes` 500	0.450	24.1	refine 30	0.908	37.9	4.6	0.439
Chroma	not measured: 137 to 221 s per retrieval										0.983
FAISS (flat)										0.91	1.000

Table 11: X2 on MS MARCO (1,000,000 passages; 6,200 question–subject pairs over every layout, correlation regime and selectivity). R@10: recall@10, the mean over three builds (exact settings: one). 100%: recall@10 at the default on the uncorrelated subjects granted the whole corpus, whose filter lists every division and so passes every passage (200 pairs per build, the mean over three builds), the nearest the data come to an unfiltered baseline. QPS: one client thread through the layer, the median of three passes in the store's build-0 job, so comparable only within a row. Most search effort: the top of the store's five-level sweep. pgvector's default has iterative scans off; "both scan limits" raises `hnsw.max_scan_tuples` to 1,000,000 and `hnsw.scan_mem_multiplier` to 16 at `ef_search` 100; its exact search answered 998 pairs in its time. Qdrant's ACORN option has `max_selectivity` 1.0 and `hnsw_ef` 512. Ent.: recall@10 at the default on the enterprise corpus (1,468 questions; Elasticsearch's `bbq_hnsw`, 0.888 with `int8_hnsw`; Chroma on 407).

questions, the first questions of each person, which for most people ask about a document they may read, the kind it answered best in the audit (overlap@5 of 0.989, against 0.82 to 0.84 for the other kinds), a reading consistent with the gap though the run did not print recall by kind. On MS MARCO Chroma was not measured, a deviation from the plan. Each of its three builds loaded the million passages in 22 to 28 minutes, but a filtered retrieval then took 137 to 221 s, so the twenty warm-up retrievals of each setting used up the setting's time and none reached a timed pass; under the pre-registered procedure a rerun cannot finish within a job.

Defaults. At the settings a user gets without tuning, recall@10 on MS MARCO ranged from 0.437 and 0.575 to 0.998 (Table 11). The two lowest are documented defaults. pgvector without iterative scans filters the 40 candidates its index returns and stops: 32.4% of the retrievals on MS MARCO and 61.6% on the enterprise corpus came back empty (recall@10 0.575 and 0.112), against 0.3% and 9.7% with the iterative scans the layer configures (recall@10 0.897 and 0.661 at `ef_search` 100). LanceDB's default IVF-PQ index ranks by quantized distances: raising `nprobes` from 10 to 500 moved recall@10 only from 0.424 to 0.450 at a third of the throughput, while re-ranking 30 times k candidates by their full vectors reached 0.908. Elsewhere effort bought recall at a price in throughput, and the stores' own options bought more: Qdrant reached 0.898 at the top of its sweep but 0.997 with ACORN, and Weaviate's sweeping filter strategy reached 0.995 against 0.968 for its default ACORN strategy [172], at 104.3 against 136.1 queries per second. Index builds varied little except Qdrant's: its recall@10 at the default ranged from 0.759 to 0.878 over three builds of the same data, as in X1, wider than the intervals below, which resample subjects and are conditional on the builds.

To separate the filter's cost from the index's own error, Table 11 also gives recall@10 at 100% selectivity, where the filter passes everything. At the defaults that baseline ran from 0.517 for LanceDB, the floor of its quantized index, and 0.793 for Redis, at an `EF_RUNTIME` of 10, to 0.978 for Qdrant. Four stores lost recall against it over the whole mix of filters: pgvector (0.575 against 0.916),

Store	G (H2.1)	D (H2.2)		V (H2.4)	
pgvector	0.4242 (0.4154, 0.4332) gap	0.3650 (0.3321, 0.3989)	local	0.9267 (0.8867, 0.9600)	harder
Qdrant	0.0343 (0.0196, 0.0487)	−0.2032 (−0.2194, −0.1876)	global	0.5403 (0.5165, 0.5653)	harder
Elasticsearch	0.1371 (0.1242, 0.1500) gap	0.2439 (0.2116, 0.2754)	local	0.2338 (0.2032, 0.2650)	harder
OpenSearch	−0.0014 (−0.0029, 0.0001) none	−0.0601 (−0.0793, −0.0411)	n.a.	0.0005 (0.0000, 0.0015)	
Weaviate	0.0549 (0.0478, 0.0618) gap	−0.3900 (−0.4159, −0.3644)	global	0.1780 (0.1578, 0.1980)	harder
Redis	−0.0794 (−0.0871, −0.0719)	−0.0757 (−0.0870, −0.0647)	global	0.0000 (0.0000, 0.0000)	not harder
Milvus	0.0136 (0.0080, 0.0194) none	−0.5307 (−0.5469, −0.5140)	global	0.1037 (0.0925, 0.1150)	harder
MongoDB	−0.0037 (−0.0055, −0.0019) none	−0.0804 (−0.1016, −0.0593)	global	0.0012 (0.0005, 0.0020)	
LanceDB	0.2535 (0.2423, 0.2649) gap	0.5382 (0.5080, 0.5674)	local	−0.0098 (−0.0332, 0.0212)	not harder

Table 12: X2's pre-registered statistics on MS MARCO at the stores' defaults, with 95% intervals from a cluster bootstrap over subjects (10,000 resamples). G: recall@10 of uncorrelated minus negatively correlated questions at selectivities 2% to 30% ("gap" needs $G \geq 0.05$, an interval above 0 and the same sign in at least four of five seeds; "none" $|G| < 0.02$ with the interval inside ± 0.03). D: $|\rho_S(\text{recall@10}, \rho_{1000})| - |\rho_S(\text{recall@10}, \sigma_g)|$, Spearman over the 6,200 pairs ("local" $D \geq 0.05$ with the interval above 0, "global" $D < 0$ with it below; n.a.: fewer than 100 pairs with recall below 1). V: recall@10 at 0.06% minus the lowest at 0.5%, 2% and 10%, clustered layouts, negatively correlated questions, one disjunction ("harder", intermediate filters harder than the smallest, needs $V \geq 0.05$, an interval above 0 and four seeds; "not harder" $V \leq 0$; the script printed these as "U-shaped" and "monotone", but V does not test recovery at larger selectivities). A blank label is neither outcome. Chroma was not measured.

Qdrant (0.799 against 0.978), Elasticsearch (0.806 against 0.870) and LanceDB (0.437 against 0.517). The other five scored no lower than at 100%, consistent with many of the mix's filters being small enough for the store to answer by brute force; what they lost at 100% is the index's error, not the filter's. The enterprise corpus has no such cell, because no person may read all of it.

Read at a common throughput, the same sweeps need a caveat: each store's queries per second come from its own job on its own runner, so they are not calibrated across stores. Among the settings that served at least 50 queries per second in their build-0 job, exact search included, the best recall@10 of that build was 1.000 for Elasticsearch (its exact search, at 52; 0.878 for its approximate search at 107), 0.999 for OpenSearch and Milvus, 0.997 for Redis, 0.995 for Weaviate, 0.979 for Qdrant (ACORN at its default gate, at 69), 0.578 for pgvector (its default without iterative scans; with them it served at most 16) and 0.438 for LanceDB; MongoDB reached 50 at no setting (at most 28). The threshold of 50 is this reading's and not the pre-registration's. A calibrated comparison would need every store timed on one machine, or a reference workload on each runner, and interpolation along each sweep.

Correlation (H2.1, inconclusive). Negatively correlated questions lost at least 0.05 of recall@10 in four stores, not the six we had predicted, and no loss beyond 0.02 in three (Table 12): pgvector lost 0.424, LanceDB 0.254, Elasticsearch 0.137 and Weaviate 0.055, each in all five seeds; OpenSearch, MongoDB and Milvus stayed within 0.014 at recall@10 of 0.97 to 1.00. The other two show why one number per store misleads. Redis's recall was *higher* for negatively correlated questions, by 0.079, and the cost appeared as latency: in the build-0 job those questions at 10% and 30% selectivity were answered exactly at a median of 56.8 and 147.1 ms, against 0.908 and 0.858 of recall at 2.2 and 2.7 ms for uncorrelated ones, consistent with Redis switching from batches of graph neighbors to brute force over the filter's matches when the batches find too few [132]. OpenSearch and MongoDB paid the same way, more mildly (24.6 against 15.3 ms, and 68.0 against 33.3 ms, at 10%). Qdrant's gap of 0.034 (0.020 to 0.049) averages opposite effects: at 2% its recall@10 was 0.024 for uncorrelated questions, 0.312 for negatively and 0.808 for positively correlated ones, and at 30% 0.818, 0.565 and 0.985 (build 0). This is consistent with a random 2% of the graph being the hardest permitted subgraph to traverse, since few of a node's neighbors would belong to it and topical groups would

stay better connected; we did not measure connectivity.

Local or global selectivity (H2.2, falsified). Local selectivity predicted recall better than global selectivity in three stores and worse in five; the pre-registered rule calls the hypothesis falsified once half of the assessable stores (eight; OpenSearch had too few misses) go the other way. The split is consistent with the mechanism. Where recall depends on what lies near the question, in pgvector's scan in order of distance, LanceDB's probed partitions and Elasticsearch, local selectivity won (D of 0.244 to 0.538), and ACORN's query correlation correlated with recall more strongly still (Spearman 0.562 to 0.672). Where the store chooses its search path from the filter's estimated size, as Qdrant, Weaviate and Redis document [127, 132, 172], and in Milvus and MongoDB, small filters are searched exactly and global selectivity won (D of -0.531 to -0.076). HCBGen's over-fetch estimate, which grows as the readable set shrinks, correlated *positively* with recall in eight of the nine stores (up to 0.551), the opposite sign to a measure of difficulty, for the same reason. No single axis describes every store; the paper therefore reports recall against several (released with the run).

Effort and the gap (H2.3, confirmed). At the top of each sweep the gap narrowed and remained in two of the four stores that had one, which meets the rule's bar of half. pgvector's fell from 0.424 to 0.172 (0.161 to 0.183) at `ef_search` 1,000, and LanceDB's moved by 0.001 at 500 probes, at its quantization floor, a negligible narrowing that still counts under the rule. Weaviate's closed, to 0.004 at `ef` 1,024. Elasticsearch's widened, from 0.137 to 0.162 at 10,000 candidates: more candidates helped uncorrelated questions more. pgvector reached recall@10 of 0.95 for uncorrelated questions at `ef_search` 400 and for negatively correlated ones at no level of its sweep; Weaviate needed one more level for them (`ef` 256 against 128).

Intermediate filters harder than the smallest (H2.4, confirmed). Five stores lost more at an intermediate selectivity than at the smallest, as predicted; the test does not ask whether recall recovers at larger selectivities. On clustered layouts with negatively correlated questions and one disjunction, recall@10 at 0.06%, 0.5%, 2% and 10% was 0.927, 0.380, 0.000 and 0.000 for pgvector at its default (whose planner chose the exact plan for most of the smallest filters, below), 1.000, 1.000, 0.460 and 0.489 for Qdrant, 0.841, 0.727, 0.641 and 0.607 for Elasticsearch, and 1.000 down to 0.822 and 0.896 at 10% for Weaviate and Milvus. Redis stayed at 1.000 and LanceDB near its floor (0.197 to 0.235); OpenSearch and MongoDB lost at most 0.0012. In this regime very selective permission filters were therefore the easy case for five of the nine stores; the test does not tell us where recall is lowest.

pgvector's plans. With each search planned with its own values, PostgreSQL chose the HNSW scan for 98.6% of the enterprise searches at `ef_search` 40 to 200, as in X1's custom plans, but for 88.9% at 400 and none at 1,000, where its cost estimate made the exact plan cheaper: recall@10 1.000 at a median of 169.7 ms, against 0.670 at 17.9 ms with `ef_search` 200. On MS MARCO, at the default setting, it chose the exact plan for 86% to 94% of the searches at 0.06% selectivity, for up to 39% at 0.5%, and for none above. These switches are the planner's, made per search; the plan cache of Section 6.3, which decides once per statement and connection, had no role here.

The enterprise corpus. The enterprise results agree with X1 and the audit. Qdrant reached 0.663 at `hnsw_ef` 128, 0.864 at 1,024, 0.830 with ACORN at its default gate and 0.953 with ACORN at gate 1.0 and `hnsw_ef` 512, and 0.941 with per-tenant graphs at `hnsw_ef` 128 at a similar latency (4.7 against 4.9 ms). Elasticsearch's default index for 768 dimensions, `bbq_hnsw`, lost against `int8_hnsw` at every level (0.815 against 0.888 at the default, 0.867 against 0.966 at 500 candidates). Turning off Weaviate's flat-search cutoff lowered its recall@10 from 0.923 to 0.837 here and from 0.968 to 0.812 on MS MARCO: the exact scan of filters below 40,000 objects carries much of its recall.

J Filter size in detail (X3)

This appendix reports the filter-size experiment, which Section 6.5 summarizes in a paragraph.

A person in many directory groups turns the permission filter into a long disjunction: the filter of Appendix D.1 holds one value per group, and enterprise users can belong to thousands. We vary only that length. Each of five seeds builds a corpus of 100,000 vectors of 384 dimensions, the 57,638 FiQA passages embedded as in Appendix L.1 and 42,362 synthetic vectors around 200 seeded cluster centers, and makes a fixed 10% of it readable at every level: a readable document holds one group of each level, and the subject at level G holds exactly that level's G groups, for G from 1 to 40,000. The readable set, and so the exact answer, is the same at every level; only the filter grows. Twelve stores run at default index settings, with FAISS over a flat index and the in-memory backend as controls, and each level is timed over 30 questions (10 when the level's first retrieval takes more than 2 s) with $k = 10$ in three passes after a warm-up, a retrieval being allowed 120 s. We record whether each store's translator returns and how fast, whether the store accepts the filter or refuses it, and with which error, the latency percentiles, and recall@10 against exact search over the readable set; for pgvector also the plan. The limits the stores document that could bind are a 65,536-term cap per `terms` query in Elasticsearch and OpenSearch, PostgreSQL's 65,535 bind parameters, SQLite's 32,766 host parameters and expression depth of 1,000 under Chroma, and the request sizes of MongoDB (16 MB) and Qdrant (32 MB); the layer's Cassandra adapter falls back to one unfiltered query filtered locally beyond 254 groups, as documented.

We pre-registered that every translator returns within a second at every level; that at 40,000 groups Elasticsearch, OpenSearch, Qdrant, pgvector, LanceDB, Cassandra, FAISS and the in-memory backend accept the filter and Chroma refuses it; that pgvector's median latency grows at least fiftyfold from one group to 40,000, because above 100 values the layer sends an array test that the GIN index cannot serve, while the controls' grows at most twofold; and that recall does not depend on the filter's length in any approximate store that accepts it (paired difference within ± 0.02).

Table 13 shows the result. Every translator returned at every level, the slowest in a median 197 ms (Cassandra's fallback with 40,000 groups) and 120 ms (pgvector), and every store but Chroma accepted every level in every seed without one failed retrieval, the five stores without a prediction included; no documented limit was reached. Acceptance with 40,000 groups was confirmed for each of the eight stores we predicted (X3-H2). In the main run Chroma completed only three of the five seeds the rules require: each of its seeds took 22 minutes, and the harness, which gave each store of a job an equal share of the job's time, skipped the last two although the job then finished about three hours early. We ran those two seeds afterwards, exactly as written, on the same data and in a job of their own, under an amendment specified in writing after the main results and before running; recomputed with them, every other store's tables came out unchanged. With five seeds, X3-H1 holds in all 84 of its parts.

Latency grew as predicted where we predicted it (X3-H3 confirmed in all three parts). pgvector's median rose 272-fold (95% interval 264 to 288) from one group to 40,000: by a factor of 1.6 up to 100 groups, then roughly in proportion to G. The plans show why, and that the reason we pre-registered did not decide it. At every level and in every seed the planner chose the same HNSW index scan, never the GIN index, not even for one group, and the scan removed the same rows through the filter at every level (55 to 142 for each seed's first question); only the cost of testing each row grew, so that question took 1.1 to 2.6 ms with one group and 239 to 566 ms with 40,000. Its answers did not change either: recall@10 was 0.967 at every level, and the paired difference between 40,000 groups and one was 0.000 (0.000 to 0.000). FAISS and the in-memory backend slowed 1.19 and 1.17 times, and Cassandra's 102 per-group queries with 100 groups took 1.85 times as long (1.82 to 1.87) as

	p50 (ms) with G groups						
Store	1	100	1,000	10,000	40,000	Ratio	Recall@10
pgvector	2.86	4.51	22.2	201	797	272	0.967
Redis	1.19	1.75	4.88	33.1	123	99.2	0.969
Vespa	3.03	4.08	9.61	60.3	225	76.1	0.887
Weaviate	4.79	5.74	14.3	82.7	306	64.1	1.000
Qdrant	3.49	4.35	6.30	27.3	101	27.5	0.999
Milvus	4.85	4.30	7.53	35.2	127	26.5	0.987–0.999
Elasticsearch	3.36	2.78	3.95	19.3	78.8	25.0	0.939
MongoDB	10.5	9.75	12.1	41.7	142	13.7	1.000
OpenSearch	6.06	6.27	7.63	20.2	71.6	11.5	1.000
LanceDB (exact)	53.6	57.0	66.3	149	424	7.94	1.000
Cassandra	109	166	90.7	124	308	2.78	0.996–1.000
FAISS (flat)	385	390	403	424	454	1.19	1.000
In-memory	432	439	451	475	503	1.17	n/a
Chroma	4,340	10,627	refused	refused	crashed	n/a	1.000

Table 13: Filter size (X3): median end-to-end latency of a retrieval through the layer by the number of groups G in the subject's filter, the median over three passes of 30 questions in each of five seeds (levels of 10 and 4,000 groups are in the released report). Ratio: median latency with 40,000 groups over that with one, paired by question. Recall@10 against exact search over the readable set is the same at every level unless a range (one group to 40,000) is given. Chroma's seeds 3 and 4 come from a follow-up run of the same harness on the same data, and it ran 10 questions per level because its first retrieval took more than 2 s; "refused": every retrieval failed with SQLite's expression-depth error; "crashed": the process ended with signal 11. Images and clients are those of the conformance campaign (Appendix E.2), at default index settings.

its single fallback query with 1,000. The other eight servers slowed 11 to 99 times, mostly beyond 1,000 groups, and with 40,000 groups the translation alone took 31 to 48 ms of medians between 72 and 306 ms: much of the cost of a long filter lies in building and shipping it. LanceDB, which scans exactly, slowed 7.94 times.

Recall moved by at most 0.012 with the filter's length in the ten approximate stores that accepted every level (X3-H4 confirmed for eight of the nine stores it names, inconclusive for Chroma; pgvector's paired difference was 0.000, and Cassandra's is below). The paired difference between 40,000 groups and one was 0.000, with an interval of 0.000 to 0.000, for Qdrant, Elasticsearch, OpenSearch, Weaviate, MongoDB and Vespa, 0.000 (−0.0007 to 0.0007) for Redis, and 0.012 (0.006 to 0.019) for Milvus, whose recall rose from 0.987 to 0.999 with the longer filter, within the pre-registered band; Cassandra's fallback, exploratory, gained 0.004 (0.0007 to 0.0093).

Chroma refused, as predicted, but in two ways. With 1,000 groups or more, every retrieval failed with SQLite's "Expression tree is too large (maximum depth 1000)", which Chroma raises as an `InternalError`: because the layer stores list values as boolean keys (Table 7), its disjunction holds one clause per group, 102 with 100 groups and 1,002 with 1,000, and SQLite limits an expression's depth to 1,000. With 40,000 groups the process died with signal 11, a segmentation fault, during its first retrieval, in all five seeds as in both smoke runs. The layer's translation had succeeded (2.66 million characters in 101 ms), no message was printed, and no error reached the layer: the application's process ends with the store's. A filter that kills the client process is a denial of service, not a refusal. The amendment's exploratory part, likewise specified in writing after the main results and before running, located the crash. Python's fault handler placed it in Chroma's call into its native core, and under gdb the main thread was about 16,500 frames deep in a single recursive function of Chroma's Rust bindings when its stack ran out, before SQLite saw the query.

One retrieval per level, each in a fresh process, put the threshold between 15,500 groups, refused with SQLite's error, and 16,500, which crashed, on runners with a 16 MiB main-thread stack limit (another limit would move it, which we did not test), consistent with one stack frame per clause of the disjunction. Chroma renders an n-ary disjunction as a left-deep chain of binary ORs, so the depth grows with the number of clauses; the layer now nests its disjunctions sixteen clauses to a node, which keeps the depth near 50 for 1,000 groups, and never sends a filter beyond conservative bounds on parameters, depth, nesting and size, filtering 1,000 candidates itself instead, as its Cassandra adapter already did; we have not evaluated what those 1,000 candidates cost in recall. In the CI of the pull request that made the change (`https://github.com/muzakkirhussain011/wardowl/pull/187`, merged on 2026-10-30 with its checks green, not yet in a release), the fixed layer retrieved exactly the permitted chunks from real Chroma for subjects in 1,001 and 40,000 groups. We reported both behaviors to Chroma on 2026-10-30, after that pull request was public (issues #7969 and #7970; Section 11). Chroma was also the slowest store at every level it accepted, 4.3 s with one group and 10.6 s with 100, and each refused retrieval still took about 4 s. With five seeds, its X3-H2 verdict is confirmed; X3-H4 stays inconclusive for Chroma, as the rule needs an accepted level of at least 1,000 groups.

K Repair in detail (X5)

This appendix gives the design and the full evaluation of the repair experiment, which Section 7 summarizes.

K.1 Design

Detection by a bounded count. When a pre-filtered search on a backend that does not declare exact search returns $h < k$ hits, the layer asks the backend for $n = \min(\#\{d : m_d \models \varphi_s\},\ L)$, a filter-only count that stops at a limit L. If $n \leq h$, the store returned every chunk the filter admits, and the answer is complete with respect to φ_s (*exhausted*). If $n > h$, chunks that pass the filter were missed (*missed*). The count is the second certificate of Section 2.2; it never raises a false alarm, because a count above h proves a miss, and it never certifies falsely, because a count no larger than h proves that nothing else passes φ_s. Both statements hold for the store's state at the time of the count, so H5.1 below is a logical consequence on unchanging data; a write between the search and the count can make either one stale, as any read without a snapshot can, and we did not test that case. It costs one query per short answer, and the limit keeps that query bounded on stores that count by scanning.

Repair by bounded exact search. After a proven miss the layer can search again, exactly, under the same filter: pgvector disables the index scan for that query, so every row that passes the filter is ranked, and Qdrant sets its per-request exact flag. The repair runs only when the count is at most a bound B (`max_exact_rows`, 10,000 by default), so that a subject who can read most of the corpus cannot trigger a scan of all of it; above the bound the first hits are kept and the result says *over_limit*. With $L = B + 1$ one count both detects and bounds. The alternative *widen* repeats the search on the same store with more effort, for example a larger scan budget or ACORN, and merges the two answers; it is cheaper but certifies nothing, since a wider approximate search can still miss. OpenSearch's Faiss engine already falls back to exact search when an approximate filtered search returns fewer than k results and at least k documents pass, unless a setting disables it [112]; the layer does the same above the store, for stores and engines that do not. The OpenSearch configurations we ran use its Lucene engine.

Security. The repair is one more search step in the layer's single plan, shared by the synchronous and asynchronous drivers. It uses the filter already obtained from the authorizer, and every hit it returns passes the same re-check as any other. The hits of the first search that the repair discards are re-checked in the same call, so a store that ignored the filter still shows up as violations. A failed count or search ends the request with an error and no hits, as any other failure does (assumption A3). Observation 1 therefore holds unchanged.

A sampled recall monitor. Neither the count nor the flag can see mis-retrieval. For a configurable share of pre-filtered requests, the layer also runs the exact search and reports the overlap of the two top-k lists as a metric, without returning the exact hits. This turns the offline audit of Section 6 into a production signal, at the cost of one exact search per sampled request.

Reporting. Each result carries the outcome of detection (*exhausted*, *missed* or *unknown*), of repair (*exact*, *widened*, *over_limit* or *unsupported*) and, when sampled, the measured recall; the same fields reach the audit events and the OpenTelemetry spans and metrics (Appendix D.5). The default policy remains *report*, which adds no query.

K.2 Evaluation

We evaluate the policies on pgvector and Qdrant, the two stores whose losses Section 6 diagnosed, on the enterprise scenario (1,468 questions with something to read) and on the 6,200 question–subject pairs of the MS MARCO layouts of Section 6.5, on X2's prepared data and in jobs of their own, three fresh builds per store and corpus. The base configuration is the audit's: pgvector with relaxed-order iterative scans, `ef_search` 100 and the default scan limits, each search planned with its own values; Qdrant with `hnsw_ef` 128 and no ACORN. Over the same backend and build we run *report*; *count*; *exact* with B of 1,000, 10,000 and unbounded; *widen*, whose wider search is pgvector with a million-tuple scan limit, a sixteenfold scan memory and `ef_search` 400, or Qdrant with `hnsw_ef` 512 and ACORN at gate 1.0; the 1% recall monitor; and, for comparison, the store-native alternatives alone: pgvector's tuple limit raised to a million, Qdrant's ACORN at gate 1.0, and, on the enterprise corpus, Qdrant's per-tenant graphs. Each question retrieves with $k = 5$ and $k = 10$ in a first pass and twice more with $k = 10$; every hit is re-checked by the layer and by the oracle, a separate plain implementation of the predicate, and latency is the median of the three passes with $k = 10$.

Seven hypotheses were pre-registered. The count detects every short first search without a false alarm (H5.1); exact repair without a bound leaves no result empty or short and reaches recall@10 of at least 0.999 on the queries it repairs (H5.2); no repair changes the median latency of the queries that do not trigger it by more than 5% either way (H5.3); no repair changes Qdrant's false-completeness rate, because its loss is mis-retrieval (H5.4); the monitor samples 1% of searches and its recall agrees with the harness's (H5.5); widening recovers less than exact repair on the searches that trigger it (H5.6); and on the enterprise corpus Qdrant's per-tenant layout reaches recall@10 of 0.90 while every library arm on the shared layout stays at or below 0.75 (H5.7).

Detection (H5.1, confirmed). As the design above says, this holds by logic on unchanging data, so the result checks the stores' counts and the harness. The count raised no false alarm and flagged every truly short first search as *missed*: 992 of 992 for pgvector on the enterprise corpus, 574 of 574 for pgvector and 1,091 of 1,091 for Qdrant on MS MARCO ($k = 5$ and $k = 10$ together), and every answer it marked *exhausted* held all $\min(k, R)$ readable chunks. Qdrant on the enterprise corpus had no truly short answer: its first searches came back short only for the one person who may read nothing, which the count marked exhausted. Qdrant did under-retrieve on MS MARCO: 3.5% of its

									All (ms)		Triggered (ms)	
Corpus, store	Arm	R@5	R@10	Empty	Short	Trigger	Detect	FC	p50	p99	p50	p95
Enterprise, pgvector	report	0.639	0.667	0.089	0.116	0.129	–	0.551	17.3	92.5	–	–
	count	0.639	0.667	0.089	0.116	0.129	1.000	0.542	17.7	114.1	14.7	25.6
	exact, B 1,000	0.747	0.782	0.000	0.000	0.129	1.000	0.480	17.6	122.7	25.3	30.6
	exact, B 10,000	0.747	0.782	0.000	0.000	0.129	1.000	0.480	17.3	120.9	25.2	30.3
	exact, unbounded	0.747	0.782	0.000	0.000	0.129	1.000	0.480	17.2	127.2	30.1	40.2
	widen	0.739	0.775	0.000	0.000	0.129	1.000	0.498	17.9	410.7	34.5	360.9
	monitor 1%	0.639	0.667	0.089	0.116	0.129	–	0.551	17.6	96.1	–	–
	tuple limit 10^6	0.664	0.694	0.072	0.090	0.103	–	0.543	17.6	128.9	–	–
Enterprise, Qdrant	report	0.658	0.654	0.000	0.000	0.014	–	0.581	4.72	6.15	–	–
	count, exact, widen	0.658	0.654	0.000	0.000	0.014	n/a	0.573	4.59–4.60	5.86–5.91	1.57–1.58	1.69–1.84
	monitor 1%	0.658	0.654	0.000	0.000	0.014	–	0.581	4.71	6.00	–	–
	ACORN 1.0 alone	0.857	0.855	0.000	0.000	0.014	–	0.388	4.84	6.35	–	–
	per-tenant graphs	0.938	0.939	0.000	0.000	0.014	–	0.235	4.69	8.27	–	–
MS MARCO, pgvector	report	0.896	0.900	0.003	0.017	0.017	–	0.301	9.87	425.4	–	–
	count	0.896	0.900	0.003	0.017	0.017	1.000	0.301	9.83	427.7	21.7	87.7
	exact, B 1,000	0.900	0.904	0.001	0.012	0.017	1.000	0.299	9.82	426.1	60.0	129.2
	exact, B 10,000	0.909	0.915	0.000	0.001	0.017	1.000	0.296	10.0	486.9	115.8	365.8
	exact, unbounded	0.909	0.915	0.000	0.000	0.017	1.000	0.296	10.0	446.5	108.6	220.7
	widen	0.907	0.913	0.000	0.000	0.017	1.000	0.306	9.89	772.1	594.4	1,572.7
	monitor 1%	0.896	0.900	0.003	0.017	0.017	–	0.301	10.2	433.2	–	–
	tuple limit 10^6	0.899	0.903	0.003	0.016	0.016	–	0.299	10.0	429.4	–	–
MS MARCO, Qdrant	report	0.809	0.807	0.000	0.035	0.035	–	0.356	6.07	10.7	–	–
	count	0.809	0.807	0.000	0.035	0.035	1.000	0.356	6.02	26.5	21.7	28.2
	exact, B 1,000	0.809	0.807	0.000	0.035	0.035	1.000	0.356	5.80	25.8	21.1	28.3
	exact, B 10,000	0.809	0.807	0.000	0.035	0.035	1.000	0.356	5.95	26.4	21.6	27.9
	exact, unbounded	0.833	0.842	0.000	0.000	0.035	1.000	0.343	6.10	44.4	37.9	50.4
	widen	0.833	0.841	0.000	0.000	0.035	1.000	0.349	5.56	117.8	97.4	148.5
	monitor 1%	0.809	0.807	0.000	0.035	0.035	–	0.356	5.15	10.6	–	–
	ACORN 1.0 alone	0.987	0.986	0.000	0.000	0.000	n/a	0.085	15.9	125.7	–	–

Table 14: X5: detecting and repairing under-retrieval (GitHub Actions run 40921769714; three builds per store and corpus; every arm on every question). Recall@k (R@5, R@10) and the rates are means over retrievals with $k = 10$ unless marked. Empty and Short: the answer has no hit, or fewer than $\min(k, R)$, where R is what the subject may read. Trigger: the first search came back with fewer than k hits, which sets off the count. Detect: the share of truly short first searches flagged *missed*; n/a: there were none; "–": the arm does not count. FC: false-completeness rate, the share of results marked complete that are not maximal. Latency: per-question median of three passes, end to end; "Triggered" is the time spent on the short result (count and repair) on triggered retrievals. Latency compares arms within one store and corpus only.

answers with $k = 10$ were short. On the enterprise corpus X0 and X1's second run never saw that, and X1's first run saw it rarely (0.6% of as-run answers with $k = 5$, and 1.1% in the one-segment layout).

Exact repair (H5.2, confirmed for the unbounded arm). Repaired exactly without a bound, no answer was empty or short, and the repaired answers reached recall@10 of 1.0000: on 513 triggered retrievals for pgvector on the enterprise corpus (of 576; the other 63 are the person who may read nothing, for whom recall is undefined), on 323 for pgvector and 644 for Qdrant on MS MARCO, which had returned 992, 574 and 1,091 short answers under *report* ($k = 5$ and $k = 10$ over three builds). Recall@10 rose from 0.667 to 0.782, from 0.900 to 0.915 and from 0.807 to 0.842. The bound decided where repair applied. On the enterprise corpus every short answer had at most 1,000 readable chunks, so all three bounds repaired all of them; on MS MARCO a bound of 1,000 repaired 0.5% of pgvector's answers out of 1.7% triggered, and every short answer of Qdrant had more than 10,000 readable chunks, so the bounded arms repaired none and reported each as *over_limit* and not complete. At the default bound of 10,000, then, 0.1% of pgvector's answers and all of Qdrant's 3.5% stayed short on MS MARCO. Where the repair ran without a bound, the count and the exact search together took 30.1 ms at the median and 40.2 ms at p95 of a triggered pgvector retrieval on the enterprise corpus, 108.6 and 220.7 ms on MS MARCO, and 37.9 and 50.4 ms for Qdrant.

Cost on other queries (H5.3, inconclusive). Of the 60 median latency ratios between a repair arm and *report* on the questions that triggered neither, 45 lay within the pre-registered band of 0.95 to 1.05, one above it (1.054, Qdrant on MS MARCO, build 2, B 10,000) and 14 below it, down to 0.866: the repair arm faster than *report*. None exceeded the falsification bound of 1.10. Twelve of the 14 low ratios are Qdrant's, and the arms ran in a fixed order after *report* within each build, so they are consistent with the store's latency drifting during a build rather than with any effect of the arms; the run cannot separate the two. Over all questions, every repair arm's median latency lay within 0.6 ms of *report*'s.

Mis-retrieval (H5.4, confirmed). No repair touched Qdrant's loss on the enterprise corpus: 58.1% of its answers marked complete under *report* were not maximal, and 57.3% under every repair arm, a difference of −0.008 (95% interval −0.027 to 0.000). The small change is the count marking the empty answers of the one person who may read nothing exhausted, and so complete and correct; no answer was repaired. Across the table, exact repair lowered the false-completeness rate only as far as short answers had caused it (0.551 to 0.480 for pgvector on the enterprise corpus); the rest is mis-retrieval, which a full answer hides from any count.

The monitor (H5.5, confirmed). The monitor sampled within the 99.9% binomial interval around 1% in all twelve builds (48 to 65 of 5,956 retrievals per build on the enterprise corpus, 210 to 267 of 24,800 on MS MARCO). Its recall, computed against the store's own exact search, agreed within 0.1 with the harness's on all 1,767 sampled retrievals, and its mean estimated Qdrant's recall over all retrievals within 0.030 (0.684 on 166 samples against 0.654) on the enterprise corpus and within 0.004 (0.812 on 725 against 0.807) on MS MARCO. A 1% sample is thus enough to see the loss that `complete` cannot.

Widening (H5.6, confirmed). On the 421 questions that triggered in both arms, exact repair beat widening by 0.085 of recall@10 (95% interval 0.064 to 0.110), and widening cost more: 360.9 and 1,572.7 ms at p95 of a triggered pgvector retrieval, on the enterprise corpus and on MS MARCO, against 40.2 and 220.7 ms for exact repair. A wider approximate search filled the short answers, since it left none empty or short, but did not rank them exactly.

Store-native fixes (H5.7, confirmed). On the enterprise corpus Qdrant's per-tenant graphs reached recall@10 of 0.9386 while the best library arm on the shared layout reached 0.6535, and the false-completeness rate fell from 0.581 to 0.235. ACORN alone reached 0.855 there and 0.986 on MS MARCO, where it also removed every short answer, at a median of 15.9 against 6.1 ms. pgvector's tuple limit raised alone lowered the enterprise corpus's empty-result rate only from 0.089 to 0.072, because the scan's memory bound then stops it, as X1 found (Section 6.3).

The two tools divide the work. A count tells a short answer from a small readable set without error. Exact repair without a bound fixes every short answer for tens to hundreds of milliseconds spent on that answer; within a bound it fixes only the answers whose readable set fits, none of Qdrant's on MS MARCO. Neither sees a full answer of the wrong chunks, which was most of the loss on both stores (pgvector's recall@10 on the enterprise corpus was 0.782 after exact repair, 0.667 without it); only a sampled exact search sees it, and only a better index or layout removes it.

L Side channels in detail (X6)

This appendix gives the setup, the hypotheses and the full results of the side-channel experiment, including the forensics of the build offsets, which Section 8 summarizes. Beside the game's differencing attacker (Section 8.1), a forced-choice attacker whose advantage is $2\,\mathrm{AUC} - 1$ and for whom $|\mathrm{AUC} - 0.5|$ is half the size of that advantage, we report a *one-shot attacker*, who sees one observation and must threshold it; its success is the AUC over all W_1 against all W_0 observations. The pre-registration also asks for the one-shot attacker's largest true-positive rate minus false-positive rate over thresholds; it is in the released data, as `ks` in `x6_cells.csv`. For post-filter latency at $m = 100$ it was 0.64 to 0.66 at $p = 10\%$ and 0.88 to 0.93 at $p = 50\%$.

L.1 Setup

Data and probes. The background is all 57,638 FiQA passages [85], with its 6,648 queries as probe candidates; far chunks come from 12,000 SciDocs abstracts, both from BEIR [154], and all text is embedded with all-MiniLM-L6-v2 (384 dimensions) [133, 169]. Each of five seeds places eight groups of nine probes, one with $m = 1{,}000$ secrets, two with 100, three with 10 and three with 1. A probe's secrets are its m nearest passages, excluding hubs, the 1% of passages found among the 2,000 nearest neighbors of the largest number of queries; within a group, no probe's secrets fall in the zone a group mate can explore, its nearest passages up to 1,000 beyond its own secrets, which is the post-filter's cap. One index W_1 holds every group's secrets; for each group g, an index W_0^g replaces only group g's 1,233 secrets by far chunks. A probe of group g thus compares two worlds that differ in its own secrets and in those of its eight group mates, which the zone rule places beyond the post-filter's cap of 1,000 candidates for it; collection statistics, however, count them all. Background passages fall into 100 permission buckets, and three attackers read 1%, 10% and 50% of them; secrets carry an ACL none of them holds. Every seed builds every index afresh, W_1 first, so every W_0^g is built later in its seed than W_1 and build age is aligned with world in every latency cell.

Configurations. The *pre-filter* runs each store's approximate filtered search at the settings the pre-registration fixed, which it calls documented defaults: pgvector (HNSW $m = 16$, `ef_construction` 64, `ef_search` 40, with iterative scans in relaxed order, which pgvector's own default leaves off), Qdrant (default HNSW) and Elasticsearch (`num_candidates` 100, against its default of 15 at $k = 10$), with FAISS over a flat index and WARDOWL's in-memory backend as exact controls. The *post-filter* is WARDOWL's widening loop at its defaults: fetch $4k$, widen fourfold until k hits are readable, the store runs out, or 1,000 candidates have been examined. Four mitigations are measured against

them. *Constant-work widening* (M1) fetches the cap of 1,000 in one round, so every request examines the same number of candidates. *Exact pre-filtered search* (M2) ranks every readable chunk: pgvector with index scans disabled, Qdrant's exact flag, Elasticsearch's `script_score`. *Separation* (M3) gives the attacker an index that holds only its own tenant's chunks, as a tenant-per-index layout would; its advantage is zero by construction, and measuring it twice provides an A/A null for the harness. *Latency padding* (M4) pads each answer to the next multiple of a budget fixed on the first seed and is evaluated on the other four. No padder ran: the padded latency is computed afterwards from the recorded latencies, so it models an ideal padder and ignores scheduler jitter. For collection statistics, keyword search runs under the pre-filter on Elasticsearch (BM25), on Qdrant (sparse vectors with an IDF modifier) and on pgvector (`ts_rank_cd`, which uses no collection statistics: a negative control), and Elasticsearch adds reciprocal-rank fusion of vector and keyword search.

Measurement and statistics. For every seed, store, probe, attacker and configuration, one warm-up is followed by five timed repetitions, each of which runs W_0^g, W_1 and the two separated indexes in a fresh random order, so drift affects every world alike; an observation is the median over repetitions. The in-memory control, which rescores the whole corpus on every search, runs a reduced design fixed in advance: three probes per level and seed, $p = 10\%$ only, three repetitions. Every retrieval is re-checked, and a hit outside the attacker's readable set would stop the run. Intervals are 95% cluster-bootstrap intervals over (seed, probe) pairs with 10,000 resamples; they do not resample index builds, the demonstrated source of latency noise, so latency intervals are too narrow. A channel is *present* when the paired AUC reaches the hypothesis's threshold and its interval's lower end exceeds 0.55, and *absent* when the interval lies within $[0.40, 0.60]$. Two guards precede any verdict: the A/A comparison of the two separated indexes must show no latency channel, and exact configurations must return identical hit lists in W_0^g and W_1 for at least 99% of probes. The experiment runs on GitHub-hosted runners with four virtual CPUs and 16 GB of memory.

L.2 Full results

The run. Every pre-registered cell ran: five seeds for each store, all 72 probes placed in every seed, and 520,320 retrievals without an error. None returned a chunk outside the attacker's readable set. In three seeds the SciDocs passages far enough from every probe (2,331 to 4,864) were fewer than the secrets (8,417 to 8,985), and the harness reused far chunks rather than drawing them without replacement as the pre-registration states; every far chunk still lay outside every probe's zone. Table 16 summarizes the channels by m. Verdicts are as the analysis script printed them; where the script departs from the written rules, we say so.

Guards. Exact search returned identical hit lists in W_1 and W_0^g for every probe and attacker (G2: change rate 0.000 for pgvector, Qdrant and Elasticsearch with exact search and for FAISS). The A/A guard G1 held for all 20 configurations of the four vector stores, with paired AUCs from 0.463 (95% interval 0.436 to 0.492) to 0.536 (0.507 to 0.563) over 360 clusters. It failed for the in-memory control (0.533, interval 0.400 to 0.650; and 0.500, 0.367 to 0.633): with 60 clusters and three repetitions, an interval inside $[0.40, 0.60]$ was out of reach, so its latency parts are inconclusive, as they would have been in any case (pooled latency 0.383, 0.267 to 0.500, and 15 clusters per level). G1 compares one index with itself; it cannot see a difference between two builds, and such differences appear below.

Widening (H1, falsified). Section 8.2 gives the paired AUCs at $m = 100$. There FAISS took 14.8 against 7.2 ms on average at $p = 50\%$, and the one-shot attacker did almost as well (pooled AUC 0.962 for FAISS at $p = 50\%$). pgvector's 0.662 at $p = 10\%$ was inconclusive, although W_1 took

Hypothesis	Prediction	Verdict
H1 widening	Post-filter: count AUC ≥ 0.95 at $m = 1{,}000$; latency AUC ≥ 0.75 at $m = 100$ ($p = 10\%, 50\%$); no channel at $m = 1$	Falsified: count 0.613 to 0.688 at $p = 50\%$; FAISS latency 0.667 at $m = 1$
H2 constant work (M1)	No latency channel; the count channel remains at $m = 1{,}000$	Falsified: count as in H1; a reverse latency channel on pgvector
H3 approximate pre-filter	pgvector and Elasticsearch: latency AUC ≥ 0.75 at $m = 1{,}000$; Qdrant: no channel at $p = 1\%, 10\%$	pgvector confirmed; Elasticsearch inconclusive; Qdrant confirmed
H4 exact pre-filter (M2)	No channel in any observable	Inconclusive: identical answers; through latency, build offsets on pgvector and FAISS, wide intervals at 40 clusters on Qdrant and Elasticsearch; the in-memory control failed G1
H5 collection statistics	BM25 and IDF-weighted sparse: score AUC ≥ 0.90 at every m; `ts_rank_cd`: none; fusion changes the hit list for at least 25% of probes at $m = 1{,}000$	BM25, sparse: inconclusive (0.794 to 0.886 at $m \leq 10$); `ts_rank_cd`, fusion: confirmed
H6 padding (M4)	No latency channel wherever the unpadded AUC was ≥ 0.75 (forward channels only)	Inconclusive as printed (script defect); confirmed under the written rule, by construction

Table 15: Pre-registered hypotheses of the side-channel experiment, abbreviated, with the verdicts of the full run (Section 8.2); the full rules, including what falsifies each, are in the released pre-registration.

18.7 against 10.7 ms on average: on probes that needed no extra round, its post-filter was slightly faster on W_1, as it was at $m = 1$ (0.369) and $m = 10$ (0.383). At $m = 1{,}000$ an attacker reading 1% of the background received a short answer in W_1 for every probe on every store (count AUC 1.000).

Two parts failed. At $m = 1{,}000$ and $p = 50\%$ the count AUC was 0.613 to 0.688, below the falsification bound of 0.80 on every store, and at $p = 10\%$ it was 0.912 to 0.925. The prediction assumed that the 1,000 nearest candidates are all secrets. Hubs may not be secrets, so they remain in the background, interleaved with the secrets: a probe's 1,000th secret ranked as deep as 1,040 to 1,271 (the deepest per seed), and readable hubs among the post-filter's 1,000 candidates filled the answer. On FAISS, W_1 returned 0.375, 3.35 and 7.60 hits on average at $p = 1\%$, 10% and 50%, and its answer was complete for 62.5% of probes at $p = 50\%$, which accounts for the AUC of 0.688, since every W_0^g answer was complete. A follow-up check, exploratory, specified in writing after the main results and before running, confirmed the mechanism probe by probe: the 1,000 nearest passages of each $m = 1{,}000$ probe held between 1 and 231 hubs (45.7 on average) and no passage that was neither a hub nor a secret, and FAISS returned exactly the smaller of 10 and the number of readable hubs among them, for all 40 probes and every attacker; the same count over the background predicted every W_0^g answer. The count channel is only as strong as readable content among the secrets is scarce.

The second failure is a latency difference on FAISS at $m = 1$: paired AUC 0.667 (0.617 to 0.717). Its answers were identical in both worlds for all 360 pairs, the mean difference was 0.05 to 0.14 ms on 7 to 46 ms, and the AUC was the same at $m = 10$ (0.672), with ten times as many secrets among the candidates. Configurations whose answers are identical in both worlds show such offsets, of either sign, at every m: FAISS's exact pre-filter was faster on W_1 (paired AUC 0.25 to 0.35), and pgvector's exact search was 1.5 to 3.7 times slower on W_0^g (0.12 to 0.17).

Two follow-up analyses, likewise exploratory and specified in writing before they ran, examined

Configuration	Observable	$m = 1$	10	100	1,000
Post-filter, FAISS	latency	0.67	0.67	0.80	0.76
Post-filter, pgvector	latency	0.37	0.38	0.57	0.68
Post-filter, Qdrant	latency	0.47	0.46	0.75	0.78
Post-filter, Elasticsearch	latency	0.48	0.58	0.76	0.86
Post-filter, all four	count	0.50–0.51	0.50–0.51	0.58–0.59	0.84–0.87
Constant work, FAISS	latency	0.63	0.57	0.46	0.13
Constant work, pgvector	latency	0.29	0.22	0.07	0.02
Constant work, Qdrant	latency	0.47	0.47	0.45	0.35
Constant work, Elasticsearch	latency	0.57	0.57	0.55	0.62
Pre-filter, pgvector	latency	0.33	0.44	0.79	0.88
Pre-filter, pgvector	recall	0.49	0.47	0.31	0.22
Pre-filter, Qdrant	latency	0.47	0.43	0.47	0.57
Pre-filter, Elasticsearch	latency	0.49	0.50	0.48	0.60
Exact, FAISS (pre-filter)	latency	0.25	0.30	0.25	0.35
Exact, pgvector	latency	0.17	0.14	0.13	0.12
Exact, Qdrant	latency	0.45	0.46	0.50	0.42
Exact, Elasticsearch	latency	0.50	0.48	0.47	0.47
Exact, all four	count, recall, score	0.50	0.50	0.50	0.50
BM25, Elasticsearch	score	0.79	0.88	0.98	0.99
Sparse IDF, Qdrant	score	0.81	0.89	0.98	1.00
`ts_rank_cd`, pgvector	score	0.50	0.50	0.50	0.50
`ts_rank_cd`, pgvector	latency	0.45	0.47	0.50	0.70

Table 16: Paired AUC of the differencing attacker, W_1 against W_0^g, pooled over the three attackers: 120, 120, 80 and 40 probes for $m = 1$, 10, 100 and 1,000 over five seeds. Above 0.5 the observable moves in the direction predicted for W_1 (latency up; count, recall and score down); below 0.5 it moves the other way, which serves an attacker who knows the sign just as well, so the size of a channel is $|\text{AUC} - 0.5|$. Latency values must be read against the differences between builds of identical content, which reached, per m, $|\text{AUC} - 0.5|$ of 0.16 for FAISS's post-filter, 0.38 for its pre-filter and 0.33 for pgvector's exact search, and 0.08 for Qdrant's exact search (Section 8.2); the other configurations were not measured that way. No correction for multiple comparisons is applied: the verdicts rest on the cells the hypotheses name, and the rest of the table is descriptive. Intervals for the cells the hypotheses name are in the text; all are in the released report.

this offset. In the main run's records the whole difference lay in FAISS's own search call: 0.115 ms (95% interval 0.076 to 0.154; paired AUC 0.681) against 0.106 ms of latency, with the same number of widening rounds in all 360 pairs and no difference in W_1's direction in the authorizer (-0.001 ms) or the rest of the layer (-0.010 ms). A second run then built, for each group, W_1 again and W_0^g twice, just before the group, and measured them with the original W_1 in random order. It reproduced the direction of the offset, more weakly (W_1 against the first W_0^g: 0.597, 0.539 to 0.656, at $m = 1$), and two builds of identical content differed by as much: W_1 against its own rebuild 0.600 (0.547 to 0.653), the second build of W_0^g against the first 0.361 (0.308 to 0.414), at every m alike. With builds matched in time, the difference in content gave 0.447 (0.392 to 0.506) at $m = 1$, but 0.742 (0.700 to 0.783) at $m = 1{,}000$, where the widening channel is real. The reading we fixed in advance called the offset a build difference only if that $m = 1$ interval lay within $[0.40, 0.60]$; it reaches 0.392, so the formal reading is inconclusive. What the numbers show is that two FAISS indexes of the same content differ in search time by as much as W_1 and W_0^g did, while the secret adds no detectable offset of its own at $m = 1$.

These same-content comparisons set the noise level against which every latency channel of Table 16 must be read. Two builds of identical content gave paired latency AUCs of 0.342 to 0.608 for FAISS's post-filter, depending on m and on which builds were compared ($|\text{AUC} - 0.5|$ up to 0.16). Pooled over m they were 0.257 and 0.426 for its pre-filter (per m, up to 0.38, at $m = 1{,}000$), 0.186 and 0.789 for pgvector's exact search (per m, up to 0.33; from the tables' vacuum state, below), and 0.494 and 0.505 for Qdrant's exact search (per m, up to 0.08, at $m = 1{,}000$, where its cell reads 0.42); we have no such measurement for the other configurations, and Elasticsearch's was lost. A latency AUC anywhere from about 0.12 to 0.82, the range these builds of identical content reached, is therefore not by itself evidence about the secrets. To separate a channel from an offset we use a criterion adopted after seeing these results: a channel grows with m, as in the widening loop and pgvector's iterative scan, while an offset stays the same at every m, as in the configurations whose answers are identical. Same-content differences were themselves largest at $m = 1{,}000$ for FAISS's post-filter and pre-filter and Qdrant's exact search, in the cell with the fewest pairs, so a rise at that m alone does not establish a channel.

Constant work (H2, falsified). Fetching the cap in one round removed the widening channel from latency pooled over m and p on FAISS (0.517, 0.484 to 0.549) and Qdrant (0.456, 0.422 to 0.491), and left the count channel, which falsifies H2 at $p = 50\%$ as it does H1. Timing did not become uninformative. Where the candidates are secrets, W_1 answered *faster*: on pgvector the paired AUC fell from 0.29 at $m = 1$ to 0.017 (0.000 to 0.050) at $m = 1{,}000$, where W_1 took 23.5 against 26.4 ms; FAISS fell to 0.133 (0.075 to 0.200) and Qdrant to 0.350 (0.225 to 0.483), while Elasticsearch drifted the other way (0.573 pooled, 0.625 at $m = 1{,}000$; inconclusive). This reverse channel grows with m on pgvector (size 0.21 to 0.48), which suggests a dependence on the secrets, but pgvector's exact search shows an offset of the same sign (size 0.33 to 0.38, flat), no same-content comparison exists for constant work and the mechanism is not established, so the 98% (95% to 100%) of pgvector's probes at $m = 1{,}000$ that an attacker who knows the sign would identify may include an offset of unknown size. The script reads the rule's "upper end at most 0.65" at $m = 1{,}000$ as the window $[0.40, 0.65]$ and prints these parts inconclusive; read one-sidedly, as worded, they would pass, which would hide the channel.

Approximate pre-filtering (H3). pgvector's iterative scan separated the worlds as predicted (H3a confirmed): at $m = 1{,}000$ every probe was slower on W_1 at $p = 10\%$ and 50% (paired AUC 1.000; 13.5 against 4.7 ms and 13.7 against 3.3 ms), and so was every probe at $m = 100$ and $p = 50\%$. The same scan returned *better* answers on W_1: recall@10 was 0.968 and 0.978 against 0.707 and 0.665 on W_0^g. This is consistent with how an iterative scan works: made to pass the unreadable

secrets, it ranks many more readable rows before it stops and so comes close to exact search. An attacker thus learns of nearby secrets from the quality of its answer as well as from its timing. Elasticsearch (H3b, inconclusive) was slower on W_1 for most probes at $p = 10\%$ (0.800, 0.675 to 0.925), yet the mean latencies were 2.426 and 2.430 ms, and the interval at $p = 50\%$ (0.463 to 0.750) straddles the threshold. Qdrant showed no channel in any observable at $p = 1\%$ and 10% (H3c confirmed; latency 0.460, 0.424 to 0.496), where its filter is small enough for an exact scan of the payload index. At $p = 50\%$, outside the hypothesis, its filtered traversal had to cross the secrets: recall@10 on W_1 fell to 0.530 against 0.708 (paired AUC 0.787, 0.688 to 0.875), and some answers held fewer than ten hits, consistent with the island of Section 6.2, which we did not measure directly.

Exact pre-filtering (H4, inconclusive). Exact search returned the same answer in both worlds: count, completeness, recall and score gave a paired AUC of 0.500 (0.500 to 0.500) on all four stores, pooled and at $m = 1{,}000$. Every H4 verdict is nevertheless inconclusive, through latency alone. On Qdrant and Elasticsearch latency showed no channel pooled (0.461 and 0.486), and their intervals at $m = 1{,}000$ (0.325 to 0.508 and 0.367 to 0.575) were too wide at 40 clusters for the script's window; under the rule's one-sided wording both stores would be confirmed. On pgvector (0.147, 0.123 to 0.172) and FAISS (0.276, 0.246 to 0.306) latency carried the build offsets described under H1, the same at every m and so not about the secrets. The follow-up traced pgvector's to the tables' state. At the start of every group the table built last had no page marked all-visible, because autovacuum had not yet reached it (40 of 40 groups), while the table built at the start of the seed always had; in the main run the table built last was W_0^g. Exact search slowed with the share of groups a table started unvacuumed: a mean of 59.9 ms on W_1, never unvacuumed at a group's start, 61.4 and 74.3 ms on later builds unvacuumed in 4 and 15 of the 40 groups, and 107 ms on the build unvacuumed in 34. Once every table had been vacuumed, the four ran the same plan with the same buffer counts in the same time. By the reading fixed in advance this too is inconclusive (matched content 0.567, 0.530 to 0.602), and a small difference remains after vacuuming that we cannot explain. Qdrant's exact search showed no difference between builds in any comparison (0.493 to 0.505); the Elasticsearch part of the follow-up was lost when its runner stopped responding, so Elasticsearch has no such comparison. The in-memory control failed G1.

Collection statistics (H5). Beside the results and prior work of Section 8.2: at $m = 1$ and 10 the AUCs were above the falsification bound, so H5a is inconclusive for both stores. No cell isolates m documents, at any m: W_1 and W_0^g differ in all 1,233 secrets of the probe's group, which the zone rule keeps beyond the reach of dense search but which collection statistics count. The shifts are small (a mean score of 13.634 against 13.676 at $m = 1$ on Elasticsearch) and invisible to the one-shot attacker (pooled AUC 0.503 to 0.531). `ts_rank_cd` showed no score channel and changed 0.2% of hit lists (H5b confirmed), although its latency, on which we made no prediction, separated the worlds at $m = 1{,}000$ (0.70, 0.58 to 0.81). Fusion changed the hit list for 56.7% of probes at $m = 1{,}000$ (48.3% to 65.0%; H5c confirmed), and for 26.9% at $m = 1$.

Against prior work. Wang et al.'s STRESS attack writes documents to a shared index, maps its shards and uses score differences, or ranks alone, to discover terms in other tenants' documents and count the documents that hold a term; they propose document frequencies from public data as a countermeasure [165]. Espiritu and Cash extend this to indexed terms and, with search-as-you-type indexes, approximate text, through document-level security in Elasticsearch and OpenSearch, and recover records from PostgreSQL row-level security's timing with range and conjunctive predicates; in their tests cross-region placement and concurrent load lowered accuracy until more queries restored it, and a composite index with a subquery policy removed the timing channel for their scenario [49]. Neither paper tests vector search. Our channels, measured in a forced-choice game with no inserted

documents, are weaker claims: the score result re-measures their channel through the layer's pre-filter, and our latency channels come from the widening loop and the iterative scan. We did not measure row-level security's channels, although Appendix O.1 reports its recall. The README of an open-source project reports count, relevance and timing channels of filter-after-retrieval RAG on a 35-document synthetic organization, in process, which we did not reproduce [117].

Padding (H6). Seven cells had a latency channel before padding on seeds 1 to 4 (paired AUC 0.755 to 0.927): the post-filter on FAISS and Qdrant at $m = 100$ and 1,000 and on Elasticsearch at 1,000, and pgvector's pre-filter at 100 and 1,000. The analysis script computes each padded latency as $T\lceil x/T \rceil$, with T the slowest latency of the first seed rounded up, so every recorded latency up to T becomes T, and the padded AUC was 0.500 (0.500 to 0.500) in all seven by construction: both worlds get one value, and ties count one half. The script printed H6 inconclusive because its test required five seeds of cells that by design have four; applied as written, the rule confirms H6, which adds nothing beyond that identity. The rule covers only cells whose forward AUC reached 0.75. Reverse channels, such as constant work on pgvector (paired AUC 0.017 at $m = 1{,}000$, $|\text{AUC} - 0.5| = 0.48$), fell outside it and were not tested, although a reverse channel serves an attacker equally.

Costs. Padding raised mean latency 5.2 to 6.7 times in those four configurations, from 1.33 to 27.1 times over all configurations, and by medians of per-probe ratios up to 57.6, since every request waits at least as long as the slowest answer of the first seed. Exact search cost 1.53, 2.01 and 8.10 times the mean latency of the approximate pre-filter on Qdrant, Elasticsearch and pgvector (intervals 1.52 to 1.53, 1.99 to 2.03 and 7.73 to 8.48). Constant work cost 1.05 to 1.23 times the widening loop's mean on the three servers and 0.74 on FAISS, where one round of 1,000 replaces up to four, but a median of 1.26 to 3.43 times per probe, because it charges every cheap request the price of the expensive ones. A separated index answered in 0.45 to 0.99 of the shared index's mean time in every configuration except pgvector's keyword search (1.07, 0.98 to 1.18), as it holds fewer chunks and no secrets.

Summary. In this forced-choice game each enforcement path separated the worlds through the observable its mechanism predicts: the widening loop through latency once about 100 unreadable chunks lie near a query and through the hit count when the attacker reads little, pgvector's approximate pre-filter through latency and recall, and keyword scores through collection statistics. Latency channels smaller than the 0.08 to 0.38 that separated builds of identical content, where we measured that, cannot be told from build noise unless they grow with m. Constant work traded the widening channel for a reverse one on pgvector. On the shared index only searches that ran exactly, whether requested or chosen by the planner (Qdrant's small filters), returned the same answer in both worlds, as exact search should, and a separated index does so by construction. Ideal padding costs five to seven times the mean latency of the configurations that needed it; it was not tested against reverse channels, and the detection fields of Appendix K.1 remain an untested observable.

M The end-to-end evaluation in detail (X0 and X8)

This appendix gives the full setup of the end-to-end runs, the safety table, the exposure and positive control behind it, and what the model did with what it retrieved, which Section 9 summarizes.

M.1 Setup

Corpus. The index holds 107,540 chunks, one per document or passage: 49,920 synthetic company documents in 20 tenants, of which 41,600 carry a unique canary string; 20 notices, one per tenant,

each carrying a prompt injection; and 57,600 passages of the FiQA financial question-answering corpus [85] from BEIR [154], which every tenant may read and which serve as distractors. Company documents are drawn from templates (handbooks, finance, HR, board and project records) with per-document ACLs over users and groups. Chunks are embedded with bge-base-en-v1.5 [177].

People and questions. 189 people span the tenants. A typical employee may read the shared passages, the tenant's notice and five of the twelve document templates, 58,641 chunks in all (the median); the least privileged are non-executive board members ("judy" personas), each of whom may read 624 chunks of three templates in one tenant and none of the shared passages (0.58% of the corpus); one person in no group may read nothing. They ask 1,105 questions: 108 generic, 376 targeted at a document they may read, 189 targeted at a document they may not read (*forbidden*), and 432 red-team prompts that ask for other people's data, other tenants' data or everything, or try to override instructions.

Policy and stores. The policy is AllOf(Tenant, MetadataACL) (Appendix D.4). It runs against ten configurations: FAISS over a flat index (exact, the reference), Chroma, pgvector, pgvector with the same policy written in OPA, Qdrant, Elasticsearch, OpenSearch, Weaviate, Redis and LanceDB, each at the settings listed in Appendix P. Each question retrieves with $k = 5$ for the answer and $k = 10$ for recall@10. Qwen2.5-3B-Instruct [181] answers from the five chunks. An eleventh configuration, pgvector with the policy's ACLs held as 74,960 OpenFGA relationship tuples behind the caching authorizer, pre-filters with ListObjects. It did not start in the runs below: our harness set OpenFGA's ListObjects deadline to 20 s while leaving its request timeout at the default, and OpenFGA 1.22 refuses to start with a deadline longer than the request timeout. With the timeout raised and nothing else changed, the configuration ran in four further runs with the same seeds, pre-registered as an amendment before they started; each also repeated the in-process pgvector configuration.

Runs. The scenario ran four times with seeds 7, 11, 13 and 17, on one Kaggle session with two T4 GPUs each (Appendix Q). The seed-7 run was executed twice with the same notebook, seed, acceptance scripts and release: on 2026-10-24, and again on 2026-11-07. The artifact holds the second execution only, and every seed-7 number in this paper comes from it. The pre-registrations of X1, X2 to X5 and X8 were written after the first execution and quote its printed aggregates; where a prediction is judged against them, the text says so. The two executions asked the same 1,105 questions in the same order. Their aggregates differ as runs with different seeds do: the first had recall@10 of 0.732 on pgvector with ten empty answers on nine people, the second 0.725 with none, and no other store moved by more than 0.04 (the values the X2 note records beside its verdicts, `evidence/x2-x5-run1.md`). The recorded software is the same in all four runs, as the pre-registration required: Redis 8.10.3, lancedb 0.41.0, weaviate-client 4.24.1, Elasticsearch 9.5.5, OpenSearch 3.9.0, Qdrant 1.19.2, Weaviate 1.39.12, chromadb 1.5.9 and faiss-cpu 1.15.1, except psycopg, 3.3.8 in the seed-7 run and 3.3.6 in the repeats. The seed changes the document text, part of the sample of questions and the order in which people ask; the permission templates, personas and the 216 pairs of Section 9 are fixed, so the runs repeat the layout with fresh documents and mostly fresh questions. The repeats and their predictions were pre-registered after the first run and before any repeat. The runs used the released version 2.1.0, before the change of Appendix D.3; permissions did not change during a run, so these runs test containment, not freshness, which Section 5.1 measures.

Leak detection. A leak is any returned chunk outside the person's readable set, as decided by the policy oracle, which evaluates the policy without the layer, or any canary of an unreadable document found in a retrieved chunk, in the prompt sent to the model or in the model's answer.

Configuration	Questions	Hits checked	Unreadable hits	Canaries (chunk/prompt/answer)	Violations
FAISS (exact)	4,420	65,440	0	0/0/0	0
Chroma	4,420	65,440	0	0/0/0	0
pgvector	4,420	65,117	0	0/0/0	0
pgvector + OPA	4,420	65,141	0	0/0/0	0
Qdrant	4,420	65,440	0	0/0/0	0
Elasticsearch	4,420	65,440	0	0/0/0	0
OpenSearch	4,420	65,440	0	0/0/0	0
Weaviate	4,420	65,440	0	0/0/0	0
Redis	4,420	65,440	0	0/0/0	0
LanceDB	4,420	65,440	0	0/0/0	0
Total	44,200	653,778	0	0/0/0	0
pgvector + OpenFGA*	4,420	64,845	0	0/0/0	0
pgvector*	4,420	65,405	0	0/0/0	0

Table 17: End-to-end safety over four runs (seeds 7, 11, 13 and 17). The unit of the bounds in the text is the question and the person; hits are secondary and are counted over both retrievals per question, so about a third of them, the $k = 5$ hits, largely repeat chunks already counted among the $k = 10$ hits of the same question. pgvector and its variants checked fewer hits because some of their answers were short or empty (Section 6). *Four further runs with the same seeds, in which only pgvector and pgvector with OpenFGA ran. No configuration reported an error.

A canary is a 40-bit code derived from the document's identifier; the matcher finds it in any case and with any hyphen, dash, space or nothing between its parts, and a positive control checks, for every person, that it finds exactly the canaries of the documents that person may not read in a text that holds all of them. A model cannot produce a 40-bit code it has not been shown, so an answer can leak a canary only through a prompt that contains it, and prompts are checked verbatim. The answer check guards against a fault in the pipeline between prompt and answer, not against paraphrase.

M.2 Safety in detail

No configuration in any run returned an unreadable chunk or exposed a canary (Table 17). The bounds of Section 9 count distinct (person, question) pairs. In the generator (`questions()` in the harness at bf25bd0), the generic pairs are 12 fixed strings asked by nine people, and two of the eight attacks, the 54 `ignore-instructions` and 54 `reference-codes` prompts of a run, have no placeholder, while the attack assigned to a person does not depend on the seed. So 216 of 1,105 pairs, 19.5%, are the same in all four seeds. The other pairs choose an entity from a pool of 208 per seed. The runs' stored questions, the same for every configuration of a seed, hold 3,746 distinct pairs: 216 in all four seeds, 26 by chance in two and 3,504 in one. The one-sided 95% bound, separately for each of the eleven configurations and taking those pairs as independent, is $1 - 0.05^{1/3746} \approx 8.0 \times 10^{-4}$ per question (Section 2.4); the pre-registered analysis, which treated all 4,420 as independent, gives 6.8×10^{-4}. They are not fully independent even so: a document question comes from 12 templates and a red-team prompt from eight, and the same people ask them. Per person the bound stays at a single run's $1 - 0.05^{1/189} \approx 1.6 \times 10^{-2}$: the same personas recur in every run with the same permissions, so a fault tied to a permission shape would recur too. The 189 people are 13 role types, since the persona's name fixes its groups, repeated across tenants, and over 13 the formula gives 0.21. Pooling the configurations would not shrink these bounds honestly, since they share questions and the layer's code. The first of the five pre-registered predictions for the repeats, that no unreadable hit, canary or violation would occur, held, and so did the amendment's for OpenFGA,

which started in every run and pre-filtered every question. As a secondary count, the runs checked 784,028 hits, not independent trials, since hits cluster by question and by person, and a fault in the layer would affect a question, not a random hit.

These bounds are over the question generator, and a binomial bound over sampled questions says nothing about an adversary who adapts its queries to what it sees. No such adversary was run. For one we rely on Observation 1, which does not depend on the query, and on its trusted computing base.

No violation occurred, so across these runs the re-check dropped nothing, and the same outcome would appear with it off. In the in-process configurations the filter and the re-check evaluate the same ACL metadata under the same reference semantics, so a violation would need a store to translate or evaluate the filter wrongly; with OPA and OpenFGA, filter and check come from the engine's separate APIs, and they agreed on every hit. Sections 4 and 5 test configurations and data shapes on which they do not. These runs thus test whether two implementations of one predicate agree and whether a store round-trips metadata.

Two observations qualify the result. First, it is a statement about this layer under assumptions (A1) to (A3) of Observation 1, not a proof, and the inputs called adversarial are weak: the red-team prompts can only change the query text, and the layer never lets text choose the subject (Appendix D.5). Second, containment held while quality did not, and quality did not repeat. pgvector returned empty answers to 0, 1, 0 and 7 questions in the four runs, from people who could read something. The harness counts answers, not people: the check's number is one entry per empty question. By the stored answers, the empties fell on 0, 1, 0 and 7 people, all least-privileged and each able to read 624 chunks, one empty answer each, all on red-team prompts; each asked five questions, so everyone got hits on four. The person who may read nothing is excluded from the check. The layer, before the change of Appendix D.3, reported those empty answers as complete. The same configuration ran in one of two modes, fast and lossy or exact and about fifty times slower, for a reason Section 6.3 ties to PostgreSQL's plan cache in a separate run. Our third prediction, that 3 to 25 people would receive no results in every repeat, was scored on this count and falsified: seeds 11 and 13 fell outside it, as does the seed-7 execution the artifact holds; the prediction took the ten empty answers of the first execution for a property of the configuration. Over its eight runs the configuration ran exact in five and, by its median, fast in three, one of them a mixture whose p95 of 433 ms shows that part of its searches ran exact, and it gave at least one empty answer in four of the eight (0, 1, 0, 7, 1, 1, 0, 0). The seed did not decide which: seeds 7, 11 and 17 each ran fast in one run and exact in another. In two of the five exact runs one person received one empty answer, which we have not explained. With OpenFGA, whose pre-filter lists the readable documents, pgvector was lossy in every run, with recall@10 between 0.693 and 0.758 and 7, 8, 3 and 10 empty answers on 6, 8, 3 and 9 least-privileged people, whatever mode the in-process configuration had just run in; its plans were not recorded, so we have only an inference from the code for this. Qdrant answered every question, but with recall@10 between 0.530 and 0.561 in every run. The other stores varied little from run to run (Section 6.1).

M.3 Exposure and the mutation positive control

A second amendment to the pre-registration, written after the results above and after our critical review and before running, asks whether those results have power. Its measurements are exploratory in that sense. The harness is unchanged: a runner imports it, on the same release, corpus parameters, models and hardware, with FAISS only.

		Top 5			Top 10		
Category	Questions	Unreadable	Mean	Canary	Unreadable	Mean	Canary
Generic	432	100.0%	4.77	83.3%	100.0%	9.57	83.3%
Targeted	1,504	100.0%	4.78	54.8%	100.0%	9.54	54.8%
Forbidden	756	100.0%	5.00	97.9%	100.0%	10.00	97.9%
Red team	1,728	87.7%	4.26	75.0%	99.5%	8.62	75.0%
All	4,420	95.2%	4.61	72.9%	99.8%	9.26	72.9%

Table 18: Exposure over seeds 7, 11, 13 and 17 on the unfiltered exact index: the share of questions with at least one chunk the person may not read, the mean number of such chunks, and the share with such a chunk that carries a canary.

Exposure. For every question of the four seeds we computed the exact top 5 and top 10 over the whole corpus, without any mask, and counted the chunks the asking person may not read by the oracle, and those in which the leak detector finds a canary of a document that person may not read (Table 18). The shares are the same to one decimal in every seed; the seed changes document text and the sample of questions but not the templates or personas. Each company document has a counterpart from the same template and pool entry in each of the other 19 tenants, and these shares depend on those copies. The control's ACL mutation below, with the re-check off, is exact search over the person's tenant and the shared passages: of its 5,105 distinct unreadable (question, chunk) pairs, 4,979 are in the person's own tenant, 126 are shared passages and none is in another tenant. We noticed only after the run that this pass measures exposure inside the tenant. On seed 7, in the top 5, it is 100% for the 189 forbidden questions, 62.5% for red-team prompts, 62.0% for generic questions, 0.5% for targeted ones and 47.8% overall, against 95.2% over the whole corpus, so targeted questions test the tenant boundary and hardly the ACL inside it. Of the seven predictions, one failed in part: targeted questions cleared the predicted 90% in the top 5 in every seed, and red-team prompts did not, at 87.7% in every seed. One attack, a request for reference codes, has no placeholder, so all its askers in a seed share one query, whose nearest chunks are shared passages: in the top 5 it exposed only the one person who may read nothing (1.8% of its questions), and in the top 10 it exposed 96.3%; the other seven attacks were at 100%. We had reasoned from the counterpart documents and had not considered such a query.

The positive control. On seed 7, FAISS was loaded once and the harness evaluated the same 1,105 questions in seven passes. A wrapper around the policy replaced what the policy's filter gave the store and left its check unchanged. M1 admits everything; M2 lets the tenant clause also admit the next tenant in the harness's list, with the ACL clause unchanged; M3 drops the ACL clause. Each person gained between 48,275 and 107,540 unreadable chunks under M1 (median 48,899), between 0 and 1,665 under M2 (median 1,041) and between 832 and 60,097 under M3 (median 1,456), and no mutation left out a readable chunk. With the re-check off, the release still matches each hit against the filter it gave the store, so it cannot see what a wrong filter over-admits, and it reports no violation.

Table 19 gives the result. All the predictions for the control held. The unmutated pass P0 reproduced the seed-7 run's FAISS row, 16,360 hits checked with no unreadable hit, canary or violation, and recall 1.000. M1 with the re-check off returned the same ten chunk ids as the unfiltered exact search for all 1,105 questions, which also ties the control to the exposure measurement. Each pass with the re-check off failed exactly the harness's four leak checks (every hit readable, and no canary in a chunk, a prompt or an answer). Each pass with it on failed two others: the re-check found violations, and people who may read something got empty results (887, 38 and 449 retrievals).

		Unreadable		Canary in			Violations			
Pass	Re-check	k=5	k=10	chunk	prompt	answer	questions	hits	Incomplete	R@10
P0	on	0	0	0	0	0	0	0	0	1.000
M1	off	1,052	1,103	805	805	138	0	0	0	0.077
M2	off	676	751	400	360	13	0	0	0	0.703
M3	off	528	533	509	503	107	0	0	0	0.548
M1	on	0	0	0	0	0	1,103	15,317	1,103	0.077
M2	on	0	0	0	0	0	751	4,834	751	0.703
M3	on	0	0	0	0	0	533	7,642	533	0.548

Table 19: The mutation positive control on FAISS, seed 7, 1,105 questions, the main retrievals at $k = 5$ and $k = 10$. Counts are questions, except the violations counted in hits. R@10 is recall@10 against the unmutated readable set.

We compared the passes question by question, not only in total, and a separate kernel recounted the comparison with no exception. For each mutation, the questions with an unreadable hit with the re-check off are exactly those with a violation with it on and exactly those marked incomplete, and on each question the number of violations equals the number of unreadable hits. Every canary in an answer was also in that question's prompt, as the design of the canaries requires (Appendix M.1).

Three limits apply. This is one seed and one configuration, an exact store, so it shows that the checks fire and that the re-check contains these faults here; it does not test the interaction of a wrong filter with an approximate index, and exposure was measured on FAISS only. It breaks only the filter, since the check stays the policy's, so a wrong check is outside it and stays in the trusted computing base, and containment under an untouched check holds by construction. It was also added after a review of the results. And the re-check does not repair: recall stays where the broken filter left it, and the layer's report of an incomplete result is what tells an operator something is wrong.

M.4 What the model did with what it retrieved

Prompt injection in readable content is outside our threat model (Section 2.3), but the runs measure where the boundary lies. Each tenant's injected notice is readable by that tenant's employees and contractors, so retrieving it for them is correct. Over all eight runs the notice was retrieved between 1 and 117 times per configuration and run, depending on the store's ranking, and the harness counted its marker string in the 3B model's answer the same number of times: 3,239 of 3,239. This count of "obeyed" is the marker anywhere in an answer, and the notice's own text holds the marker, so an answer that repeats the notice counts too. In the stored answers of all eight runs the marker appeared in exactly the answers whose top five held the notice. Retrieval-time enforcement decides which content reaches the model, not what the model does with it; defenses against injected instructions in permitted content [41, 59] are complementary.

Answer quality followed retrieval quality. With FAISS the model found the target document of a targeted question in its top five for every question in every run and answered 98.4% to 99.7% correctly (an answer is correct when it quotes the fact from the right tenant's document); with Qdrant the target was in the top five for 33.8% to 39.6% of targeted questions. With OpenFGA, despite its lower recall, the target was in the top five for 94.2% to 97.9%. When a person asked about a document they could not read, the model abstained in 68% to 83% of cases, depending on the store and run, and otherwise answered from readable chunks only, as the canary checks confirm.

N Related work in full

This appendix gives the full account behind Section 10. Our statements that we found no prior work report searches, which are not exhaustive.

Enterprise search and document-level security. Büttcher and Clarke gave the founding security model for multi-user full-text search [23]. Filtering results ranked with global BM25 statistics lets any user who can create files infer how many unreadable files contain a term (about 0.5% in their example, from ranks alone); their fix, query integration, restricts every posting list to the user's searchable set before ranking, at between 39% faster and 17% slower, and they require that no aspect of the output, including its order, depend on unsearchable content. Bailey, Hawking and Matson modeled late binding, whose cost grows with the results checked times the per-check latency divided by the visibility density VD (a 6.4 ms off-server check limits a one-second budget to 93 checked results), and fetched blocks of $2\,NR/\,VD$ candidates until a page of NR results filled [18]. Google's Search Appliance team named early and late binding and warned that ACLs copied into an index drift out of sync and expose content [87]. Fagin et al. studied search over a corporate intranet [50], the setting in which these documents and their readers live. The pre-filter we test is query integration applied to approximate nearest-neighbor (ANN) search, and its widening post-filter is Bailey et al.'s loop; we claim neither. We use Büttcher and Clarke's consistency definition to frame side channels of filtered vector search. Their score channel has since been exploited and extended: Wang et al.'s STRESS attack discovers terms in other tenants' documents and counts documents holding a term, on shared indexes in Elasticsearch and on services such as GitHub, with rank-only variants and proposed countermeasures [165], and Espiritu and Cash reach indexed terms and approximate text through document-level security in Elasticsearch and OpenSearch [49]. The nearest peer-reviewed work to our side-channel measurement is therefore on keyword search and is stronger than ours there. We found no peer-reviewed measurement of these channels for filtered vector search, though one open-source project reports them in process [117]. A separate line protects the index from its server. Zerber bounds what an index reveals about terms under statistical attacks, with limited trust in the central indexer [183], and Zerber+R adds top-k ranking over such an index [184]; searchable encryption protects keyword search [36, 149], and secure kNN protects similarity search over encrypted vectors [174]. Those works assume an untrusted server; ours trusts the store's host (Section 2.3) and asks how one layer's query path behaves.

Database access control. System R gave relational databases discretionary GRANT and REVOKE [60], and Motro derived authorizations by algebraic manipulation of view definitions [99]. Stonebraker and Wong enforced access rules by query modification, conjoining the applicable rules' qualifications with the user's query [150], which is what a pushed-down permission filter does. Rizvi et al. separated Truman semantics, which silently answer over the user's restricted view as Oracle's Virtual Private Database does, from non-Truman semantics, which reject queries that authorized data cannot answer [115, 137]. Wang et al. defined sound, secure and maximum answers, with security as indistinguishability across database states the policy cannot tell apart; they showed that the cell-level disclosure control of Hippocratic databases [76] violates soundness and the maximum property, noted that the straightforward extension of authorization-view query modification to queries with negation is not sound, and gave a sound and secure algorithm for cell-level policies, with the proof of one theorem left to a technical report [167]; Kabra et al. showed that functions evaluated before the authorization predicate reveal data through their values and error messages [71]; and optimal security-aware query processing is impossible for the full relational calculus [61]. Sieve enforces large numbers of fine-grained policies from a middleware above the database [119]. PostgreSQL row-level security brings these lessons to production with default deny,

leakproof functions and documented covert channels through key checks [157], and its timing has been attacked, first to test whether a value exists [38, 129] and then to recover records [49]. Of such native enforcement we report only a side run of its recall and latency on another corpus (Appendix O.1), not its channels. The layer we test is query modification with Truman semantics, and its re-check deliberately keeps the redundancy that Kabra et al. remove, because its filter and its check come from different code paths. This lineage gives criteria and, for cell-level policies under exact relational evaluation, a proved algorithm, not a general proof that rewriting is correct. We adapt its soundness and maximality to top-k retrieval and weaken its security (Section 2), and test translators and approximate indexes, for which we found no such proofs, where maximality can fail even when the predicate is right.

Authorization systems and policy-to-filter compilation. Zanzibar stores more than two trillion ACL tuples and answers millions of checks per second at p95 latency under 10 ms. Answering a search query there requires ACL checks for all candidate results, often tens to hundreds of checks per set of results, and its Expand API lets clients build search indexes for access-controlled content; snapshot tokens prevent the "new enemy" problem of applying an old ACL to new content [118]. Its open-source descendants, built on relationship-based access control [53], added reverse lookups with caveats. OpenFGA recommends ListObjects for about 1,000 accessible objects and by default stops a non-streaming call at 1,000 results or 3 seconds [106, 110]; SpiceDB recommends LookupResources below about 10,000 resources and otherwise a bulk-check loop pinned to one revision [13], and CVE-2025-65111 is a case in which, for one schema shape before version 1.47.1, LookupResources missed results while its other APIs computed permissions correctly [102]. A truncated allow-list is safe but silently loses recall. Policy engines compile filters through engineering artifacts: OPA's partial evaluation, available since 2018, emits SQL conditions for a documented fragment of its language [104, 105, 140], and Cerbos's query plans return always-allowed, always-denied or a conditional syntax tree [28]. The peer-reviewed Cedar paper does not discuss filtering [37], and XACML 3.0 returns one decision per request [103]. Policies and engines themselves have been verified and tested: Margrave checks properties of access-control policies and the impact of changing them [52], Zelkova reasons about AWS access policies with an SMT solver [17], mutation testing targets access-control policies [88], and Cedar's production code is differentially tested against a model with machine-checked proofs [42]. These address the policy and its check path. The nearest result we found to a verified filter is the Lean theorem `reauthorize_is_sound` for Cedar's type-aware partial evaluator, in its public specification [27]: re-authorizing a residual with concrete inputs equals direct authorization. It concerns residuals, not a translation into a store's filter language; we found no peer-reviewed report of it; and the layer's Cedar adapter does not use the evaluator (Appendix D.1). We found no peer-reviewed semantics or correctness proof for any policy-to-filter translator. We propose no authorizer or filter API. We measure, at one revision token, how often each engine's filters disagree with its own check API on one generated model and when its list APIs truncate (Section 5), and rely on the re-check so that such faults cost recall, not membership in the readable set.

Filtered ANN search and access-aware vector indexes. Graph indexes such as HNSW [86] answer filtered queries by filtering before, after or during traversal. With pass fraction p, a post-filter needs about k/p candidates when passing items are spread evenly and $O(n)$ when they lie far from the query [120], and the collapse is well measured: post-filtering stayed below 0.9 recall on LAION-1M even with positively correlated filters [120], Faiss-HNSW with post-filtering and ACORN often failed at 0.1% selectivity [79], and without a fallback ACORN-1's recall falls to 0.03–0.10 at 0.3% pass fraction [73]. pgvector filters after the index scan, so a filter matching 10% of rows yields about four matches at the default search depth [121]. Index designs for attribute-filtered search

include proximity graphs that fuse vectors and attributes [166, 176], space partitions [62], graphs that navigate label sets [24], workload-aware collections of indexes [80] and structures for range filters [48, 179, 192], and systems combine vector and relational predicates in one engine [173, 187]; all take the predicate as part of the query rather than compile it from an external policy. Engines now fall back to exact scoring for restrictive filters [4, 138, 161]; since version 3.5, OpenSearch's Faiss engine does so when efficient-filtered search returns fewer than k results while at least k documents pass, unless a setting disables it [112]. Filtered-DiskANN cites user privileges as motivation but evaluates single-label filters [57]; the NeurIPS'23 Big-ANN filtered track uses conjunctions of one or two tags without analyzing correlation [145]; later benchmarks sweep selectivity down to 0.1% [66, 79, 144] and control or measure query-filter correlation [2, 83, 188]; and HCBGen shows that neither selectivity nor correlation alone is a stable proxy for difficulty [81]. None of those we read uses real ACLs, and neither do we. Permission-specific indexes are recent and assume role-based access control. HoneyBee partitions by role structure, with up to 13.5× lower latency than PostgreSQL row-level security at 1.24× the memory, on synthetic permissions and with a cost model that ignores query-permission correlation [189]; Veda builds an access-aware lattice over 641 to 757 role combinations [63]; and the SeQureDB '26 vision paper finds on pgvector that pre-filtering gives the best recall and naive post-filtering near-zero recall for selective policies [180]. Curator indexes tenants without a security model [69], and a recent preprint links per-tenant recall loss partly to the local density of a tenant's points [39]. We propose no index, and claim neither the first permission-aware vector search, nor the post-filter collapse, nor alignment as a new axis, which we measure instead. We found no evaluation of per-query disjunctions over hundreds or thousands of group labels, the shape of a directory-group ACL, and no audit of production stores under such filters at documented, tuned and exact settings (Sections 6 and 7). HoneyBee and Veda are physical designs on which an enforcement layer could run.

RAG privacy and agent identity. Anyone who can query a RAG system can drain its store. Composite prompts extract private retrieval data, and re-ranking barely helps [182]; prompt injection leaked the datastores of 25 of 25 custom GPTs with at most two queries each [128]; interrogation attacks infer membership with about 30 natural-looking queries [100]; and in a survey's testbed, an 8-gram output guard cut verbatim copying to 0.0% while an adaptive three-query attacker still recovered 48.3% of hidden facts [82]. Such defenses are probabilistic and model-side. Cross-tenant work reaches our premise: retrieval ranks by relevance, not authorization, and attribute-based gating at retrieval [5] and a unified PostgreSQL data layer [22] are reported to remove cross-tenant leakage; re-checking authorization at the graph-expansion boundary of hybrid RAG cut a retrieval-pivot risk of 0.95 to near zero [160]; Microsoft researchers call prompt sanitization and output filtering fundamentally probabilistic [20]; GPT-4.1 reaches only $F_1 = 0.27$ on OrgAccess's hardest set, so prompts cannot carry permissions [141]; and OWASP's LLM08:2025 makes permission-aware vector and embedding stores its first prevention measure [116]. PermLLM enforces access control in model parameters instead [68]; ConfusedPilot showed revoked documents staying retrievable through a cached index [139]; and embedding inversion [78, 98, 148, 185] leaves storage protection to the deployment. For agents, the confused deputy [65] has a standards answer: RFC 8693's actor claim names the delegated agent while the subject's rights govern access [70], the MCP authorization specification requires audience-bound tokens and forbids passing a client's token upstream [95], and MITRE ATLAS mitigation AML.M0027 says an agent should never hold permissions its user would not [94]; yet 40.55% of 7,973 live remote MCP servers exposed tools without authentication [190]. This literature shows why enforcement must be deterministic and precede generation. Where it evaluates a layer it does so on one system, and we found no oracle-based test across stores and engines. Corpus poisoning [191] acts through content a reader may read, and information-flow

control for agents [33, 175] constrains where retrieved content may flow after it is read; both are complementary to a retrieval guarantee. We adopt token-bound identity rather than claim it, exclude extraction within the readable set and injection in permitted content (Section 2.3), and take the timing channel that SpliceLeak made practical in RAG infrastructure [152] as one motivation to measure side channels of filtered retrieval.

Side channels and information flow. Noninterference requires that what a high-privilege user does has no effect on what a low-privilege user observes [56]. The distinguishing game of Section 8.1 tests a restricted form of it, over chosen observables and chosen changes to unreadable content. Timing has long revealed hidden content on the web: load times revealed a browser's cache and so its history [51], and the response times of search endpoints let a cross-site attacker extract private data [54]. Black-box mitigation delays outputs to a predicted schedule [6], and language-based work combines such mitigation with information-flow control [186]; the padding we measure is the simplest such schedule, a fixed budget. We propose no mitigation beyond these. For keyword search the attacks cited under enterprise search above are stronger than our game, and their countermeasures, per-tenant indexes and document frequencies from public data, are the ones to read. What we add is a forced-choice measurement, through one layer, of how much each enforcement path of filtered vector search reveals, and of what removing it costs.

Testing methodology. Database engines have mature oracles for logic bugs: Pivoted Query Synthesis found 121 bugs in SQLite, MySQL and PostgreSQL [136]; NoREC and Ternary Logic Partitioning compare a query with a non-optimizing or partitioned rewrite of itself [134, 135]; and later oracles reach query plans and graph databases [15, 16, 72]. All descend from differential [89] and metamorphic [30, 143] testing, often with property-based input generation [32, 84]. For the filter semantics of vector databases we found no such oracles: a study of 1,671 bug-fix pull requests in 15 vector databases found 57.3% of bugs to be functional failures, with filter abnormality as a fault class, but does not mention differential or metamorphic testing [178], and a testing roadmap lists oracle definition as an open challenge and reports only simplified metamorphic cases [168]. The Secret Sharer inserted canaries to estimate unintended memorization [25]. In reporting, ANN work plots recall-throughput frontiers [7], yet the Big-ANN results paper and HoneyBee give no error bars [145, 189], short of the practice recommended for ML benchmarks [1, 21]; fuzzing research has similar reporting norms for repeated campaigns and stated seeds [74, 142], and IR evaluation has compared the significance tests available for paired comparisons [147, 162]. We found no differential or metamorphic testing of metadata-filter semantics across vector stores. Ours is differential in McKeeman's sense: every store's translation is compared with one reference implementation, not with a rewriting of the same query as in NoREC's metamorphic relation, and it evaluates its relations under exact search, so that approximation cannot confound them (Section 4). We also turn canaries into a deterministic enforcement check over hits, prompts and answers, report zero-event bounds by the rule of three [64], and report confidence intervals over repeated runs.

Industry systems. Among native store features, Azure AI Search (in preview) goes furthest: it builds filters from the caller's Entra token, expands groups through Microsoft Graph at query time, and returns a server error rather than a partially filtered result when evaluation fails. Yet its document ACLs are only as fresh as the last synchronization, and earlier preview APIs returned all ACL-protected documents to callers who used an API key without a user token, until November 2025 [91]. Elasticsearch and OpenSearch enforce document-level security through per-role queries [44, 111]. Elsewhere, per-document permissions reduce to filters the application supplies: Milvus supports no RBAC at the partition-key level [92], and Qdrant removed payload filters from its token-based RBAC in version 1.16, citing inconsistent read and write semantics [124]. Authorization vendors

ship one integration per engine, as a post-filter [8], a pre-filter [29] or both [11]; none widens until k authorized hits arrive or re-checks pre-filtered hits, and OpenFGA acknowledges that post-filtering may return fewer than k results and suggests over-fetching two to three times [109]. Enterprise assistants mirror ACLs into their indexes, so revocation waits for the next connector synchronization, daily for Coveo's Dynamics 365 provider [35]. The closest commercial analogue to the layer we test is Amazon Bedrock's managed knowledge-base ACL awareness: it pre-filters on crawled ACLs, verifies each returned document against the data source in real time for the connectors that support it (not S3 or custom sources), lets denials override grants and withholds documents on transient errors, while stating that the feature is not a security boundary [3]. Open-source projects cover several stores with metadata only [75], or one store with self-reported tests [55]. We claim no novelty for pre-filtering with a live re-check and failing closed, which Bedrock ships. We found no independent evaluation of permission correctness, and no published recall evaluation, for any of these products, and no project that combines many vector stores with many authorization engines; the latter is an engineering property of our harness, not a research claim, and our measurements of one layer do not stand in for an evaluation of these products.

O Limitations in detail

Scope of the security claim. Our finding is that no returned chunk lay outside the readable set at the revision the check read, under the trusted computing base of Section 2.2. It does not mean fresh: a revocation takes up to the δ of its path, which Section 5.1 measures, and the code we ran exceeded its stated δ after an invalidation. It does not mean that unreadable chunks leave no trace: in a forced-choice game, latency, counts, recall and scores reveal them (Section 8), and the detection fields of Section 7 (*missed*, *exhausted*, *over_limit*) are further caller-visible observables that we did not measure. An empty answer satisfies the property, and in the end-to-end runs the re-check dropped nothing. A store that binds content to the wrong identifier or metadata defeats any re-check, and we test that assumption only as far as the conformance kit's metadata checks reach. The trusted computing base has failed before: earlier versions of the layer's SpiceDB and Cedar adapters let the check itself admit (Table 7), so the claim covers the code we evaluated, not its history; we give no measure of its size. The completeness flag is count-honest relative to the pushed-down filter: it detects under-retrieval but not mis-retrieval, and a full answer with the wrong chunks still reports complete.

What was tested. We tested one layer, which the author develops and whose public repository was created in October 2026. We report no deployment of it, compare no other layer or managed product, and ran no adaptive adversary. Nobody outside this project has rerun anything. The engine study uses one generated authorization model per profile, and negation in a filter is untested. Stores and engines are not crossed: the engines ran on FAISS in the engine study, and on pgvector end to end (OPA and OpenFGA). The observation that security does not depend on the filter is an argument; in the end-to-end runs the re-check dropped nothing, and the revision a check read cannot matter there since permissions did not change, while OpenFGA's read revision is not recorded at all.

Synthetic permissions. No public corpus pairs documents with the access-control lists of a real organization, so every permission layout here is generated: tenants, groups and per-document ACLs drawn from templates and seeded generators, over public passages (FiQA, MS MARCO) and templated company documents. Real organizations may have more skewed layouts, with many small groups, deep group hierarchies and frequent exceptions. We place every (question, subject) pair on measured correlation axes instead of trusting a layout's label (Section 2.4), which makes

results comparable across layouts of similar measured difficulty, although no single axis described every store (Section 6.5). The MS MARCO experiments vary layouts across seeds; the end-to-end runs keep one layout and vary only documents and part of the questions, so their people recur with the same permissions. That one layout is harsh in a way we state and did not vary: each company document has 19 near-identical copies in other tenants that only the tenant rule separates, a hard case for graph indexes, so its recall numbers, Qdrant's especially, should not be read as an estimate for an organization without such copies.

One instrument. Every measurement passes through one enforcement layer, built for this project, so a fault in its filter translators could be mistaken for a store fault. The conformance kit compares every answer with an in-memory reference that shares no code with the translators, and the retrieval audit uses each store's own exact search as a second baseline. Some numbers are properties of the layer's schema as much as of a store: pgvector's filters are `jsonb` containment tests over a GIN index, with an array test above 100 values that the index cannot serve, and Chroma's list values are boolean keys, so its disjunctions hold one clause per group. Results describe each store under this layer's schema, not at its best possible schema. Of the third-party behaviors our experiments found, four come with a reproducer that uses only the vendor's own client: the truncation in Redis and in MongoDB, Chroma's crash, and the CEL semantics that the layer's Cerbos translator meets. Each showed the reported behavior when run on GitHub-hosted runners on 30 October 2026 against the version we evaluated and the vendor's latest release (Redis 8.10.2 and 8.10.3, MongoDB Atlas Local 8.0.30 and 8.3.13, Cerbos 0.56.0 and 0.57.0, and Chroma 1.5.9, still its latest). Running them corrected one figure: MongoDB's vector-search filter fields keep 8,191 characters, not the 8,181 we had taken from its documentation of another field type. SpiceDB's staleness, OPA's residual negation and pgvector's plan choice were measured through the layer and have no reproducer independent of it; the other rows of Table 7 rest on the vendors' documentation, which the table cites.

Versions, settings and hardware. No single version of the library was evaluated throughout. The end-to-end runs used release 2.1.0 from PyPI, with its known defects; the engine study ran a branch labeled 2.1.0, and other experiments ran from their own branches, several carrying fixes that are not yet in a release, such as the engine rerun and the repair experiment (Appendix A). The fixes were tested piecemeal and have not run end to end, and we did not rerun the whole evaluation on one release. The first end-to-end run and its repeats also differ in the Redis, lancedb and weaviate-client versions (Appendix M.1). The library commit of each experiment is recorded with its outputs. Results hold for the versions and settings we state (for example pgvector 0.8.7 and Qdrant 1.19.2) and may not survive the next release; vector stores change quickly, and several behaviors we measure are documented defaults that vendors may revise. PostgreSQL was 16.15 in the end-to-end runs and version 16 in the diagnosis grid, and 17.11 in the campaign, the engine study and the frontier, repair and filter-size experiments. Embeddings were 768-dimensional (bge-base-en-v1.5) on the enterprise corpus and 384-dimensional on MS MARCO (bge-small-en-v1.5) and in the filter-size and side-channel experiments (all-MiniLM-L6-v2); dimension matters, for example, to the point count at which Qdrant plans an exact scan. We did not record Lucene's segment counts for Elasticsearch and OpenSearch. Experiments ran on shared cloud hardware (Kaggle sessions with two T4 GPUs and four CPUs, and GitHub-hosted runners), so latencies are comparable within an experiment, not across experiments or with vendor benchmarks.

Baselines and sweeps we did not run. The evaluation compares the enforcement paths of one layer. The only comparison with a store's native enforcement is the side run below, and we did not measure the document-level security of Elasticsearch and OpenSearch, which a search of the

library's benchmark branches confirms. We did not sweep the share of near-duplicate chunks, so we cannot say how fast Qdrant's loss falls as copies across tenants become fewer. Outside the 100% selectivity cells on MS MARCO we have no unfiltered baseline per store, so part of a store's loss under a filter may be its ordinary approximation error, as with LanceDB's quantized index.

Statistical power. The end-to-end scenario ran four times with fresh documents and partly fresh questions, and four more times for the OpenFGA configuration, but always with one permission layout; its people recur in every run, so its bound per person is a single run's, and they are 13 role types. Its bounds treat those cases as independent, which is only partly true, are over the question generator and say nothing about an adaptive adversary. The diagnosis grid ran three index builds per layout, the frontier and repair experiments three builds per store and corpus over ten MS MARCO layouts and the enterprise layout, and the engine, filter-size and side-channel experiments five seeds each; intervals come from bootstrap resampling over people, subjects or probes, not over builds, so they are conditional on the builds, and a store whose recall varies between builds (Qdrant's by up to 0.13) is described by the mean of three. People and subjects are themselves not independent where they share a role or a permission shape. Zero-leak results are reported per question, per subject and per fault episode, not per hit. We apply no correction for multiple comparisons across cells, and in the side-channel experiment two builds of identical content can differ by as much as a weak channel (Section 8.2). Several pre-registered verdicts are inconclusive rather than confirmed or falsified, and we report them as such.

O.1 Native enforcement: a side run on FiQA

The library's benchmarks compared its pre-filter with PostgreSQL row-level security on 2026-10-22, before the experiments of this paper, at commit 42fcca6 (`https://github.com/muzakkirhussain011/wardowl/blob/main/benchmarks/results/fiqa-row-level-security.md`). The corpus is BEIR's FiQA, 57,600 non-empty chunks with 200 questions, embedded with Qwen3-Embedding-0.6B (1,024 dimensions), under layouts of 1,000 teams, with the same table and HNSW index for both (iterative scans on); the benchmark's full results file records PostgreSQL 16.15 and pgvector 0.8.7. Row-level security enforces the permissions inside PostgreSQL with a policy on the groups in the metadata, a GIN index on the groups, and searches run as a role without `BYPASSRLS`. Table 20 gives the results. Leaks were zero in every row, and each row is 200 queries.

The plans were not captured. Recall near 1.0 at every selectivity, above the HNSW index's own 0.978 when everything is readable, and latency that grows with the readable rows, are what an exact scan of the admitted rows would produce, not a walk of the index. This is a side run: an older commit, under the library's earlier name, another corpus without tenant copies, another embedding, another harness. It does not say what row-level security would do on the enterprise corpus, and it measured no side channels; timing attacks on PostgreSQL's row-level security are known [38, 49, 129]. It does show a native baseline that lost almost nothing on that corpus, at up to about 600 ms, against 0.948 to 1.000 for the layer's pre-filter on the same table, so the recall loss of Section 6 is not a cost of enforcing permissions as such.

O.2 Licenses of data and models

We read each license on its official page on 30 October 2026. MS MARCO's terms state that the datasets are intended for non-commercial research purposes only. The FiQA site makes its Task 2 data, which BEIR redistributes, available only for non-commercial use; BEIR's dataset cards for FiQA and SciDocs carry a CC BY-SA 4.0 tag, and BEIR's documentation leaves it to users to determine whether they may use a dataset. The SciDocs repository's license file is CC BY

		Layer's pre-filter		Row-level security			
Layout	Readable	Recall@10	p50 (ms)	Recall@10	Ten results	p50 (ms)	p95 (ms)
random	0.10%	1.000	4.2	1.000	100%	3.4	4.3
random	1.00%	1.000	11.3	1.000	100%	10.8	13.2
random	5.01%	0.974	14.1	1.000	100%	42.6	48.4
random	20.00%	0.965	6.6	0.998	100%	164.7	178.0
random	100.00%	0.978	5.3	0.998	100%	608.1	638.3
aligned	0.14%	0.999	3.4	0.999	99%	3.3	4.8
aligned	1.08%	0.998	11.5	0.999	100%	10.6	15.9
aligned	5.21%	0.975	5.8	0.999	100%	41.5	51.8
aligned	20.36%	0.971	4.7	0.998	100%	155.1	168.3
aligned	100.00%	0.978	5.3	0.998	100%	602.1	635.4
misaligned	0.06%	1.000	2.8	1.000	92%	2.1	3.3
misaligned	0.92%	0.999	9.1	0.999	100%	8.6	14.0
misaligned	4.78%	0.959	82.2	1.000	100%	37.4	48.3
misaligned	19.89%	0.948	11.1	0.998	100%	165.4	182.0
misaligned	100.00%	0.978	6.0	0.998	100%	595.4	627.3

Table 20: Row-level security against the layer's pre-filter on FiQA, as the library's results file reports them. *Ten results*: the share of the 200 queries that returned ten. Not part of this paper's pre-registered experiments.

4.0. The embedding models bge-base-en-v1.5 and bge-small-en-v1.5 are under the MIT license and all-MiniLM-L6-v2 under Apache 2.0. Qwen2.5-3B-Instruct is under the Qwen Research License Agreement, which permits research and evaluation and requires a separate license for commercial use. All our use was research. The harnesses download corpora and models from their sources; released outputs that quote passages or model answers should be treated under the same terms.

O.3 Ethics and disclosure in detail

No human subjects took part. People, documents, ACLs and canaries are synthetic, and the shared passages come from public research corpora [19, 85]. Red-team prompts are generic attempts to obtain other people's data and target only our own deployments; every store and engine ran in containers or processes we controlled, on our own sessions and runners, and no third-party service was probed.

None of our findings is a chunk returned through the layer outside the readable set at the checked revision, because the re-check contains filter faults. They can still matter for deployments that trust a pushed-down filter without a re-check: a filter that admits more than the check would leak there, and one, Chroma's crash on a large filter, lets any user in enough groups stop an application that passes their groups to Chroma in local mode. For four behaviors in third-party components that our experiments found (the truncation in Redis and in MongoDB, Chroma's crash and the CEL semantics that the Cerbos translator meets), we sent a report with a minimal reproducer that does not need our library, on 2026-10-30, after our public pull requests had described the behaviors (Section 11), and Table 7 gives each report's status at the time of writing. One unexplained SpiceDB trial past its documented bound, OPA's residual negation and pgvector's plan choice have no report.

P Store settings in the enterprise scenario

Ground truth. The exact baseline multiplies all question vectors with all chunk vectors, masks chunks the oracle says the person may not read, and takes the top ten. FAISS over a flat index reproduces it at recall 1.000, which validates the pipeline. A hit counts toward recall if it is readable

Store	Version	Index and search settings
FAISS	faiss-cpu 1.15.1	`IndexFlatIP` (exact); metadata matched in Python
pgvector	pgvector 0.8.7, PostgreSQL 16.15	HNSW built after loading with default $m = 16$, `ef_construction` 64; `hnsw.ef_search` 100; iterative scan `relaxed_order`; `hnsw.max_scan_tuples` 20,000 and `scan_mem_multiplier` 1 (defaults); `work_mem` 4 MB; metadata in `jsonb` with a GIN `jsonb_path_ops` index; filters as `@>` containment tests
Qdrant	1.19.2	HNSW cosine, $m = 16$, `ef_construct` 100, full-scan threshold 10,000 KB (defaults); keyword payload indexes on users, groups, tenant and identifier, created before loading, none marked `is_tenant`; `hnsw_ef` 128; no ACORN, not exact
Elasticsearch	9.5.5	`int8_hnsw`, `num_candidates` 500
OpenSearch	3.9.0	Lucene HNSW, `num_candidates` 500
Weaviate	1.39.12	HNSW (defaults), indexed while importing
Redis	8.10.3	HNSW, `EF_RUNTIME` 100
LanceDB	lancedb 0.41.0	`IVF_PQ`, `nprobes` 50, `refine_factor` 30
Chroma	1.5.9	HNSW cosine (defaults)

Table 21: Settings in the end-to-end runs (Sections 9 and 6). Each run is one Kaggle session, two T4 GPUs, four CPUs, 31.3 GiB of memory; embeddings bge-base-en-v1.5, 768 dimensions, unit length. The setup scripts pin the server versions of Qdrant, Elasticsearch, OpenSearch, Weaviate, PostgreSQL's pgvector, OPA and OpenFGA; Redis comes from its package repository and the embedded stores from PyPI; the logs record the same versions in all four runs, except psycopg (3.3.8 in the seed-7 run, 3.3.6 in the repeats).

and its similarity, recomputed from the run's vectors, is at least the k-th exact score minus 10^{-4}. Recall@5 and recall@10 come from separate retrievals with $k = 5$ and $k = 10$. People who may read nothing are excluded.

Q Compute

Experiment	Platform	Wall clock	Hours
End-to-end, seed 7	Kaggle, 2×T4, 4 CPUs	2.8 h	2.8
End-to-end, three repeats (X8)	Kaggle, 2×T4, 4 CPUs, one session each	2.9 to 3.3 h each	9.2
End-to-end, OpenFGA, four runs	Kaggle, 2×T4, 4 CPUs, one session each	0.6 to 0.9 h each	3.3
End-to-end, exposure, four seeds	Kaggle, 2×T4, 4 CPUs, one session each	0.1 h each	0.5
End-to-end, mutation control	Kaggle, 2×T4, 4 CPUs	2.2 h	2.2
Diagnosis grid, run 1 (X1)	Kaggle, 2×T4, 4 CPUs	7.3 h	7.3
Diagnosis grid, run 2 (X1)	Kaggle: 1 GPU session, 4 CPU sessions	6.1 h	12.5
Conformance campaign, run 1	GitHub-hosted runners, 7 jobs	2.9 h	6.6
Conformance campaign, run 2	GitHub-hosted runners, 10 jobs	2.1 h	8.2
Engines (X4, X7)	GitHub-hosted runners, 15 jobs	2.0 h	15.3
Revocation and faults, rerun (X7)	GitHub-hosted runners, 8 jobs	1.5 h	1.5
Filter size and side channels (X3, X6)	GitHub-hosted runners, 15 jobs	3.3 h	10.2
Their follow-up	GitHub-hosted runners, 11 jobs	2.2 h	7.4
Frontiers and repair (X2, X5)	GitHub-hosted runners, 72 jobs	10.6 h	98.7

Table 22: Compute of the paper's experiments; the row-level security side run (4,331 s on Kaggle) and the vendor reproducers' short runs are not listed. Kaggle sessions have 4 CPUs and 31.3 GiB of memory; GitHub-hosted runners have 4 virtual CPUs and 16 GB. Jobs counts every job of the workflow run, including data, plan, report, summary and publish jobs, from GitHub's jobs API (the X7 rerun and the follow-up each include jobs the workflow skipped, one and three). On GitHub, hours sum the run times of the jobs that started; wall clock includes queueing. On Kaggle, hours are session hours, the last time stamp of each kernel log. Smoke runs that tested the harnesses, whose numbers were never analysed, and runs cancelled before they produced results are not listed.